The modern Transformer is a small set of choices: convergent architecture in frontier LLMs, 2017–2026

Theory KB — Paper 1 of 5

2026-05-04 (draft)

Abstract

We argue that frontier LLMs in 2026 are converging on a small, identifiable set of architectural choices along a few orthogonal axes: KV-sharing (MHA → MQA → GQA → MLA), attention implementation (FA1 → FA2 → FA3, NSA), feed-forward style (dense → fine-grained MoE with shared experts), normalization (LN → RMSNorm, Post-LN → Pre-LN with $1/\sqrt{N}$ residual init), positional encoding (sinusoidal → RoPE/YaRN), tokenization (BPE → SentencePiece byte-level), and multi-token output (speculative decoding → MTP). For each axis we formalize the canonical 2017 baseline, trace the choices made between 2017 and 2026, and identify the localized open frontiers — MLA-vs-MHA at scale, NSA-vs-FA3, SSM-hybrids at frontier — where the convergence is still contested.

Contents

1 Introduction

The Transformer is not one architecture; it is a family parameterized by a small set of orthogonal choices. From the 2017 encoder-decoder of Vaswani et al. (2017) through the GPT-2 decoder-only collapse (Radford et al., 2019), the Chinchilla-era dense scaling regime (Hoffmann and et al. , DeepMind), and the 2024-2026 frontier models typified by DeepSeek-V2 and V3 (DeepSeek-AI, 2024a,b), the field has converged on a recognisable skeleton – a stack of identical blocks alternating

an attention sublayer and a feed-forward sublayer over a residual stream of shape $\mathsf{B} \times \mathsf{S} \times \mathsf{D}$ – while exploring each sublayer along an independent axis. **The thesis of this paper is that those axes are few, locally separable, and now well enough understood to write down. Naming them is a precondition for saying anything precise about the open architectural questions of 2026.**

We organise the architecture along ten axes:

1. *Tokenization, embedding, and unembedding* — the discrete-token interface, including subword choice and the tied-versus-untied I/O decision (§3).
2. *Positional encoding* — the mechanism that breaks self-attention's permutation symmetry, with a clean lineage from sinusoidal through rotary (§4).
3. *KV-sharing* — how the H attention heads share their key and value projections; a single axis with four named waypoints (§5).
4. *Attention implementation* — holding the scaled-dot-product formula fixed and varying the GPU schedule (§6).
5. *Feed-forward block* — the activation/gating structure of a single FFN and the dense-vs-MoE multiplicity (§7).
6. *Normalization* — layer norm versus RMS norm, and where in the residual stream the norm sits (§8).
7. *State-space alternative* — the SSM and SSM-attention hybrid path that does *not* use scaled dot-product attention as the per-token mixer (§9).
8. *Multi-token output* — the architectural and inference-time choices that break the one-forward-pass-per-token coupling (§10).
9. *Multimodal extension* — how non-text inputs reach the residual stream (§11).
10. *Inference adjuncts* — the KV-cache lifecycle, quantization regime, and constrained-decoding apparatus that constrain which choices the prior axes can take in production (§12).

The axes are orthogonal in the precise sense that, at the math layer, a frontier model is specified by an *independent* choice on each of (1)-(10) plus a small number of scale dials (L, D, H, d_h, V, E for MoE). What this paper documents is that 2026 frontier models are converging on a small neighbourhood in this product space.

Intuition

The picture is a small parameter space, not a survey. A frontier 2026 model is a point in a product $\Pi_{\text{axis-1}} \times \cdots \times \Pi_{\text{axis-10}}$, where each factor $\Pi_{\text{axis-}k}$ is a discrete set with at most five reasonable choices. Returning to canonical math: the residual stream stays $\mathsf{B} \times \mathsf{S} \times \mathsf{D}$ throughout, the attention sublayer's per-head workspace stays $\mathsf{B} \times \mathsf{H} \times \mathsf{S} \times d_h$, and the LM head still maps to logits of shape $\mathsf{B} \times \mathsf{S} \times \mathsf{V}$. What changes per axis is *which projection, parameterisation, or schedule* fills each box. The convergence claim of §13 is the claim that the realised set of axis-tuples in 2026 production models is a small subset of the combinatorial space.

The convergence story is concrete enough to be falsified. Across the axes above, there is now a recognisable *frontier preset*: SentencePiece or tiktoken-family BPE with V in the 128k–256k range and untied I/O embeddings; RoPE at every layer with one of a small family of long-context extension schemes; GQA or MLA on the attention sublayer with the FlashAttention-3 kernel as the default schedule; SwiGLU as the per-expert FFN, dense or sparse-MoE depending on scale; RMSNorm in a Pre-LN (often Peri-LN) placement with $1/\sqrt{N}$ residual scaling at initialisation; and either single-token output with speculative decoding bolted on, or DeepSeek-V3-style multi-token-prediction heads. We document each axis with the canonical equation transcribed verbatim and the lineage cited to primary sources, then in §13 cross-tabulate the frontier models on the same axes to make the convergence visible in a single figure.

The open question this paper does and does not answer

A natural objection – which we take seriously, and which any synthesis paper of this kind must – is that the convergence we document might be *model-zoo bias*: an artefact of the small set of frontier labs whose architectures are publicly disclosed. Every ground-truth axis tuple in our table comes from a paper or technical report; Anthropic, OpenAI, and xAI have not published architecture details for their frontier models, and the list of disclosed architectures is dominated by Meta, DeepSeek, Mistral, Qwen, and Google open-weight releases. We address this question directly in §13; we do not preview the answer here. We do flag that the question is the load-bearing one for the paper's empirical claim, not a quibble.

What this paper is and is not

What it is. A monograph on architectural convergence in frontier LLMs from 2017 to 2026, organised by axis rather than by year or by lab. Every load-bearing technical claim is cited to a primary source (paper or technical report); every equation is transcribed verbatim from the cited paper, in the tensor-shape notation fixed in §2; and every unresolved disagreement between sources is tagged `[CONTRADICTION]` and gathered for direct discussion in §14. The shape of the writing is dense rather than discursive: the goal is that a reader who already knows what an attention block does can locate, in one place, the canonical math for each axis at each waypoint and a sourced statement of where the field sits in 2026.

What it is not. A tutorial, a survey, or a complete catalogue. Pre-2017 deep-learning history is sketched in §2.1 only as far as needed to motivate the 2017 canonical; we cite Vaswani et al. (2017) for the ConvSeq2Seq comparison and proceed. Training, post-training, reasoning, interpretability, and evaluation are deferred to Papers 2–5 of this series; this paper is strictly architectural, with the (deliberate) exception of §12, which folds inference-time concerns in because they constrain which architectural choices the prior axes can take.

Roadmap

§2 is the foundation: it sketches the pre-2017 lineage that the Transformer replaced (RNN seq2seq, additive attention, ConvSeq2Seq) and then states the 2017 canonical encoder– decoder of Vaswani et al. (2017) with every variable defined, every shape labeled, and the four equations on which §§3–12 will pivot transcribed verbatim. The remaining body sections then walk one axis each, in roughly the order the axis enters the forward pass:

- §3 walks subword tokenization (BPE, SentencePiece, tiktoken-family byte-level), the tied- versus- untied I/O embedding decision and its scale-dependence, the vocabulary-size sweep from $\sim$ 32k to 256k, and the still-open tokenizer-free contender (BLT).
- §4 traces sinusoidal → learned → T5 relative bias → ALiBi → RoPE → NoPE, explains why RoPE became dominant, and treats long-context extension (NTK-aware, YaRN, LongRoPE) as a positional question.
- §5 is the anchor section on the KV-sharing axis: MHA → MQA → GQA → MLA, with the cache-cost-vs-quality trade curve and the `[CONTRADICTION]` that MLA's quality match against MHA at frontier scale is single-source from DeepSeek-V2.
- §6 keeps the attention *formula* fixed and varies the GPU schedule: FlashAttention-1 → FA2 → FA3 → Native Sparse Attention (NSA), foregrounding the distinction between exact tile-reordering and approximate sparsification.
- §7 walks dense ReLU → GELU → SwiGLU and the dense-vs-MoE multiplicity (Switch top-1 → Mixtral top-2 → DeepSeekMoE fine-grained-plus-shared), with the routing-collapse problem and load-balancing mathematics.
- §8 walks LayerNorm → RMSNorm along the type axis and Post-LN → Pre-LN → DeepNorm / Peri-LN along the placement axis, motivating frontier-2026's RMSNorm + Pre-LN with $1/\sqrt{N}$ residual init.
- §9 steps off the attention scaffold entirely: S4/S5 → Mamba → Mamba-2 along the SSM lineage, and the SSM-attention hybrid family (Jamba, Samba, Zamba2, Hymba) that the 2024-2026 long-context literature has shifted toward.
- §10 draws the distinction between inference-only speculative decoding (DeepSeek-AI, 2024b, Leviathan et al. 2023; EAGLE; Medusa) and the architectural choice of multi-token prediction (MTP), with DeepSeek-V3 as the worked integration.
- §11 catalogues the three-piece pattern – vision encoder, projector, LLM body – across the cross-attention bridge (Flamingo), linear-projection (LLaVA), and native- multimodal (Llama 4, Gemini, GPT-4o-class) integrations.
- §12 folds in the architectural constraints imposed by KV-cache lifecycle (paged attention), quantization regimes (INT8, INT4, FP8, NVFP4, BitNet), and structured-output enforcement (logit bias, Outlines FSM, XGrammar).

§13 cross-tabulates the frontier-2026 disclosed models – DeepSeek V3, Llama 4, Mistral Large, Qwen3, Gemini-class, Claude-class, GPT-class – on the ten axes, colour-coded to show where they agree and where they diverge; this is where the "model-zoo bias" question gets its direct answer. §14 concentrates the `[CONTRADICTION]` markers from the body into one place: MLA versus MHA at scale, NSA versus FA3 head-to-head, SSM-hybrid quality at frontier, the dense-MoE quality gap at matched active parameters, the long-context extrapolation method comparison, and MTP at non-DeepSeek scale. For each, we name what evidence would close the question and predict whether 2027-class models will. The closing thesis is that the open architectural questions are now *localised* – one or two axes at a time, with the rest of the skeleton stable – which is what makes the convergence claim non-trivial. We turn first to the foundation: §2.

2 Pre-2017 prequel and the 2017 canonical

The modern Transformer is a small set of choices made on top of a 2017 canonical that was itself a tightly-engineered response to the known pathologies of the recurrent and convolutional sequence-to-sequence architectures it replaced. **The thesis of this section: every axis of variation walked in the rest of this paper (KV-sharing, attention-implementation, FFN, normalization, positional encoding, multi-token output, multimodality) is a localized perturbation of the six-block encoder-decoder of Vaswani et al. (2017).** Establishing what that 2017 baseline actually computes – with every variable defined, every shape labeled, and the four equations on which §3–§12 will pivot transcribed verbatim – is the load-bearing job of this section. We start with what the Transformer replaced, why it replaced them, then state the 2017 architecture in the notation we will carry through the paper.

2.1 What the Transformer replaced

By the end of 2016 the dominant family for sequence transduction was the recurrent encoder-decoder with attention. Three pathologies were known well enough to be the abstract of Vaswani et al. (2017) itself ("dispensing with recurrence and convolutions entirely")[1] (Vaswani et al., 2017, abstract).

RNN/LSTM seq2seq: the sequential bottleneck. The recurrent encoder consumes one token at a time, computing $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$ for $t = 1, \ldots, S$, which makes the per-step depth equal to the sequence length and forecloses parallelism along S. Long-range gradient flow through this recurrence vanishes (or, less commonly, explodes) at a rate set by the operator norm of the Jacobian $\partial \mathbf{h}_t / \partial \mathbf{h}_{t-1}$; the LSTM cell

deepen-cite: hochreiter1997-lstm §3 (gating equations: input, forget, output gates plus cell-state recurrence)

replaces a saturating non-linearity with a gated linear cell-state pathway $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$ to soften the decay, but does not remove it. Equally damaging at training time, the $O(S)$ wall-clock step count multiplies with the depth-L stack so the encoder's wall-clock training scales as $O(L \cdot S)$ even with full hardware parallelism along the batch axis. Vaswani et al. (2017) §1 names this directly: "This inherently sequential nature precludes parallelization within training examples, which becomes critical at longer sequence lengths ... "

Encoder-decoder attention: the right idea, wrong base. Vaswani et al. (2017)'s "attention" did not appear from nothing.

deepen-cite: bahdanau2014-attention §A.2 (additive attention scoring function $a(\mathbf{s}_{i-1}, \mathbf{h}_j) = \mathbf{v}_a^\top \tanh(W_a \mathbf{s}_{i-1} + U_a \mathbf{h}_j)$ and the soft-alignment context $\mathbf{c}_i = \sum_j \alpha_{ij} \mathbf{h}_j$)

introduced the mechanism: at each decoder step i, learn weights $\alpha_{ij} = \mathrm{softmax}_j(a(\mathbf{s}_{i-1}, \mathbf{h}_j))$ over the encoder's hidden states $\{\mathbf{h}_j\}_{j=1}^{S_{\mathrm{src}}}$, and read out $\mathbf{c}_i = \sum_j \alpha_{ij} \mathbf{h}_j$ as a content-addressed summary of the source. The attention idea was sound and survived intact; what Vaswani et al. (2017) would discard is the *recurrent base* that the 2014–2016 encoder-decoders bolted attention onto, and the *additive* compatibility function in favor of a scaled dot-product (§2.3 below).

ConvSeq2Seq: parallel but local. A natural alternative to recurrence is convolution.

deepen-cite: gehring2017-convseq2seq §2 (GLU-gated 1D convolutions over source and target with multi-step attention)

[1] KB excerpt: `kb/excerpts/vaswani2017.md#abstract`.

replaces both encoder and decoder recurrences with stacks of GLU-gated 1D convolutions, gaining within-example parallelism along S at the price of a $O(\log_k S)$-deep receptive field for kernel size k: long-range dependencies require either deep stacks or attention bolted on top. This was the strongest direct rival to the Transformer at the moment of submission and forms the headline comparison in Vaswani et al. (2017, Table 2), where the Transformer matches or exceeds ConvS2S on WMT'14 EN–DE BLEU at a fraction of the training FLOPs.

Intuition

Three failure modes, one root cause: each of the pre-2017 designs forces inter-position information transfer through a structurally constrained pipe. RNNs route everything through a D-dim recurrent state at $O(S)$ sequential depth; CNNs route through a fixed kernel of $O(\log_k S)$ depth. Attention discards the pipe. Returning to math: the canonical attention function (Eq. (Vaswani–Eq. 1) below) computes an $S \times S$ score matrix in one parallel step, making every position directly addressable from every other position at $O(1)$ "hops" and $O(S^2)$ FLOPs. The trade is depth-for-quadratic, and – as the rest of this paper documents – the field paid the quadratic price and built the engineering apparatus to absorb it.

2.2 The 2017 canonical: encoder-decoder structure

The Transformer of Vaswani et al. (2017) is a stack of L identical blocks operating on sequences of D-dimensional vectors, with no recurrence and no convolution. We carry the tensor-shape conventions of table 1 through this section and the rest of the paper.

Symbol	Meaning	Vaswani-base value
B	batch size	— (training-dependent)
S	sequence length (tokens)	up to 512 (training); – at inference
D	residual-stream / model dimension	512
H	number of attention heads	8
d_h	per-head dimension, $d_h = \mathsf{D}/\mathsf{H}$	64
d_{ff}	FFN inner dimension	2048 ($= 4\mathsf{D}$)
V	tokenizer vocabulary size $\lvert V\rvert$	∼37k (BPE, EN–DE) (Vaswani et al., 2017, §5.1)
L	blocks per stack (encoder; decoder)	6 each

Table 1: Tensor-shape and capacity notation used throughout this paper, with the Vaswani-base values from Vaswani et al. (2017, §3.1, §5.1). The residual stream carries shape $\mathsf{B}\times\mathsf{S}\times\mathsf{D}$; after the head-split inside attention it carries $\mathsf{B}\times\mathsf{H}\times\mathsf{S}\times d_h$. Subsequent sections (§3 onward) hold this notation fixed and label any divergence (MLA's d_c, MoE's E and k, GQA's G).

The encoder is a stack of $L = 6$ identical layers; each has two sublayers, a multi-head self-attention block and a position-wise feed-forward block, with a residual connection and layer normalization wrapping each sublayer[2]. The decoder also has $L = 6$ layers but inserts a third sublayer – encoder-decoder cross-attention – between the self-attention and the FFN, and masks the self-attention so that position i can attend only to positions $\leq i$ (Vaswani et al., 2017, §3.1). Writing the sublayer

[2]KB excerpt: `kb/excerpts/vaswani2017.md#sec-3-1`.

abstractly as F and its input as $\mathbf{x}$, the canonical placement is what Vaswani et al. (2017) state verbatim[3]:

$$\mathbf{y} \;=\; \mathrm{LayerNorm}(\mathbf{x} + F(\mathbf{x})) \qquad \text{(Vaswani–§3.1)}$$

(Vaswani et al., 2017, §3.1). This is the *Post-LN* placement (norm *after* the residual add), and it is the placement Radford et al. (2019) will discard in GPT-2 in favor of Pre-LN (§8); the choice has nontrivial consequences for training stability at depth (Xiong et al., 2020), but in the 2017 canonical we keep Eq. (Vaswani–§3.1) exactly. All sublayers and the embedding layers produce outputs of the same dimension $\mathsf{D} = 512$, "to facilitate these residual connections" (Vaswani et al., 2017, §3.1).

2.3 Scaled dot-product attention

The attention sublayer is the load-bearing departure from the recurrent and convolutional baselines. Vaswani et al. (2017) define scaled dot-product attention on per-head matrices $Q, K \in \mathbb{R}^{\mathsf{S}\times d_k}$, $V \in \mathbb{R}^{\mathsf{S}\times d_v}$ as[4]

$$\mathrm{Attention}(Q, K, V) \;=\; \mathrm{softmax}\!\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \qquad \text{(Vaswani–Eq. 1)}$$

(Vaswani et al., 2017, §3.2.1, Eq. 1). The inner $QK^\top$ is the $\mathsf{S} \times \mathsf{S}$ score matrix; row i of the post-softmax result is a probability distribution over keys, and the multiplication by V on the right reads out a convex combination of value rows – *content-based retrieval* from a set of key-value pairs. The $\sqrt{d_k}$ scaling keeps the pre-softmax logit variance independent of d_k at initialization: for q, k with i.i.d. unit-variance components, $\mathrm{Var}(q \cdot k) = d_k$ (Vaswani et al., 2017, §3.2.1, fn. 4), and unscaled dot products grow with d_k "pushing the softmax function into regions where it has extremely small gradients" (Vaswani et al., 2017, §3.2.1).

Analogy

Eq. (Vaswani–Eq. 1) is a soft, learnable, parallel associative-memory lookup. The keys are addresses; the values are contents; the query is what we are asking. The softmax over $QK^\top/\sqrt{d_k}$ is the soft index that, were it a one-hot vector, would pick a single (K_j, V_j) pair; in the soft case it is a content-weighted average. Returning to canonical math: the row i output is $\sum_j \mathrm{softmax}_j(q_i^\top k_j/\sqrt{d_k})\, v_j$, which is precisely a weighted-sum read of V keyed by the content alignment between q_i and the k_j's.

Multi-head. Rather than running a single attention function on D-dimensional queries, keys, and values, Vaswani et al. (2017) project them H times to d_k, d_k, d_v dimensions and run Eq. (Vaswani–Eq. 1) in parallel[5]:

$$\begin{aligned}\mathrm{MultiHead}(Q, K, V) &= \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_{\mathsf{H}})\, W^O, \\ \mathrm{head}_i &= \mathrm{Attention}(QW_i^Q, KW_i^K, VW_i^V), \end{aligned} \qquad \text{(Vaswani–Eq. 2–3)}$$

(Vaswani et al., 2017, §3.2.2, Eq. 2–3), with projection matrices $W_i^Q, W_i^K \in \mathbb{R}^{\mathsf{D}\times d_k}$, $W_i^V \in \mathbb{R}^{\mathsf{D}\times d_v}$, and $W^O \in \mathbb{R}^{\mathsf{H}d_v\times \mathsf{D}}$. The 2017 base chooses $\mathsf{H} = 8$ heads and $d_k = d_v = d_h = \mathsf{D}/\mathsf{H} = 64$, so the per-head projections *partition* the model dimension across heads and the total compute matches a single-head full-width attention (Vaswani et al., 2017, §3.2.2). "Multi-head attention allows the

[3] KB excerpt: `kb/excerpts/vaswani2017.md#sec-3-1`, the same anchor as immediately above.
[4] KB excerpt: `kb/excerpts/vaswani2017.md#sec-3-2-1`.
[5] KB excerpt: `kb/excerpts/vaswani2017.md#sec-3-2-2`.

model to jointly attend to information from different representation subspaces at different positions; with a single attention head, averaging inhibits this" (Vaswani et al., 2017, §3.2.2)[6]. After projection the attention tensors carry shape $\mathsf{B}\times\mathsf{H}\times\mathsf{S}\times d_h$; the score matrix has shape $\mathsf{B}\times\mathsf{H}\times\mathsf{S}\times\mathsf{S}$; the post-softmax-times-V output returns to $\mathsf{B}\times\mathsf{H}\times\mathsf{S}\times d_h$ and the W^O projection collapses heads back to $\mathsf{B}\times\mathsf{S}\times\mathsf{D}$. The shape $\mathsf{B}\times\mathsf{H}\times\mathsf{S}\times d_h$ is the lens the rest of this paper is read through.

Three uses of attention. The 2017 model uses multi-head attention in three places (Vaswani et al., 2017, §3.2.3)[7]: encoder self-attention (queries, keys, and values all from the previous encoder layer), decoder *masked* self-attention (same sources, but with positions $> i$ masked to $-\infty$ inside the softmax to preserve autoregression), and encoder-decoder cross-attention (queries from the previous decoder layer; keys and values from the encoder output). The decoder-only LLMs of §3 onward keep only the second of these three – masked self-attention – and the resulting single causal stack is what survives into 2026.

2.4 Position-wise feed-forward network

Each block contains a position-wise feed-forward sublayer applied identically and independently to each position[8]:

$$\mathrm{FFN}(\mathbf{x}) \;=\; \max(0, \mathbf{x}W_1 + b_1)\,W_2 + b_2 \qquad \text{(Vaswani–Eq. 2 (FFN))}$$

(Vaswani et al., 2017, §3.3), with $W_1 \in \mathbb{R}^{\mathsf{D}\times d_{\mathrm{ff}}}$, $W_2 \in \mathbb{R}^{d_{\mathrm{ff}}\times\mathsf{D}}$, and the canonical choice $d_{\mathrm{ff}} = 4\mathsf{D}$ (so $d_{\mathrm{ff}} = 2048$ in the base configuration). Vaswani et al. (2017) note this is also "two convolutions with kernel size 1" (Vaswani et al., 2017, §3.3) – the FFN does not mix across positions, only across channels at each position. The same (W_1, b_1, W_2, b_2) is shared across all S positions but *not* across the L layers; each layer learns its own pair. The activation is ReLU ($\max(0,\cdot)$). The factorization of every block into "mix across positions, then mix across channels" is the structural property the rest of the paper preserves: §5 replaces the attention sublayer's KV-sharing pattern, §7 replaces the FFN's ReLU-2-matrix form (with SwiGLU and then MoE), and the strict position/channel split survives every variant.

2.5 Sinusoidal positional encoding

Because the model contains no recurrence or convolution, position information must be injected explicitly[9]. Vaswani et al. (2017) add a fixed sinusoidal encoding to the input embeddings at the bottom of each stack:

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/\mathsf{D}}), \qquad PE_{(pos,2i+1)} = \cos(pos/10000^{2i/\mathsf{D}}), \qquad \text{(Vaswani–PE)}$$

(Vaswani et al., 2017, §3.5), where pos is the integer position and i indexes the embedding dimension. The two halves of the embedding are filled with a sinusoid of geometrically progressing wavelengths from 2π to $10000 \cdot 2\pi$, and Vaswani et al. (2017) motivate the choice as making relative position linearly recoverable: "for any fixed offset k, PE_{pos+k} can be represented as a linear function of PE_{pos}" (Vaswani et al., 2017, §3.5). They also report that learned absolute embeddings match in quality, and pick the sinusoidal version for the extrapolation-to-longer-sequences argument it

[6]KB excerpt: `kb/excerpts/vaswani2017.md#sec-3-2-2-motivation`.
[7]KB excerpt: `kb/excerpts/vaswani2017.md#sec-3-2-3`.
[8]KB excerpt: `kb/excerpts/vaswani2017.md#sec-3-3`.
[9]KB excerpt: `kb/excerpts/vaswani2017.md#sec-3-5`.

appears (in 2017) to license. The lineage that follows – learned absolute, T5 relative bias, ALiBi, RoPE – is the subject of §4; RoPE in particular makes a different bet on *multiplicative* position injection inside the attention dot product, and is the 2026 default.

2.6 Embeddings, weight tying, and softmax output

The token-ID-to-vector and vector-to-logit interfaces are also specified in the paper[10]. The input embedding $E_{\text{in}} \in \mathbb{R}^{\mathsf{V}\times\mathsf{D}}$ maps token IDs to D-vectors; the LM head $E_{\text{out}} \in \mathbb{R}^{\mathsf{D}\times\mathsf{V}}$ projects the final residual stream to vocabulary logits; a softmax produces next-token probabilities. Vaswani et al. (2017) adopt the *weight-tying* scheme of Press and Wolf (2017): the matrix used as E_{in} is also used (transposed) as E_{out}, so a single $\mathsf{V} \times \mathsf{D}$ table serves both interfaces, and "in the embedding layers, we multiply those weights by $\sqrt{\mathsf{D}}$" (Vaswani et al., 2017, §3.4). In the notation of §3 this is

$$E_{\text{out}} \;=\; E_{\text{in}}^{\top}, \qquad \text{(Press–Vaswani tying)}$$

(Press and Wolf, 2017). Tying saves $\mathsf{V} \times \mathsf{D}$ parameters – a substantial fraction of the 2017 base model's ~ 65M parameters when $\mathsf{V} \approx 37$k and $\mathsf{D} = 512$. §3 discusses why frontier-2026 *untied* the I/O embeddings as V and D both grew (the relative parameter share of the embedding table fell, and the inductive bias that input and output token spaces should share geometry weakened); GPT-2 (Radford et al., 2019, §2.3) kept tying, but LLaMA-3 and DeepSeek-V3 do not.

2.7 The block in one picture, and what comes next

Combining Eqs. (Vaswani–§3.1), (Vaswani–Eq. 2–3), and (Vaswani–Eq. 2), one decoder block in the 2017 canonical takes residual stream $X^{\ell} \in \mathbb{R}^{\mathsf{B}\times\mathsf{S}\times\mathsf{D}}$ to $X^{\ell+1}$ via

$$X^{\ell+1/2} = \text{LayerNorm}\big(X^{\ell} + \text{MaskedMHA}(X^{\ell})\big), \tag{1}$$

$$X^{\ell+1} = \text{LayerNorm}\big(X^{\ell+1/2} + \text{FFN}(X^{\ell+1/2})\big), \tag{2}$$

with positional information added once at the bottom via Eq. (Vaswani–PE), the embedding via Eq. (Press–Vaswani tying), and the LM head re-using $E_{\text{in}}^{\top}$. The full hyperparameter row of the base model is $L = 6$, $\mathsf{D} = 512$, $\mathsf{H} = 8$, $d_h = 64$, $d_{\text{ff}} = 2048$, $\mathsf{V} \approx 37$k (Vaswani et al., 2017, §3.1, §5.1).

What changes after 2017, by axis. Every subsequent section of this paper changes *exactly one* of the components just defined, holding the others fixed. §3 takes the tokenization, the input embedding, and the unembedding, and walks subword → byte-level → tokenizer-free, plus the tied-vs-untied decision of Eq. (Press–Vaswani tying). §4 takes the sinusoidal Eq. (Vaswani–PE) and walks learned → T5 relative bias → ALiBi → RoPE → NoPE. §5 keeps Eq. (Vaswani–Eq. 1) unchanged but varies how the H heads in Eq. (Vaswani–Eq. 2–3) share their K and V projections (MHA → MQA → GQA → MLA). §5 keeps both Eqs. (Vaswani–Eq. 1) and (Vaswani–Eq. 2–3) mathematically unchanged but schedules the same computation differently onto GPU memory hierarchy (FA1 → FA2 → FA3 → NSA). §7 replaces the ReLU-2-matrix Eq. (Vaswani–Eq. 2) with SwiGLU and then with MoE. §8 replaces LayerNorm in Eq. (Vaswani–§3.1) with RMSNorm and moves it to Pre-LN. §10 adds heads to the LM head of §2.6. §11 extends the embedding interface of §2.6 to non-text inputs.

That *everything else* – the residual structure, the position/channel factorization, the softmax-over-keys, the head partition, the $\mathsf{B}\times\mathsf{S}\times\mathsf{D}$ residual stream as the workspace – survives across all eight axes is the empirical fact this paper is built around. We turn first to the discrete-token interface of §3.

[10] KB excerpt: `kb/excerpts/vaswani2017.md#sec-3-4`.

3 Tokenization, embedding, and unembedding

The discrete-token interface of a Transformer LLM is implemented by three parameter blocks that bracket the residual stream: a tokenizer that maps text to integer IDs in $\{0, \ldots, |V|-1\}$, an input embedding $E_{\text{in}} \in \mathbb{R}^{|V| \times \mathsf{D}}$ that lifts those IDs into the residual stream of shape $\mathsf{B} \times \mathsf{S} \times \mathsf{D}$, and an LM head $E_{\text{out}} \in \mathbb{R}^{\mathsf{D} \times |V|}$ that projects the final residual stream back to logits of shape $\mathsf{B} \times \mathsf{S} \times \mathsf{V}$. The thesis of this section is that the field has converged on a small lineage of choices along this triple: subword BPE in either the SentencePiece or byte-level (tiktoken) family, with a clear scale-dependent split on weight tying — tied at $\lesssim$ 1B parameters, untied at $\gtrsim$ 8B — and a still-open contender, the tokenizer-free byte-patch model (BLT), which is the first 2024 architecture to match a token-based 8B baseline at fixed FLOPs (AI, 2024a). The choices are independent, but they interact: vocabulary size sets the embedding-table cost, which sets the parameter share at which untying becomes free.

3.1 Subword tokenization: BPE and SentencePiece

Sennrich et al. (2016) adapts *byte-pair encoding* (Gage 1994; Sennrich et al., 2016, §3.2)[11] from data compression to NLP word segmentation. The training procedure is greedy bottom-up: initialize the symbol vocabulary with the character (or byte) alphabet, represent each word as a sequence of those symbols plus an end-of-word marker, then iterate the pair-merge loop. The reference implementation as transcribed in Sennrich et al. (2016, §3.2, Algorithm 1) is:

Listing 1: BPE training (Sennrich et al. 2016, §3.2)

```
import re, collections
def get_stats(vocab):
    pairs = collections.defaultdict(int)
    for word, freq in vocab.items():
        symbols = word.split()
        for i in range(len(symbols)-1):
            pairs[symbols[i], symbols[i+1]] += freq
    return pairs
def merge_vocab(pair, v_in):
    v_out = {}
    bigram = re.escape(' '.join(pair))
    p = re.compile(r'(?<!\S)' + bigram + r'(?!\S)')
    for word in v_in:
        w_out = p.sub(''.join(pair), word)
        v_out[w_out] = v_in[word]
    return v_out
num_merges = 10
for i in range(num_merges):
    pairs = get_stats(vocab)
    best = max(pairs, key=pairs.get)
    vocab = merge_vocab(best, vocab)
```

The final vocabulary size is the size of the initial alphabet plus the number of merge operations — *the only hyperparameter of the algorithm* (Sennrich et al., 2016, §3.2). At inference, the same merge list is replayed deterministically over new text. Crucially, BPE is *not* a left-to-right segmenter: the same prefix can yield a different first token when the suffix changes. We will return to this when contrasting with byte-patch models in section 3.5.

[11] KB excerpt: `kb/excerpts/sennrich2016.md#sec-3-2`.

Kudo and Richardson (2018) packages BPE (and a unigram-LM alternative) into a language-agnostic library with three additional design choices that became the modern default for open-weight LLMs. First, *lossless tokenization*: SentencePiece treats raw input, including whitespace, as a sequence of Unicode characters and escapes spaces with the meta-symbol ⊔ (U+2581) so that decode(encode(x)) = x holds verbatim (Kudo and Richardson, 2018, §3.1).[12] Second, an $O(N \log N)$ priority-queue training algorithm that scales to multi-TB corpora (Kudo and Richardson, 2018, §3.2). Third, *byte fallback*: out-of-vocabulary characters decompose into the 256 UTF-8 byte tokens, eliminating the UNK symbol entirely (Kudo and Richardson, 2018, §byte-fallback). The three together make a single tokenizer trainable on raw multilingual text — the choice behind LLaMA-1/2/3 (Touvron et al., 2023; AI, 2024b), Mistral, Qwen, Gemma, and OLMo 2 (Team, 2025).

Radford et al. (2019) addresses the orthogonal question of what *base alphabet* BPE should run on. A character-level base requires the full Unicode code-point space (> 130K symbols before any merge), while a byte-level base needs only 256. The naive byte-level recipe, however, is sub-optimal for a load-bearing reason (Radford et al., 2019, §2.2)[13]:

> *We observed BPE including many versions of common words like* `dog` *since they occur in many variations such as* `dog. dog! dog?`*. This results in a sub-optimal allocation of limited vocabulary slots and model capacity. To avoid this, we prevent BPE from merging across character categories for any byte sequence. We add an exception for spaces which significantly improves the compression efficiency while adding only minimal fragmentation of words across multiple vocab tokens.*

The category-respecting byte-level BPE of Radford et al. (2019) is the direct ancestor of OpenAI's tiktoken (used by GPT-3, GPT-4, GPT-4o at $|V| = 200$K, and Phi-4 at $|V| = 100$K (et al. , Microsoft, §2.1)) and of DeepSeek-V3's $|V| = 128$K byte-level BPE (DeepSeek-AI, 2024b, §2.1).

3.2 Embedding and unembedding

Given token-ID sequence $\mathbf{t} \in \{0, \ldots, |V|-1\}^N$, residual width D, and depth L, the input embedding is a learned lookup,

$$X_i^0 = E_{\text{in}}[t_i] \in \mathbb{R}^{\mathsf{D}}, \qquad E_{\text{in}} \in \mathbb{R}^{|V| \times \mathsf{D}}, \tag{3}$$

producing the initial residual stream X^0 of shape $\mathsf{B} \times \mathsf{S} \times \mathsf{D}$. After L blocks the LM head produces logits,

$$\text{logits}_i = X_i^L \, E_{\text{out}} \in \mathbb{R}^{|V|}, \qquad E_{\text{out}} \in \mathbb{R}^{\mathsf{D} \times |V|}, \tag{4}$$

of shape $\mathsf{B} \times \mathsf{S} \times \mathsf{V}$ (Vaswani et al., 2017, §3.4). In modern decoder-only LLMs X^L is the output of the final RMSNorm rather than the raw post-block residual; eq. (4) is otherwise unchanged. The LM-head matmul costs $O(N \cdot |V| \cdot \mathsf{D})$ FLOPs per position; at $|V| = 128$K and $\mathsf{D} = 4096$ this is $\sim 5 \times 10^8$ FLOPs per token, comparable to a single Transformer block.

The original encoder-decoder Transformer scales the embedding by $\sqrt{\mathsf{D}}$ before adding the positional signal (Vaswani et al., 2017, §3.4), an artifact of the Xavier-initialization variance regime; modern decoder-only LLMs typically omit the scaling and rely on the first RMSNorm to fix the norm of X^0.

[12] KB excerpt: `kb/excerpts/kudo2018-sentencepiece.md#sec-3-1`.
[13] KB excerpt: `kb/excerpts/radford2019-gpt2.md#sec-2-2`.

3.3 Weight tying

Press and Wolf (2017) observed that the input embedding U and the output projection W both index the same vocabulary and proposed constraining them:

$$W = U, \quad \text{equivalently} \quad E_{\text{out}} = E_{\text{in}}^{\top} \tag{5}$$

(Press and Wolf, 2017, §3.1).[14] The forward path under tying is $\text{logits}_i = X_i^L E_{\text{in}}^{\top}$, and a single matrix E_{in} is updated from both input and output gradients. Tying halves the parameter count of the I/O blocks (saving $|V| \cdot \mathsf{D}$ parameters) and the empirical result on RNN-era language models was a perplexity *reduction*: PTB validation drops from 78.4 to 75.8 on the medium LSTM, with similar gains on the large LSTM and on Wikitext-2 (Press and Wolf, 2017, §4). On NMT encoder-decoder models with shared source/target vocabulary, three-way tying reduced parameters by 28–52% with no BLEU degradation (Press and Wolf, 2017, §5).

The frontier-2026 picture inverts the recommendation. table 2 collects the public choices.

Model	Tied?	$\lvert V\rvert$	D	$\lvert V\rvert\mathsf{D}$ params
Vaswani 2017 base	tied	37K	512	19M
GPT-2 (1.5B)	tied	50K	1600	80M
LLaMA-1 / LLaMA-2 7B	tied	32K	4096	131M
Phi-4	tied	100K	5120	512M
Qwen3 (small)	tied	152K	varies	varies
LLaMA-3 8B	untied	128K	4096	525M ×2
LLaMA-3 70B/405B	untied	128K	8192/16384	—
OLMo 2 7B/13B	untied	∼50K	varies	varies
DeepSeek-V3 671B	untied	128K	7168	920M ×2
Qwen3 (235B)	untied	152K	varies	varies

Table 2: Tying decisions across reference models. Sources: Vaswani et al. (2017, §3.4), Radford et al. (2019, §2.3), Touvron et al. (2023), et al. (Microsoft, §2.1), AI (2024b, §3.1), Team (2025), DeepSeek-AI (2024b, §2.1), Team and Alibaba (2025).

The empirical pattern is *tied at* $\lesssim 1B$*, untied at* $\gtrsim 8B$.

Intuition

Why frontier labs untie at scale: the embedding-block parameter saving from tying is constant in $|V| \cdot \mathsf{D}$, while the non-embedding model grows roughly as $L \cdot \mathsf{D}^2$. At LLaMA-3 8B the I/O embeddings are ∼12% of model parameters; at 70B, ∼5%; at 405B, ∼1%. The compression ratio of eq. (5) goes from *meaningful* to *negligible* as scale grows, while the alleged quality cost — the input embedding being pulled toward output-space directions by the dominant LM-head gradient — is constant. Returning to math: tying constrains $E_{\text{out}} = E_{\text{in}}^{\top}$ globally, but the gradient $\nabla_{E_{\text{in}}} \mathcal{L}$ acquires a contribution from the softmax cross-entropy that scales with $|V|$, while the input-side contribution scales only with the number of times each token is embedded as input. The trade flips when $|V| \cdot \mathsf{D}$ ceases to be a meaningful share of total parameters.

[14] KB excerpt: `kb/excerpts/press2017-tied-embed.md#sec-3-1`.

Contradiction

Press and Wolf (2017, §4–5) reports tying *improves* perplexity on ∼10M-parameter LSTM language models. Frontier 2024–2026 open-weight LLMs at ≥ 8B parameters (AI, 2024b; Team, 2025; DeepSeek-AI, 2024b; Team and Alibaba, 2025) untie and report this is the better choice. The reconciling claim — that the tying-induced output-space bias only dominates once embeddings are a small share of total parameters — is currently an *intuition*, not a derivation, and the few alternatives between strict tying and full untying (e.g., pseudo-inverse tying, $E_{\text{out}} = (E_{\text{in}}^{\top})^{+}$) are recent and not yet evaluated at frontier scale.

deepen-cite: press2017-tied-embed §6

3.4 Vocabulary scaling

Vocabulary size is the lever that connects tokenizer choice to embedding cost. Smaller $|V|$ produces longer token sequences (higher *fertility*, defined as tokens-per-character), inflating attention's $O(N^2)$ cost and reducing the model's effective context horizon in characters. Larger $|V|$ produces shorter sequences but inflates the embedding and LM-head matmul. The lineage of public choices traces an upward arc: GPT-2 at $|V| = 50{,}257$ (Radford et al., 2019, §2.3); LLaMA-1/2 at $|V| = 32{,}000$ on SentencePiece BPE (Touvron et al., 2023, §2.3); LLaMA-3 at $|V| = 128{,}256$ on expanded SentencePiece-BPE (AI, 2024b, §3.1); DeepSeek-V3 at $|V| = 128{,}000$ on byte-level BPE (DeepSeek-AI, 2024b, §2.1); Phi-4 at $|V| = 100{,}000$ on tiktoken cl100k (et al. , Microsoft, §2.1); GPT-4o at $|V| \approx 200{,}000$ on tiktoken o200k; Qwen3 at $|V| = 152{,}000$ (Team and Alibaba, 2025).

The capacity cost is concrete: LLaMA-3's 4× vocab expansion relative to LLaMA-2 added approximately $128{,}256 \times 4096 = 525$M parameters per embedding matrix at 8B scale, and untied means double that. The capability gain is also concrete: LLaMA-3 reports tokens-per- English-word fertility near 1.0 (vs. ∼ 1.5 for LLaMA-2's 32K vocab), with a substantially larger improvement on non-English text where 32K SentencePiece-BPE produced fertilities of 2–4 (AI, 2024b, §3.1). The same expansion reserved a block of unused token IDs explicitly for downstream fine-tuning, partly to mitigate the *glitch-token* failure mode where rarely-trained embeddings sit near initialization and produce unpredictable model behavior when prompted (the canonical example, the `SolidGoldMagikarp` token in GPT-2/3, is a Tier C community report and not relied on as a hard claim here).

Analogy

The tokenizer is a fixed prior the model cannot learn its way out of: once E_{in} is shaped by the merge list, every later representation is conditioned on what the tokenizer chose to make a single ID versus a sequence. If "12345" is one BPE merge but "1234" is five characters, the model's digit-by-digit arithmetic competence is not symmetric across strings — a fact LLaMA-3 addresses by tokenizing digits individually (AI, 2024b, §3.1). Returning to math: the tokenizer is the function $\mathbf{t} : \text{text} \to V^N$ upstream of eq. (3); its merge list defines the equivalence classes of strings that share a row of E_{in}, and the model operates on those classes, not on the underlying bytes.

3.5 Tokenizer-free: the BLT alternative

AI (2024a) replaces the fixed tokenizer with a learned byte-patching scheme. A small byte-level

auto-regressive language model is trained alongside the main LLM and used to score next-byte entropy,

$$H(x_i) = \sum_{v \in V_{\text{byte}}} p_e(x_i = v \mid x_{<i}) \log p_e(x_i = v \mid x_{<i}), \tag{6}$$

verbatim (AI, 2024a, Eq. 1).[15] Patch boundaries are placed either when $H(x_t) > \theta_g$ (*global* entropy threshold) or when $H(x_t) - H(x_{t-1}) > \theta_r$ (*approximate monotonic* relative threshold) (AI, 2024a, §2.3). The result is an *incremental* patching function — the patching of $x_{<i}$ does not depend on $x_{\geq i}$ — which BPE provably is not, because BPE merges can re-segment a prefix when its continuation changes (AI, 2024a, §2.4).

The architecture is three modules: a lightweight Local Encoder mapping bytes to patch representations, a heavy Latent Transformer that runs once per patch, and a lightweight Local Decoder that reads patch outputs back to byte logits (AI, 2024a, §3). The Latent Transformer's runtime scales with the number of patches m, not the number of bytes n, and entropy patching produces $m \ll n$ on predictable spans (e.g., the body of a named entity is one patch; the boundary between sentences is its own patch). The headline scaling result (AI, 2024a, §1, Fig. 1):

> *Overall, BLT matches training flop-controlled performance of Llama 3 while using up to 50% fewer flops at inference. ... BLT models unlock a new dimension for scaling LLMs, where model size can now be scaled while maintaining a fixed-inference budget.*

Forum signal

As of 2026-Q2, no production frontier-LLM has shipped with a BLT-family tokenizer-free architecture; the published 8B-scale parity result (AI, 2024a) is the strongest public signal that tokenizer-free is viable at non-trivial scale, and the FAIR follow-up extending the comparison above 8B is reported in pre-print review but not yet released.

deepen-cite: blt2024 §6

3.6 Forward link

The choices in this section determine the rows of E_{in} and the columns of E_{out}, but they say nothing about *order* — two sequences that differ only by permutation produce the same multiset of embedded vectors $\{E_{\text{in}}[t_i]\}_i$. The order information is supplied by the positional-encoding choice that follows: sinusoidal (Vaswani et al., 2017, §3.5), learned, T5 relative bias, ALiBi, RoPE, YaRN, NoPE. The next section (section 4) walks that lineage, and explains why RoPE plus context-extension methods became the frontier-2026 default — a choice that, like tokenization, is one axis of the small set of choices the modern Transformer actually offers.

4 Positional encoding

Self-attention is permutation-equivariant: with no positional signal the output for $(x_1, \ldots, x_S)$ is invariant under any permutation of positions, so the model treats "dog bites man" and "man bites dog" identically (Vaswani et al., 2017). Positional encoding is the mechanism that breaks this symmetry, and the family of choices is one of the small number of axes along which the modern Transformer is parameterized. The thesis of this section: between 2017 and 2026 the field walked

[15] KB excerpt: `kb/excerpts/blt2024.md#sec-2-3`.

sinusoidal → learned → T5 relative bias → ALiBi → RoPE, RoPE became dominant for decoder-only LLMs, and the practical question of long-context extension is now *primarily a positional question* (NTK/YaRN/LongRoPE), not (only) an attention- implementation question. We close with NoPE and iRoPE, where the field asks whether explicit positional encoding is needed at all once depth and causal masking are in place.

4.1 The 2017 sinusoidal baseline

Vaswani et al. (2017) adds a deterministic positional vector $\boldsymbol{p}_m \in \mathbb{R}^D$ to the token embedding at the input of the encoder/decoder stack, before any attention layer:

$$f_{\{q,k\}}(\boldsymbol{x}_m, m) = \boldsymbol{W}_{\{q,k\}}\,(\boldsymbol{x}_m + \boldsymbol{p}_m), \tag{7}$$

with the sinusoidal choice (Vaswani et al., 2017, §3.5)[16]

$$\begin{aligned} PE_{(pos,\,2i)} &= \sin\!\big(pos \,/\, 10000^{2i/d_{\text{model}}}\big), \\ PE_{(pos,\,2i+1)} &= \cos\!\big(pos \,/\, 10000^{2i/d_{\text{model}}}\big), \end{aligned} \tag{8}$$

where pos is the token position and i indexes dimension. The wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$. Vaswani et al. (2017) motivate the closed form on two grounds: a fixed offset k admits a linear map taking PE_{pos} to PE_{pos+k}, "[allowing] the model to easily learn to attend by relative positions"; and the closed form "may allow the model to extrapolate to sequence lengths longer than the ones encountered during training" (Vaswani et al., 2017, §3.5).

The same paper notes that learned absolute embeddings produce "nearly identical results" on WMT (Vaswani et al., 2017, Table 3 row (E)) — so at the original $d_{\text{model}} = 512$, $S \leq 100$ scale, fixed-vs-learned was a non-issue. The fixed/learned dichotomy only becomes load-bearing once S grows past the training horizon, which is precisely the regime where every successor encoding (T5 bias, ALiBi, RoPE) is designed to behave gracefully.

Intuition

The sinusoidal formula is the closest to a "coordinate system" the input can carry. Each pair of dimensions $(2i, 2i{+}1)$ traces a unit-circle hand at angular rate $\theta_i = 10000^{-2i/d_{\text{model}}}$. High-frequency hands ($i \to 0$) discriminate local order; low-frequency hands ($i \to d/2$) carry coarse global position. Returning to the canonical form: equation (8) is exactly the $(\sin, \cos)$ projection of those rotating hands at position pos.

4.2 Learned absolute, T5 relative bias, and ALiBi

Three intermediate families followed the sinusoidal baseline.

Learned absolute (BERT, GPT-2). Replace the fixed $\boldsymbol{p}_m$ in (7) with a learned embedding indexed by $m \in \{0, \ldots, S_{\text{train}} - 1\}$ (Devlin et al., 2019; Radford et al., 2019). The cost: a hard cutoff at the training context S_{train}. Inference at $m \geq S_{\text{train}}$ has no defined embedding, and there is no closed-form way to extend the table.

[16] KB excerpt: `kb/excerpts/vaswani2017.md#sec-3-5`.

T5 relative bias. Raffel et al. (2019) replace the additive input-side encoding with a per-layer scalar bias on the attention logit, binned by signed relative distance:

$$\boldsymbol{q}_m^\top \boldsymbol{k}_n = \boldsymbol{x}_m^\top \boldsymbol{W}_q^\top \boldsymbol{W}_k \boldsymbol{x}_n + b^{(h)}_{m-n}, \tag{9}$$

with one $b^{(h)}_\Delta$ table per head h. The bias is added inside each attention layer rather than once at the input, which avoids the "diluted-by-depth" degradation of additive input encoding (Raffel et al., 2019).

ALiBi. Press et al. (2022, ALiBi) make the bias parameter-free and *linear in distance*: $b^{(h)}_{m-n} = -|m-n| \cdot s_h$ with a fixed per-head slope s_h, producing a built-in linear penalty for distance and — the load- bearing claim — length extrapolation past S_{train} without fine-tuning.

deepen-cite: press-alibi §3 (excerpt not yet in KB; cite "Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation", Press et al. 2022)

All three lost ground to RoPE for decoder-only LLMs by 2022–2023. Learned absolute lost on extrapolation; T5 bias's binned table did not generalize cleanly across context lengths; ALiBi's linear slope is the right shape for some workloads but not for the long-range structure attention is asked to handle in modern decoder-only training.

4.3 RoPE: rotation in the QK inner product

Su et al. (2021) replace the additive encoding with a multiplicative rotation applied to Q and K inside attention.[17] For a residual-stream activation $\boldsymbol{x}_m \in \mathbb{R}^{d_h}$ at position m (per head, with d_h even):

$$f_{\{q,k\}}(\boldsymbol{x}_m, m) = \boldsymbol{R}^{d_h}_{\Theta,m} \boldsymbol{W}_{\{q,k\}} \boldsymbol{x}_m, \tag{10}$$

where $\boldsymbol{R}^{d_h}_{\Theta,m}$ is the block-diagonal orthogonal rotation matrix with $d_h/2$ blocks of the form

$$\begin{pmatrix} \cos m\theta_i & -\sin m\theta_i \\ \sin m\theta_i & \cos m\theta_i \end{pmatrix}, \quad i = 1, \ldots, d_h/2, \tag{11}$$

and the frequencies form a geometric progression with base b:

$$\Theta = \{\theta_i = b^{-2(i-1)/d_h}\}_{i=1}^{d_h/2}, \quad b = 10000 \text{ (default)} \tag{12}$$

(Su et al., 2021, §3.2.2 Eq. 14–15). The critical property follows from orthogonality: substituting (10) into the attention dot product gives

$$\boldsymbol{q}_m^\top \boldsymbol{k}_n = \boldsymbol{x}_m^\top \boldsymbol{W}_q^\top \boldsymbol{R}^{d_h}_{\Theta,n-m} \boldsymbol{W}_k \boldsymbol{x}_n, \tag{13}$$

(Su et al., 2021, §3.2.2 Eq. 16). Position information enters the attention score *only as the relative offset* $n-m$ — when $m = n$ the rotation is the identity and the inner product is content- only; otherwise it depends on $(n-m)$ but not on m or n separately.

In practice the rotation is computed elementwise rather than as a matmul, exploiting the sparsity of $\boldsymbol{R}^{d_h}_{\Theta,m}$:

$$\boldsymbol{R}^{d_h}_{\Theta,m} \boldsymbol{x} = \boldsymbol{x} \otimes \cos(m\boldsymbol{\theta}) + \text{rotate\_half}(\boldsymbol{x}) \otimes \sin(m\boldsymbol{\theta}), \tag{14}$$

[17] KB excerpt: `kb/excerpts/su2021.md#sec-3-2-2`.

where $\boldsymbol{\theta}$ duplicates each θ_i to length d_h and rotate_half swaps each pair $(x_{2i-1}, x_{2i}) \mapsto (-x_{2i}, x_{2i-1})$ (Su et al., 2021, §3.4.2 Eq. 34). This is the form behind the `apply_rotary_pos_emb(q, k, cos, sin)` helper in HF transformers and the `ggml_rope_*` family in `llama.cpp`; two element-wise multiplies and one add per query/key, negligible overhead.

Three properties make RoPE the dominant choice in 2022–2026 frontier decoder-only LLMs (LLaMA 1–3, Mistral, Qwen, DeepSeek, Gemma, OLMo). First, the relative-only dependence (13) is arithmetic, not learned — $\boldsymbol{R}^{d_h}_{\Theta,m}$ is defined for any $m \in \mathbb{N}$ without indexing a learned table. Second, applying the rotation *at every layer on Q, K only* preserves the residual stream's content basis. Third, the long-term-decay property (Su et al., 2021, §3.3): at fixed Θ, the inner product $\langle f_q(\cdot, m), f_k(\cdot, n)\rangle$ tends to decay as $|m-n|$ grows because rotations at different frequencies dephase, matching the prior that distant tokens should interact less.

Analogy

RoPE is a clock with $d_h/2$ hands, each ticking at a different rate. At position m, hand i points to angle $m\theta_i$. Query $\boldsymbol{q}_m$ and key $\boldsymbol{k}_n$ "agree" on dimension-pair i when their hands align modulo 2π — when $(m-n)\theta_i \equiv 0$. Returning to canonical form: hand alignment is exactly the cosine factor $\cos((m-n)\theta_i)$ that emerges from expanding $\boldsymbol{x}_m^\top \boldsymbol{R}^{d_h}_{\Theta,n-m}\boldsymbol{x}_n$ in the canonical 2×2 block basis of (11).

4.4 Long-context extension: NTK-aware, YaRN, LongRoPE

The pre-trained context S_{train} is set by training data and compute. Deploying at $S_{\text{infer}} > S_{\text{train}}$ requires some adjustment because positions $m > S_{\text{train}}$ produce rotations $m\theta_i$ that are out-of-distribution: the model has never observed these phases on Q, K during training, and attention features may not generalize. The extension methods are the practical positional question for the 128K-and-up regime.

Position interpolation (PI). Chen et al. 2023 propose, for ratio $s = S_{\text{infer}}/S_{\text{train}} > 1$, replacing $m\theta_i$ with $(m/s)\theta_i$ in (11), mapping the new range into the training-seen range.

deepen-cite: chen2023-position-interpolation §3 (excerpt not yet in KB; cite "Extending Context Window of Large Language Models via Positional Interpolation", Chen et al. 2023, arXiv 2306.15595)

Cheap but degrades short-context quality because it compresses the high-frequency dimensions.

NTK-aware scaling. The insight: high-frequency dimensions (θ_i small i) execute many full rotations within S_{train} and should not be interpolated — compressing them harms local fidelity. Low-frequency dimensions execute less than one rotation in S_{train} and benefit from interpolation. The NTK-aware fix scales the *base* b instead of the positions:

$$b' = b \cdot s^{d_h/(d_h-2)}, \tag{15}$$

which compresses primarily the low-frequency dimensions while leaving high-frequency ones nearly intact (Su et al., 2021, §3.2, `kb/notes/architecture/position-encoding.md#sec-3-2`).

YaRN.

deepen-cite: peng2023-yarn (excerpt not yet in KB; cite "YaRN: Efficient Context Window Extension of Large Language Models", Peng et al. 2023, arXiv 2309.00071)

YaRN combines NTK-aware base-scaling with a per-frequency interpolation schedule: dimensions with wavelength shorter than the training context use no scaling, dimensions longer than the extended context use full PI, intermediate dimensions use a smooth ramp. YaRN additionally rescales attention logits by $1/\sqrt{t}$ where t depends on the extension factor s:

$$\text{softmax}\Big(\frac{QK^\top}{\sqrt{d_h}\,\sqrt{t}}\Big)V, \quad \sqrt{t} = 0.1\ln(s) + 1, \tag{16}$$

preserving the softmax temperature regime as the effective sequence length grows. YaRN reports roughly 10× fewer fine-tuning tokens than plain PI to extend LLaMA-2 to 128K (note: equations (15), (16) are reconstructed from the LLaMA-3 implementation and secondary references; see the deepen-cite above for primary verification).

LongRoPE / LongRoPE2. LongRoPE introduces evolutionary search over per-frequency scaling factors — a non-uniform rescaling that no closed-form method can produce, reaching 2M-token context. various (2025b) extend this to a per-position search and report near-lossless extension (98.5% of short-context performance preserved) at 80× fewer tokens than Meta's earlier LLaMA-3 long-context fine-tuning approach.

deepen-cite: longrope2-2025 §3 (KB excerpt abstract-only as of 2026-05-05; equations and the per-position search schedule still need transcription)

The pattern across PI → NTK → YaRN → LongRoPE is one of *increasingly fine-grained per-frequency control over* θ_i. Each step preserves the multiplicative structure of (10); only the rule for which θ_i get rescaled and by how much changes. Long-context extension is, in this lineage, a positional question — a choice of frequency schedule — not a choice of attention implementation. The attention kernel (FlashAttention, NSA; §6 of this paper) is unchanged.

4.5 NoPE and iRoPE

deepen-cite: kazemnejad2023-nope (excerpt not yet in KB; cite "The Impact of Positional Encoding on Length Generalization in Transformers", Kazemnejad et al. 2023, NeurIPS)

The NoPE result (Kazemnejad et al. 2023) is that decoder-only Transformers trained *without any positional encoding* can generalize as well or better than RoPE/ALiBi to longer sequences on synthetic length-generalization tasks. The mechanism is depth-induced: causal masking gives every position a unique attention pattern (position m has $m+1$ attendable tokens, so position 0 is uniquely identifiable as the only one with empty attention context), and stacked layers can recover positional information from this content-mask interaction without an explicit embedding.

AI (2026) apply this in production with iRoPE: alternate RoPE-equipped attention layers with NoPE attention layers in a 3:1 ratio, with a temperature-scaling factor in the NoPE layers. Trained natively at $S_{\text{train}} = 256$K, LLaMA 4 Scout reports a claimed 10M-token inference context — a qualitative regime change, not just better interpolation.

Speculation

Fallback framing of iRoPE. The pattern resembles a fallback: RoPE layers provide explicit local-order constraints; NoPE layers are forced to derive position from content alone, providing a channel that does not depend on the rotation when phases at $m \gg S_{\text{train}}$ go out-of-distribution. AI (2026)'s own theoretical justification for the pattern, if any, is not in this KB at this writing; the framing is offered here as a coherent reading rather than as the paper's stated mechanism.

Contradiction

Long-context extrapolation method comparison is contested as of 2026-05. Three competing positions: (a) RoPE-only with NTK-aware base scaling is sufficient for the $\leq 4\times$ extension regime (e.g. LLaMA-3's bump from 8K to 128K); (b) YaRN with a per-frequency schedule is required for the $\sim 10\times$ regime (

deepen-cite: peng2023-yarn

); (c) per-position search (LongRoPE2) and mixed RoPE/NoPE layers (iRoPE) are required for the $\geq 40\times$ regime (various, 2025b; AI, 2026). The benchmarks backing each are largely single-source: Meta reports iRoPE at 10M, Microsoft reports LongRoPE2 at 2M, with no third-party head-to-head at frontier scale. NIAH-style retrieval saturates near 100% at all of these context lengths; harder evaluations (RULER, HELMET) show steeper and methodologically inconsistent degradation patterns past 32K, making "which extension method is best" contingent on which long-context benchmark is treated as ground truth (see `kb/notes/architecture/long-context.md#sec-5`).

4.6 Summary and forward link

The 2017–2026 lineage along the positional-encoding axis is sinusoidal → learned absolute → T5 relative bias → ALiBi → RoPE → {YaRN, LongRoPE, iRoPE, NoPE-hybrids}, with RoPE as the convergent baseline since 2022. The decisive change is the move from *additive at input* to *multiplicative on Q, K at every layer*: the multiplicative form of (10)–(13) makes the positional information arithmetic-extensible, which is the property every post-2023 long-context extension method exploits.

The next axis (§5, attention KV-sharing) is largely *independent* of this one: a model can choose any of {MHA, MQA, GQA, MLA} on the head-sharing axis and any of {sinusoidal, RoPE, RoPE+YaRN, iRoPE} on the position axis without strong cross- constraints. The one notable interaction is DeepSeek-V2's *decoupled-RoPE* construction in MLA — where applying the rotation to a low-rank latent K projection breaks the cache-absorbed forward, and a separate small RoPE-only key channel is split out to preserve it (DeepSeek-AI, 2024a). We treat that interaction in detail in the next section.

5 Attention KV-sharing axis: MHA, MQA, GQA, MLA

The scaled dot-product attention function of Vaswani et al. (2017) is load-bearing for the entire Transformer, but *the function itself has not changed since 2017.* What has changed – in four discrete steps, each tied to a single primary source – is how the H query heads share the underlying K and V projections, and consequently how much memory the autoregressive KV cache consumes per token. **The thesis of this section: there is one axis, and four named points on it (MHA, MQA, GQA, MLA), and modern frontier-2026 LLMs all sit somewhere along that axis with the rest of the attention block held fixed.** The axis is parameterised by H_{kv}, the count of distinct key/value heads relative to the count of query heads H, and – in the MLA case – by a joint-compression rank d_c. We walk the four waypoints with the canonical equations transcribed verbatim, then assemble the cache-cost-vs-quality table that locates each variant on the curve.

5.1 Scaled dot-product, multi-head, the canonical 2017 form

Set notation. Let B be batch size, S the sequence length, D the residual-stream width, H the number of attention heads, and $d_h = D/H$ the per-head dimension. The residual-stream activation entering an attention block is $X \in \mathbb{R}^{B\times S\times D}$. Vaswani et al. (2017) define scaled dot-product attention[18] on per-head matrices $Q, K \in \mathbb{R}^{S\times d_h}$ and $V \in \mathbb{R}^{S\times d_h}$ as

$$\text{Attention}(Q,K,V) \;=\; \text{softmax}\!\left(\frac{QK^\top}{\sqrt{d_h}}\right)V \qquad \text{(Vaswani–Eq. 1)}$$

(Vaswani et al., 2017, §3.2.1, Eq. 1), where the $\sqrt{d_h}$ scaling keeps the pre-softmax logit variance independent of d_h at initialisation[19]. Multi-head attention runs Eq. (Vaswani–Eq. 1) H times in parallel on linearly projected inputs[20]:

$$\begin{aligned}\text{MultiHead}(X) &= \text{Concat}(\text{head}_1,\ldots,\text{head}_H)\,W^O,\\ \text{head}_i &= \text{Attention}(XW_i^Q,\; XW_i^K,\; XW_i^V),\end{aligned} \qquad \text{(Vaswani–Eq. 2–3)}$$

with $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{D\times d_h}$ and $W^O \in \mathbb{R}^{Hd_h\times D}$ (Vaswani et al., 2017, §3.2.2, Eq. 2–3). The Vaswani choice $d_k = d_v = d_h = D/H$ keeps total compute roughly equal to single-head full-width attention.

After projection, the attention tensor flow is $Q, K, V \in \mathsf{B}\times\mathsf{H}\times\mathsf{S}\times d_h$, the score $S = QK^\top/\sqrt{d_h}$ is $\mathsf{B}\times\mathsf{H}\times\mathsf{S}\times\mathsf{S}$, and the output is again $\mathsf{B}\times\mathsf{H}\times\mathsf{S}\times d_h$ before the W^O projection back to $\mathsf{B}\times\mathsf{S}\times\mathsf{D}$. The shape $\mathsf{B}\times\mathsf{H}\times\mathsf{S}\times d_h$ is the lens through which the rest of this section is read.

The KV cache and its size. During autoregressive decoding, the model emits one token at a time; for each new token at position t the freshly computed K_t, V_t are appended to caches $\mathcal{K}, \mathcal{V} \in \mathsf{B}\times\mathsf{H}\times\mathsf{S}\times d_h$ that store every prior step's keys and values, and the entire cache must be re-loaded from HBM at every step (Shazeer, 2019, §1, §2.4)[21]. The per-token, per-layer cache footprint at full MHA is

$$\text{cache}_{\text{MHA}} \;=\; 2\,H\,d_h\,b \quad \text{bytes per token per layer,} \qquad (17)$$

where b is the bytes-per-element of the cache dtype (2 for FP16/BF16, 1 for FP8, etc.); the factor 2 covers K and V. Multiplied across L layers, S sequence positions, and B concurrent sequences in a batch, the full cache size is $2 \cdot B \cdot S \cdot L \cdot H \cdot d_h \cdot b$. With $D = Hd_h$ this is also $2BSLDb$ – the cache scales linearly in both context length and model width, and at serving time it is the dominant memory consumer for long-context batches.

Shazeer (2019) formalises why this matters: the ratio of memory access to arithmetic operations during incremental decoding is $\Theta(n/d + 1/b)$ in his notation (sequence length n, model width d, batch b); "when $n \approx d$ or $b \approx 1$, the ratio is close to 1, causing memory bandwidth to be a major performance bottleneck on modern computing hardware" (Shazeer, 2019, §2.4)[22]. Modern GPU/TPU hardware has compute throughput one to two orders of magnitude above HBM bandwidth, so a memory-bound layer leaves arithmetic units idle. Reducing the cache footprint directly buys serving throughput at fixed hardware – this is the reward function the next three waypoints on the axis are optimising.

[18] KB excerpt: `kb/excerpts/vaswani2017.md#sec-3-2-1`.
[19] If q, k have i.i.d. unit-variance components, $\text{Var}(q\cdot k) = d_h$ (Vaswani et al., 2017, §3.2.1, fn. 4).
[20] KB excerpt: `kb/excerpts/vaswani2017.md#sec-3-2-2`.
[21] KB excerpt: `kb/excerpts/shazeer2019.md#sec-2-4`.
[22] KB excerpt: `kb/excerpts/shazeer2019.md#sec-2-4`.

5.2 Multi-Query Attention (MQA): one K and one V head

Shazeer (2019) proposes the simplest possible reduction: drop the H index from W^K and W^V entirely. Queries remain multi-headed; keys and values are shared across all H query heads[23]. Concretely, replace the per-head W_i^K, W_i^V in Eq. (Vaswani–Eq. 2–3) with single shared projections $W^K, W^V \in \mathbb{R}^{D \times d_h}$, and broadcast the resulting $K, V \in \mathsf{B} \times \mathsf{S} \times d_h$ across the H query heads at attention time:

$$\text{head}_i^{\text{MQA}} \;=\; \text{Attention}(XW_i^Q,\; XW^K,\; XW^V), \quad i = 1, \ldots, H. \tag{18}$$

The KV-cache shape collapses from $\mathsf{B} \times \mathsf{H} \times \mathsf{S} \times d_h$ to $\mathsf{B} \times \mathsf{S} \times d_h$ on each of K, V:

$$\text{cache}_{\text{MQA}} \;=\; 2\, d_h\, b \quad \text{bytes per token per layer}, \tag{19}$$

a factor-of-H reduction over Eq. (17). Shazeer's abstract phrases the trade-off plainly: MQA produces models that "can indeed be much faster to decode, and incur only minor quality degradation from the baseline" (Shazeer, 2019, abstract).

PaLM (Chowdhery et al., 2022) adopted MQA at scale; Falcon and a small set of 2022 production decoder-only models followed. But *minor quality degradation* did not stay minor. Ainslie et al. (2023) report that MQA suffers "quality degradation and training instability" (Ainslie et al., 2023, §1), and the failure mode is worst at the larger model scales where the cache-bandwidth saving is most welcome – because larger models scale H upward, so collapsing all H heads onto a single shared K, V becomes a more aggressive expressivity cut.

5.3 Grouped-Query Attention (GQA): G groups, interpolating

Ainslie et al. (2023) interpolate between MHA and MQA by introducing *groups*. Partition the H query heads into G disjoint groups of size H/G; within each group, all queries share a single K head and a single V head[24].

> "Grouped-query attention divides query heads into G groups, each of which shares a single key head and value head. GQA-G refers to grouped-query with G groups. GQA-1, with a single group and therefore single key and value head, is equivalent to MQA, while GQA-H, with groups equal to number of heads, is equivalent to MHA." (Ainslie et al., 2023, §2.2)

The K, V cache shape becomes $\mathsf{B} \times \mathsf{G} \times \mathsf{S} \times d_h$, and the per-token, per-layer cache footprint is

$$\text{cache}_{\text{GQA-}G} \;=\; 2\, G\, d_h\, b \quad \text{bytes per token per layer}. \tag{20}$$

GQA sweeps continuously from MHA ($G = H$, no reduction) to MQA ($G = 1$, factor-H reduction); intermediate G trades capacity for cache size on a smooth curve.

The GQA paper also contributes a quality-recovery recipe. "Uptraining" converts an existing MHA checkpoint to a GQA checkpoint by mean-pooling the original H key/value heads within each group, then continuing pre-training for a fraction $\alpha \approx 5\%$ of the original training compute[25]:

> "The projection matrices for key and value heads are mean pooled into single projection matrices, which we find works better than selecting a single key and value head or randomly initializing new key and value heads from scratch." (Ainslie et al., 2023, §2.1)

[23] KB excerpt: `kb/excerpts/shazeer2019.md#mqa-structure`.
[24] KB excerpt: `kb/excerpts/ainslie2023.md#sec-2-2`.
[25] KB excerpt: `kb/excerpts/ainslie2023.md#sec-2-1`.

On T5-XXL, GQA-8 (i.e., $G = 8$ groups out of $H = 64$ total query heads) achieves quality "comparable to MHA-XXL" at speed "comparable to MQA" (Ainslie et al., 2023, §3.2). Ainslie et al. (2023, §3.3) report that "increasing the number of groups from MQA only results in modest slowdowns initially, with increasing cost as we move closer to MHA", and select $G = 8$ as a favourable middle ground – a number that, in 2026, has become a near-universal default. LLaMA-2 70B introduced GQA-8; LLaMA-3 70B and 405B kept it; Mistral, Qwen-2.5/3, and Phi-4 ship GQA at varying group counts; the recipe of Ainslie et al. (2023) is the largest single point of architectural agreement among Western frontier LLMs (Touvron et al., 2023; Jiang et al., 2023; AI, 2024b; Team and Alibaba, 2025; et al. , Microsoft).

5.4 Multi-Head Latent Attention (MLA): low-rank joint compression

DeepSeek-AI (2024a) change the question. MQA and GQA both *select* a smaller number of K, V heads from a discrete set $\{1, \dots, H\}$. MLA instead *compresses* the joint K, V information at each token into a low-rank latent vector, then *reconstructs* per-head K_i, V_i at use time[26]. The cache holds the latent only; the per-head reconstruction matrices are weights, not cache.

Let $\mathbf{h}_t \in \mathbb{R}^D$ be the residual-stream slice at token t. MLA defines down-projection $W^{DKV} \in \mathbb{R}^{d_c \times D}$ and per-head up-projections $W^{UK}, W^{UV} \in \mathbb{R}^{Hd_h \times d_c}$ with $d_c \ll Hd_h$:

$$\mathbf{c}_t^{KV} = W^{DKV}\mathbf{h}_t, \tag{DSV2–Eq. 9}$$

$$\mathbf{k}_t^C = W^{UK}\mathbf{c}_t^{KV}, \tag{DSV2–Eq. 10}$$

$$\mathbf{v}_t^C = W^{UV}\mathbf{c}_t^{KV}, \tag{DSV2–Eq. 11}$$

(DeepSeek-AI, 2024a, §2.1.2, Eq. 9–11). The cache stores the latent $\mathbf{c}_t^{KV} \in \mathbb{R}^{d_c}$ – a single d_c-dim vector per token per layer, irrespective of H:

$$\text{cache}_{\text{MLA}}^{(\text{content})} = d_c\, b \quad \text{bytes per token per layer (content path).} \tag{21}$$

The content K and V heads $\mathbf{k}_{t,i}^C, \mathbf{v}_{t,i}^C$ are then *never materialised at inference*: because matrix multiplication is associative, W^{UK} can be absorbed into W_i^Q and W^{UV} into W^O (DeepSeek-AI, 2024a, §2.1.2), leaving an attention computation that reads $\mathbf{c}_t^{KV}$ directly. The reconstruction is bookkeeping, not a runtime cost.

Decoupled-RoPE: the wrinkle. The cache-absorption argument breaks if positional encoding is RoPE because RoPE inserts a per-position rotation $\boldsymbol{R}_{\Theta,m}^{d_h}$ between W_i^Q and W^{UK}, so the matrices no longer commute (cf. §4 on RoPE). DeepSeek-AI (2024a) resolve this by splitting each query and each key into a content part (no RoPE, low-rank) and a RoPE-bearing part (full position rotation, but small per-head dimension d_h^R and – crucially – a single *shared* RoPE key across heads)[27]:

$$\mathbf{q}_{t,i}^R \text{ from } \mathbf{q}_t^R = \text{RoPE}(W^{QR}\mathbf{c}_t^Q), \tag{DSV2–Eq. 14}$$

$$\mathbf{k}_t^R = \text{RoPE}(W^{KR}\mathbf{h}_t), \tag{DSV2–Eq. 15}$$

$$\mathbf{q}_{t,i} = [\mathbf{q}_{t,i}^C;\ \mathbf{q}_{t,i}^R], \tag{DSV2–Eq. 16}$$

$$\mathbf{k}_{t,i} = [\mathbf{k}_{t,i}^C;\ \mathbf{k}_t^R], \tag{DSV2–Eq. 17}$$

$$\mathbf{o}_{t,i} = \sum_{j=1}^{t} \text{softmax}_j\left(\frac{\mathbf{q}_{t,i}^\top \mathbf{k}_{j,i}}{\sqrt{d_h + d_h^R}}\right)\mathbf{v}_{j,i}^C, \tag{DSV2–Eq. 18}$$

[26] KB excerpt: `kb/excerpts/deepseek-v2.md#sec-2-1-2`.
[27] KB excerpt: `kb/excerpts/deepseek-v2.md#sec-2-1-3`.

(DeepSeek-AI, 2024a, §2.1.3, Eq. 14–18). Note the single $\mathbf{k}_t^R$ in Eq. (DSV2–Eq. 17): the RoPE-bearing key is shared across heads, MQA-style; only the content key $\mathbf{k}_{t,i}^C$ is per-head, and that head dimension is recovered from the latent at use time. The total per-token, per-layer cache footprint is therefore

$$\text{cache}_{\text{MLA}} \;=\; (d_c + d_h^R)\, b \quad \text{bytes per token per layer,} \tag{22}$$

(DeepSeek-AI, 2024a, §2.1.3). For DeepSeek-V2's setting $d_c = 4d_h$ and $d_h^R = d_h/2$, the cache is $(4 + 0.5)\, d_h\, b = 4.5\, d_h\, b$, and DeepSeek-AI (2024a) note this is "equal to GQA with only 2.25 groups" (DeepSeek-AI, 2024a, §2.1.4)[28] – substantially smaller than the GQA-8 default while, in their internal ablations, matching or exceeding MHA quality.

Intuition

MQA, GQA, and MLA are three answers to the same question: "how do H query heads share their K, V scratchpad?" MQA picks one shared (K, V) pair globally. GQA-G picks G shared pairs, one per group. MLA does not pick at all – it stores a single d_c-vector summary of (K, V) jointly, and *decompresses* on demand into per-head (K_i, V_i) via fixed weight matrices that the inference engine algebraically absorbs into W^Q and W^O. Returning to canonical math: MQA fixes cache $= 2d_h b$, GQA-G fixes cache $= 2Gd_h b$, MLA fixes cache $= (d_c + d_h^R)b$ with the per-head K, V never appearing in the cache at all. The first two cuts dimensions; the third refactors the storage.

5.5 Cache-cost-vs-quality tradeoff

The per-token, per-layer cache footprints assemble into one table. Capability ratings are taken verbatim from DeepSeek-AI (2024a, Table 1).

Variant	Cache / token / layer	Capability
MHA (Vaswani et al., 2017)	$2\, H\, d_h\, b$	Strong
GQA-G (Ainslie et al., 2023)	$2\, G\, d_h\, b$	Moderate–strong
GQA-8 (LLaMA-2/3 70B)	$16\, d_h\, b$	Moderate–strong
GQA-2	$4\, d_h\, b$	Moderate
MQA (Shazeer, 2019)	$2\, d_h\, b$	Weak
MLA (DeepSeek-AI, 2024a)	$(d_c + d_h^R)\, b \approx 4.5\, d_h\, b$	**Stronger** (DSV2-internal)

Table 3: KV-cache footprint per token per layer across the KV-sharing axis. Compute footprint of the attention function itself (FLOPs in Eq. (Vaswani–Eq. 1)) is unchanged at matched H, d_h; only the cache that backs autoregressive decoding moves. The $\approx 4.5 d_h b$ MLA value uses DeepSeek-V2's defaults $d_c = 4d_h, d_h^R = d_h/2$ (DeepSeek-AI, 2024a, §2.1.4).

The curve has a striking property: DeepSeek-AI (2024a)'s ablation puts MLA at GQA-2.25 cache footprint with MHA-or-better quality – i.e., *above* the MHA point on the quality dimension at *below* the GQA-8 point on the cache dimension. If reproducible, this dominates the entire MQA–GQA Pareto front and makes MLA the unambiguous choice. The contradiction below records why "if reproducible" has not yet collapsed.

[28] KB excerpt: `kb/excerpts/deepseek-v2.md#sec-2-1-4`.

Contradiction

The MLA-vs-MHA quality result at frontier scale is single-source as of 2026-Q2. The original ablation appears in DeepSeek-AI (2024a, §2.1.4) on a 16B-active-parameter DeepSeek-V2 prototype; DeepSeek-AI (2024b) extends MLA to a 671B total / 37B active model with strong downstream benchmarks but does not include a head-to-head MLA-vs-MHA at matched compute at that scale. No third-party laboratory has published an MLA reproduction at $\geq$ 70B-active scale on a non-DeepSeek training corpus. The mechanism behind the quality preservation – whether the latent joint compression is a regulariser, whether the per-head reconstruction matrices learn a richer space than direct K, V projections, whether the cache absorption interacts with pre-training loss curves – is not yet decomposed. Treat MLA's "stronger than MHA" claim as DeepSeek-internal-evidence-only at frontier scale; the GQA-8 default remains the safest reproducible recommendation outside the DeepSeek training stack (cf. Ainslie et al., 2023, §3.1).

5.6 Frontier-2026 choice landscape

Where on the axis do the leading 2026 models actually sit?

- **LLaMA-2 70B, LLaMA-3 8B/70B/405B, LLaMA-4 Scout/Maverick**: GQA, with $G = 8$ in the 70B class and proportionally adjusted elsewhere (Touvron et al., 2023; AI, 2024b, 2026). Adopted the Ainslie recipe at scale; preserved across three model generations.
- **Mistral 7B, Mixtral 8×7B, Mistral Large**: GQA throughout (Jiang et al., 2023).
- **Qwen-2.5 / Qwen-3**: GQA, with the group count adjusted per parameter scale (Team and Alibaba, 2025).
- **Gemma 2 / Gemma 3**: GQA (Team and DeepMind, 2025).
- **Phi-4**: GQA (et al. , Microsoft).
- **OLMo-2**: GQA in the standard configuration (Team, 2025).
- **DeepSeek-V2, DeepSeek-V3, DeepSeek-R1**: MLA throughout (DeepSeek-AI, 2024a,b, 2025). The Chinese open-weight ecosystem – including labs that train on the DeepSeek architecture as a baseline – has converged on MLA alongside DeepSeek; Western frontier labs have not.

The convergence pattern: **outside the DeepSeek training stack, GQA-8 is the 2026 default**; **inside the DeepSeek training stack, MLA is the 2026 default**. The rest of the attention block (RoPE positional encoding per §4, FlashAttention-style implementation per §5, RMSNorm + Pre-LN per §8) is held constant across both camps. KV-sharing is the single axis on which the Western and DeepSeek lineages diverge.

Why hasn't MLA crossed over? Three plausible reasons, each under-evidenced and worth recording. **(a) Training-recipe risk**: MLA changes both K/V representations and the RoPE plumbing simultaneously (Eqs. (DSV2–Eq. 9)–(DSV2–Eq. 18)); conservative labs prefer to swap one component at a time, and the GQA recipe of Ainslie et al. (2023) requires only a mean-pool of an existing checkpoint. **(b) Inference-stack support**: MLA's cache-absorption identity demands that the inference framework algebraically fold W^{UK} into W^{Q} and W^{UV} into W^{O} at serve time;

GQA needs only a head-broadcast in the standard attention kernel. As of 2026-Q2, FlashAttention, vLLM, and SGLang have first-class GQA support; MLA support exists but is more recently added and less battle-tested. **(c) The contradiction above**: if MLA's quality story were uncontested, the recipe risk would be paid; the absence of cross-lab confirmation makes the bet asymmetric for any lab whose downstream metric is benchmark quality rather than serving cost.

5.7 Closing: forward link to attention implementation

The KV-sharing axis is independent of the *attention implementation* axis. A model can pick any of {MHA, MQA, GQA, MLA} and any of {standard, FlashAttention-1/2/3, NSA} as two orthogonal choices: the first determines the cache shape and the per-head expressivity; the second determines how the same attention function (Eq. (Vaswani–Eq. 1)) is scheduled onto GPU memory hierarchy and tensor cores. The next section (§5) walks the hardware-implementation axis from Dao et al. (2022) through Shah et al. (2024) and Yuan et al. (2025), and shows where the two axes interact – in particular, how MLA's decoupled-RoPE wrinkle (Eqs. (DSV2–Eq. 14)–(DSV2–Eq. 17)) interacts with the FlashAttention block-tiling pattern, and why MLA serving stacks re-implement attention rather than reuse the off-the-shelf GQA kernel.

A separate forward reference: this section anchors the "compressed-KV" substrate that interpretability methods must contend with in Paper 4. The MLA latent $\mathbf{c}_t^{KV}$ is not a per-head representation; it is a joint compression. Probes, lenses, and SAEs trained on MLA-architecture activations are reading a fundamentally different geometry than the same methods applied to MHA or GQA models, and the cross-method circuit-tracing literature has not yet established whether the existing toolkit transfers. That tension is treated in Paper 4 §10 (cross-method circuits over compressed-KV models), with the present section as the architectural ground truth.

6 Attention hardware-implementation axis: FA1, FA2, FA3, NSA

§5 fixed the attention *formula* along the KV-sharing axis (MHA → MQA → GQA → MLA). This section fixes the formula — scaled dot-product attention as defined in Eq. (1) of Vaswani et al. (2017) — and varies the *implementation* on the GPU memory hierarchy. The thesis: between 2022 and 2025 the field walked FlashAttention 1 (block tiling + recomputation; Dao et al., 2022) → FlashAttention 2 (warp-level tile scheduling; Dao, 2023) → FlashAttention 3 (asynchronous + low-precision tensor-core scheduling on Hopper; Shah et al., 2024) → Native Sparse Attention (block-sparse with explicit selection; Yuan et al., 2025). The first three are *exact* reorderings of the canonical formula — bit-equivalent up to floating-point reduction order — and their wall-clock wins come entirely from reducing HBM traffic and matching the tensor-core schedule. NSA changes *which keys participate* (the per-branch $\tilde{K}_t \subset K$) but preserves the scaled-dot-product form on each branch; the gain comes from a $\sim$10$\times$ smaller N_t at 64K context. This distinction — exact reordering vs. approximate sparsification — is load-bearing when reasoning about quality.

6.1 Why naive attention is HBM-bound

The scaled dot-product attention block of §5 costs $\Theta(B \cdot H \cdot S^2 \cdot d_h)$ FLOPs and, in the naive implementation that materializes $\boldsymbol{S}, \boldsymbol{P} \in \mathbb{R}^{S \times S}$ in HBM, $\Theta(Nd + N^2)$ HBM accesses per head (with $N \equiv S$ for self-attention) (Dao et al., 2022, §3.2 Thm. 2).[29] The roofline diagnostic asks which side

[29] KB excerpt: `kb/excerpts/dao2022.md#sec-3-2`.

of the inequality

$$\underbrace{\frac{\text{FLOPs}}{\text{HBM bytes}}}_{\text{arithmetic intensity}\ \ I} \leqslant \underbrace{\frac{\text{peak FLOP/s}}{\text{peak HBM B/s}}}_{\text{ridge point}\ \ I^*} \tag{23}$$

the kernel sits on. Below the ridge ($I < I^*$), wall-clock is set by HBM bandwidth: increasing FLOPs is free until the ridge is reached. Dao et al. (2022) report the A100 ridge at $I^* \approx \frac{19.5\,\text{TF}}{1.5\text{–}2.0\,\text{TB/s}} \approx 10$ FLOPs/byte, and the H100 SXM ridge near $I^* \approx 990/3.35 \approx 295$ FLOPs/byte (Dao et al., 2022, §2.1)(Shah et al., 2024, §1).

For the naive attention block at half precision, the dominant memory movement is reading $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ once and reading/writing $\boldsymbol{S}, \boldsymbol{P}$ once each: $\sim 4Nd + 4N^2$ bytes for $\sim 4N^2 d$ FLOPs, giving $I \approx d/(1 + d/N) \approx d$ for $N \gg d$. With $d = d_h \in [64, 128]$, I sits well below both ridges, so the kernel is firmly memory-bound. Yuan et al. (2025) state the consequence directly for long-context decoding: "attention computation ... accounts for 70–80% of total latency when decoding 64K-length contexts" on softmax architectures (Yuan et al., 2025, §1).[30]

Intuition

"Attention is quadratic in S" is the pop summary, but the practical bottleneck is not the FLOP count — modern accelerators have FLOPs to spare. It is the HBM round-trip on the $S \times S$ score matrix. Returning to canonical form: peak utilization on a kernel with arithmetic intensity I FLOPs/byte is bounded above by min(peak FLOP/s, $I \cdot$ peak HBM B/s); for naive attention with $I \sim d_h \in [64, 128]$, the second term binds across A100, H100, and B200 — not the first. The four implementations in this section all attack the second term.

6.2 FlashAttention 1: tiling + recomputation

Dao et al. (2022) compute Eq. (1) of Vaswani et al. (2017) *exactly* in two steps: tile $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ into blocks of shapes $B_r \times d_h$ and $B_c \times d_h$ that fit in SRAM, and stream a numerically stable *online softmax* across $\boldsymbol{K}, \boldsymbol{V}$ tiles so that the full $S \times S$ matrix is never written to HBM.[31] The tile sizes are chosen from the SRAM budget M:

$$B_c \;=\; \left\lceil \frac{M}{4d_h} \right\rceil, \qquad B_r \;=\; \min\!\left(\left\lceil \frac{M}{4d_h} \right\rceil,\; d_h \right), \tag{24}$$

so that the four arrays $\boldsymbol{Q}_i, \boldsymbol{K}_j, \boldsymbol{V}_j, \boldsymbol{O}_i$ co-resident on chip total $\sim M$ bytes (Dao et al., 2022, Algorithm 1). On A100 with $M \approx 192$ KB and half-precision, this gives $B_c \approx B_r \approx 192$ at $d_h = 64$, ≈ 96 at $d_h = 128$.

The online softmax decomposition is the algebraic move that makes streaming work. For each query tile $\boldsymbol{Q}_i$, maintain running statistics $m_i \in \mathbb{R}^{B_r}$ (running row-wise max) and $\ell_i \in \mathbb{R}^{B_r}$ (running row-wise softmax denominator), updated on each $\boldsymbol{K}_j, \boldsymbol{V}_j$ tile by (Dao et al., 2022, §3.1)

$$\begin{aligned} m_i^{\text{new}} &= \max\!\big(m_i,\ \text{rowmax}(\boldsymbol{S}_{ij})\big), \\ \ell_i^{\text{new}} &= e^{m_i - m_i^{\text{new}}} \ell_i + \text{rowsum}\!\big(e^{\boldsymbol{S}_{ij} - m_i^{\text{new}}}\big), \\ \boldsymbol{O}_i^{\text{new}} &= \text{diag}(\ell_i^{\text{new}})^{-1} \Big(\text{diag}(\ell_i)\, e^{m_i - m_i^{\text{new}}}\, \boldsymbol{O}_i + e^{\boldsymbol{S}_{ij} - m_i^{\text{new}}} \boldsymbol{V}_j \Big), \end{aligned} \tag{25}$$

[30] KB excerpt: `kb/excerpts/yuan2025.md#sec-1`.
[31] KB excerpt: `kb/excerpts/dao2022.md#sec-3-1`.

where $\boldsymbol{S}_{ij} = \boldsymbol{Q}_i \boldsymbol{K}_j^\top / \sqrt{d_h}$ is computed on-chip from the resident tiles. After all $\lceil S/B_c \rceil$ key/value tiles have been streamed, $\boldsymbol{O}_i$ is the exact softmax-attention output for the query rows i, bit-equivalent to the naive computation up to floating-point reduction order.

The backward pass uses the second Dao et al. (2022) idea: *recomputation*. Instead of materializing $\boldsymbol{S}, \boldsymbol{P}$ in HBM during the forward pass and reading them back on the backward pass, FlashAttention stores only $\boldsymbol{O}$ and the per-row statistics $(m, \ell) \in \mathbb{R}^{S \times 2}$ (a factor-$d_h/2$ saving), and recomputes $\boldsymbol{S}, \boldsymbol{P}$ from $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ tiles in SRAM on the backward pass.[32] This is selective gradient checkpointing applied at the attention-kernel level: the forward FLOP count rises by a factor of ~ 2 on the backward recomputation, but HBM bytes drop from $\Theta(N^2)$ to $\Theta(N d_h)$, and the kernel moves up the roofline.

The IO complexity result is (Dao et al., 2022, Thm. 2): standard attention requires $\Theta(N d_h + N^2)$ HBM accesses, FlashAttention requires $\Theta(N^2 d_h^2 / M)$. For typical $d_h \in [64, 128]$ and $M \sim 100$KB, $d_h^2/M \sim 1/10$ to $1/6$, and Proposition 3 proves this is asymptotically optimal among exact algorithms. End-to-end wall-clock: $\sim 3\times$ on GPT-2 at $S = 1024$, $\sim 2.4\times$ on Long-Range Arena at $S = 1$K–4K, and the first Transformer to achieve better-than-chance on Path-X at $S = 16$K (Dao et al., 2022, §1).

6.3 FlashAttention 2: warp-level tile scheduling

Dao (2023) keep the FA1 algorithm and re-engineer the GPU work partition.

deepen-cite: dao2023 §3 (excerpt not yet in KB; cite "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning", Dao 2023, arXiv 2307.08691)

Three changes drive a $\sim 2\times$ wall-clock improvement on A100:

Outer loop over $\boldsymbol{Q}$, inner loop over $\boldsymbol{K}, \boldsymbol{V}$. FA1's outer loop iterates over $\boldsymbol{K}, \boldsymbol{V}$ tiles, requiring all warps in a threadblock to communicate via shared memory at each iteration to update $\boldsymbol{O}_i$. FA2 swaps the loop order: each threadblock owns one $\boldsymbol{Q}_i$ tile and streams $\boldsymbol{K}_j, \boldsymbol{V}_j$ in the inner loop, so the running $(m_i, \ell_i, \boldsymbol{O}_i)$ live in registers and no inter-warp softmax-reduction synchronization is needed.

Reduce non-matmul FLOPs. Tensor cores deliver $16\times$ the throughput of CUDA cores on matmul; non-matmul ops (softmax exponentiation, divisions, scaling) run on CUDA cores and become the bottleneck once matmul is well-utilized. FA2 defers the $\mathrm{diag}(\ell_i)^{-1}$ rescale of $\boldsymbol{O}_i$ in Eq. (25) to the *end* of the inner loop — the rescale only matters for the final output, not the running accumulation — and folds the $1/\sqrt{d_h}$ scale into the softmax-mask combine. Both moves reduce CUDA-core FLOPs per output element.

Parallelize over the sequence dimension. FA1 parallelizes across batch and heads but not across S. For long contexts at small batch (the inference and long-document training regimes), this underutilizes SMs. FA2 splits the sequence among threadblocks — each threadblock owns a contiguous chunk of $\boldsymbol{Q}_i$ rows — and exploits the fact that with the outer-Q loop the $\boldsymbol{O}_i$ updates are independent across query rows. This recovers full SM occupancy at $B \cdot H \ll (\#\text{SMs})$, the regime that dominates modern serving.

The combined effect on A100: $\sim 2\times$ over FA1, reaching ~ 50–73% of theoretical FP16 matmul peak vs. FA1's ~ 25–40%. The kernel arithmetic and IO complexity are unchanged from §6.2; the win is scheduling.

[32] KB excerpt: `kb/excerpts/dao2022.md#sec-3-1-recompute`.

6.4 FlashAttention 3: asynchrony and FP8 on Hopper

H100 / H200 (Hopper, SM_90a) introduces three hardware capabilities that A100's FA2 schedule cannot exploit (Shah et al., 2024): the WGMMA warpgroup matmul instruction (asynchronous, with one warpgroup — 4 warps, 128 threads — issuing a single matmul that releases the warpgroup to do other work while the tensor core executes), the TMA (Tensor Memory Accelerator, an asynchronous DMA engine for global ↔ shared memory transfers), and FP8 tensor-core matmul at 2× the FP16 peak FLOP rate.

deepen-cite: shah2024 §3 (excerpt not yet in KB; cite "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision", Shah, Bikshandi, Zhang, Thakkar, Ramani, Dao 2024, arXiv 2407.08608, NeurIPS)

FA3 redesigns the FA2 schedule around them.

Producer/consumer warp specialization. A threadblock's warps are split into a TMA-issuing producer group — which loads the next $\boldsymbol{K}_{j+1}, \boldsymbol{V}_{j+1}$ tile asynchronously into a shared-memory ring buffer — and one or two consumer warpgroups — which issue WGMMA on the current $\boldsymbol{K}_j, \boldsymbol{V}_j$ tile and run softmax. Producer and consumer execute concurrently; the latency of the next HBM load is hidden behind the matmul of the current tile.

Two-stage WGMMA + softmax interleaving. Within a consumer warpgroup, the two matmuls per tile ($\boldsymbol{Q}_i \boldsymbol{K}_j^\top$ to produce $\boldsymbol{S}_{ij}$, and the softmax-weighted $\boldsymbol{P}_{ij}\boldsymbol{V}_j$ to update $\boldsymbol{O}_i$) are issued asynchronously, and the softmax exponentiation/normalization on tile j runs in CUDA cores *while* the WGMMA for tile $j+1$ executes on tensor cores. Tensor-core and CUDA-core utilization rise simultaneously; the FA2 constraint that softmax pipelines serially with WGMMA is relaxed.

FP8 path with block-quantization. Hopper's FP8 tensor cores double FP16 throughput. Naive FP8 attention loses accuracy because softmax exponentiation amplifies the FP8 quantization error on the score matrix. FA3 quantizes $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ in *per-block* scales — each $B_c \times d_h$ key tile gets its own scaling factor stored alongside the tile — and de-quantizes lazily in the WGMMA accumulator. The reported numerical error on long-context attention is within 2× of the FP16 baseline on standard tasks (Shah et al., 2024).

The headline numbers: ∼1.5–2× over FA2 on H100 in FP16/BF16, reaching 75% of theoretical peak; an additional ∼1.5× from FP8 on top, with end-to-end quality preserved under per-block quantization. As of 2026-Q2, FA3 is the production path on Hopper for every frontier serving stack we have visibility into (vLLM, SGLang, TensorRT-LLM).

6.5 Native Sparse Attention (NSA)

The first three implementations preserve the attention formula exactly. NSA (Yuan et al., 2025) departs: at long context, it replaces the full-sequence keys/values $\boldsymbol{K}_{:t}, \boldsymbol{V}_{:t}$ with a sparse triple of branch-specific key/value sets,[33]

$$\boldsymbol{o}_t^* = \sum_{c \in \{\text{cmp,slc,win}\}} g_t^c \cdot \text{Attn}(\boldsymbol{q}_t, \tilde{K}_t^c, \tilde{V}_t^c), \tag{26}$$

where each branch applies the standard scaled-dot-product attention to a different summary of the past, $g_t^c \in [0,1]$ is a sigmoid gate derived from $\boldsymbol{q}_t$ via an MLP, and the total selected context $N_t = \sum_c |\tilde{K}_t^c|$ satisfies $N_t \ll t$.

[33] KB excerpt: `kb/excerpts/yuan2025.md#sec-3-2`.

Compression branch. A learned MLP φ maps each length-l block of keys to one compressed key, with stride $d < l$:

$$\tilde{K}_t^{\text{cmp}} \;=\; \left\{ \varphi(\boldsymbol{k}_{id+1:id+l}) \;\middle|\; 0 \le i \le \lfloor (t-l)/d \rfloor \right\} \tag{27}$$

(Yuan et al., 2025, §3.3.1). With $l = 32, d = 16$, an 8K context becomes $\sim$500 compressed keys — coarse-grained, but every position covered.

Selection branch. The compression-attention scores are reused as importance scores for selecting the top-n *full- resolution* blocks (Yuan et al., 2025, §3.3.2 Eq. 8–12):

$$\boldsymbol{p}_t^{\text{cmp}} \;=\; \text{softmax}(\boldsymbol{q}_t^\top \tilde{K}_t^{\text{cmp}}), \qquad \mathcal{I}_t \;=\; \{ i \,|\, \text{rank}(\boldsymbol{p}_t^{\text{slc}'}[i]) \le n \}, \tag{28}$$

with $\boldsymbol{p}_t^{\text{slc}'}$ summing per-head selection-importance scores over GQA-group heads (Eq. 10) so that all heads in a GQA group share the same selected blocks. Block size $l' = 64$, top-$n = 16$ blocks: 1024 selected keys per query.

Sliding-window branch. A small fixed-size window of the most recent keys (e.g. 512) handles local pattern formation in isolation (Yuan et al., 2025, §3.3.3), the explicit design rationale being that without this branch, compression and selection collapse onto local patterns and never learn long-range structure.

Group-centric kernel. The selection branch's irregular gather pattern would defeat FlashAttention's contiguous-tile loading. NSA inverts the loop: where FA2/3 load contiguous *query blocks* into SRAM, NSA loads all GQA-group heads at *one query position* into SRAM and pairs them with that position's selected sparse KV blocks (Yuan et al., 2025, §3.4). The GQA-group-shared selection (Eq. (28), Eq. 10 of the paper) makes a single sparse-block load serve all heads in the group, recovering the matmul arithmetic intensity that naive sparse attention loses.

The reported speedups at 64K context (Yuan et al., 2025, Fig. 1): 11.6× decode, 9.0× forward, 6.0× backward vs. FA2-on-GQA full attention, with NSA matching or exceeding full-attention quality on general / LongBench / reasoning benchmarks. The decode speedup matters most economically: at 64K, attention is 70–80% of decode latency (Yuan et al., 2025, §1), so an order-of-magnitude reduction in attention is roughly an order-of-magnitude reduction in total per-token serving cost in the long-context regime.

6.6 Sliding-window and ring-attention variants

Two further long-context strategies sit alongside the FA1–3 / NSA lineage and compose with it rather than replacing it.

Sliding-window attention. Mistral 7B (Jiang et al., 2023) uses a fixed window of $W = 4096$ keys per query, so the per-layer cost is $O(NW)$ rather than $O(N^2)$. At L layers, the *effective* receptive field at the top of the stack is $L \cdot W$ because each layer extends the window by another W via the residual stream — 32-layer Mistral 7B reaches a $\sim$131K-token effective receptive field with a per-layer 4K window. Sliding-window attention is implemented as a special case of FA2/3 (the score matrix is banded), so the implementation axis remains the same.

deepen-cite: jiang2023 §3.1 (sliding-window kernel) — excerpt not yet in KB

Ring attention. et al. (2023) shard the sequence axis across P devices: device p holds $\boldsymbol{Q}_p, \boldsymbol{K}_p, \boldsymbol{V}_p$ for S/P positions, and the $\boldsymbol{K}, \boldsymbol{V}$ tiles rotate around a logical ring of devices via async send/recv during the FA1-style outer loop. Each device runs FA-style on-chip tiling locally, so ring attention is FA1/2 *plus* a sequence-dimension shard with overlapped communication. Communication is hidden behind compute when $S/P \cdot d_h$ FLOPs per tile exceeds the inter-device transfer time; for $P \leq 64$ on H100/H200 this holds at any practically interesting S.

deepen-cite: ring-attention-2023 §3 — excerpt not yet in KB; cite Liu, Zaharia, Abbeel 2023, arXiv 2310.01889

Both compose orthogonally: ring-attention shards *sequence*, FA3 optimizes the *kernel* on each shard, NSA can replace the per-shard kernel with the three-branch sparse attention. The hardware-implementation axis is composable in this sense, which is one reason it stays distinct from the formula-changing axes of §4 and §5.

Contradiction

NSA-vs-FA3 head-to-head benchmarks are single-source as of 2026-Q2. Yuan et al. (2025) report 11.6× decode at 64K, but the comparison baseline is FA2-on-GQA, not FA3 with the H100 asynchrony + warp-specialization advantages. No published third-party reproduction on Hopper has compared NSA's three-branch sparse attention to FA3 on matched hardware, matched precision, and matched task suite as of 2026-05; community indications are that the gap narrows substantially when FA3 is the baseline and the context length is closer to 32K than 64K, but no peer-reviewed measurement has settled the question. Treat "NSA $\gg$ dense FlashAttention at long context" as a DeepSeek-internal result subject to revision until cross-replicated (Yuan et al., 2025).

6.7 Summary and forward link

The hardware-implementation axis is a separate axis from §5's KV-sharing axis, with one empirical asymmetry: FA1/2/3 work with any KV-sharing variant (MHA / MQA / GQA, and an MLA-specific kernel exists), but NSA is designed for GQA/MQA and would lose its group-centric kernel optimization (Yuan et al., 2025, §3.4) on MHA. The four implementations form a progression in which three properties co-vary: (i) HBM traffic, which falls from $\Theta(N^2)$ to $\Theta(N^2 d_h^2/M)$ between naive and FA1, then is unchanged through FA2/3 but masked behind asynchronous overlap, then falls again to $\Theta(N N_t d_h^2/M)$ under NSA's $N_t \ll N$ block selection; (ii) tensor-core utilization, which rises from $\sim$25% (naive FA1 path on A100) to $\sim$75% (FA3 on H100); and (iii) precision, which descends from FP16 to FP8 under FA3's per-block quantization without quality loss on standard tasks.

What does *not* change across this axis is the formula. Up to the NSA branch decomposition (Eq. (26)), every implementation in this section is computing $\mathrm{softmax}(QK^\top/\sqrt{d_h})V$ on the same Q, K, V that the KV-sharing axis (§5) and the positional axis (§4) produced. This is the practical reason axes §4, §5, and this section can be reasoned about independently in the convergence story of paper 1: a model can swap RoPE for YaRN, MHA for MLA, and FA2 for FA3 + NSA without any of those choices interfering with the others, because the formula at which they agree — scaled dot-product attention — is the fixed point.

The next section (§7) leaves attention and walks the feed-forward axis: ReLU → GELU → SwiGLU dense, then dense → Switch → Mixtral → DeepSeekMoE fine-grained-with-shared experts. The FFN axis is where the second half of the per-layer parameter budget sits, and where the routing-vs-dense convergence question of frontier-2026 is most contested.

7 Feed-forward axis: dense, GLU, MoE, fine-grained MoE

The feed-forward sublayer is where the bulk of a Transformer's parameters live and, since 2021, where the bulk of *architectural optimization* has happened. Two axes of choice run in parallel: the *activation/gating structure* of a single FFN block (ReLU → GELU → SwiGLU) and the *multiplicity* of FFN blocks per layer (dense → Switch top-1 MoE → Mixtral top-2 MoE → DeepSeekMoE fine-grained top-k with a shared expert). The frontier of 2026 is the Cartesian product: every routed expert is a SwiGLU block, and the layer wraps E such experts with a learned router and a load-balancing discipline. This section walks both axes, transcribes the load-bearing equations from the primary sources, and surfaces the single most contested empirical claim — whether sparse MoE quality matches dense quality at matched active-parameter count.

7.1 Dense FFN: the canonical 2017 form

The original Transformer's position-wise feed-forward sublayer is a two-matrix MLP applied independently at each token position. For input token vector $\mathbf{x} \in \mathbb{R}^{\mathsf{D}}$, Vaswani et al. (2017) define[34]

$$\mathrm{FFN}(\mathbf{x}, W_1, W_2, b_1, b_2) \;=\; \max(0, \mathbf{x}W_1 + b_1)\, W_2 + b_2, \qquad \text{(Vaswani–Eq. 2)}$$

with $W_1 \in \mathbb{R}^{\mathsf{D}\times d_{\mathrm{ff}}}$, $W_2 \in \mathbb{R}^{d_{\mathrm{ff}}\times \mathsf{D}}$, and the canonical inner ratio $d_{\mathrm{ff}} = 4\mathsf{D}$ (Vaswani et al., 2017, §3.3). Two structural features matter. First, FFN is *position-wise*: tensor flow is $\mathsf{B}\times\mathsf{S}\times\mathsf{D} \to (\mathsf{B},\mathsf{S},d_{\mathrm{ff}}) \to \mathsf{B}\times\mathsf{S}\times\mathsf{D}$, with no inter-position mixing — all sequence-axis communication has happened in the attention sublayer of section 5. Second, the FFN is the *parameter-heaviest* sublayer in a typical block: at $d_{\mathrm{ff}} = 4\mathsf{D}$ each FFN holds $8\mathsf{D}^2$ parameters versus $4\mathsf{D}^2$ for the four attention projections.

Radford et al. (2019) swap ReLU for GELU in GPT-2 and drop biases in the T5/decoder-only era; Shazeer (2020) writes the bias-free form as (Shazeer, 2020, §1, Eq. 2)

$$\mathrm{FFN}_{\mathrm{ReLU}}(\mathbf{x}, W_1, W_2) \;=\; \max(\mathbf{x}W_1, 0)\, W_2, \qquad \text{(Shazeer–Eq. 2)}$$

$$\mathrm{FFN}_{\mathrm{GELU}}(\mathbf{x}, W_1, W_2) \;=\; \mathrm{GELU}(\mathbf{x}W_1)\, W_2. \qquad \text{(Shazeer–Eq. 3)}$$

The activation swap leaves the parameter count and tensor flow identical; what changes is the unit-step. ReLU's hard zero cuts off gradient on negative pre-activations entirely; GELU's $x \cdot \Phi(x)$ smooths the cutoff and admits a small negative contribution near zero, which empirically aids optimization (Radford et al., 2019, §2.3).

7.2 GLU and SwiGLU: the gated FFN

Shazeer (2020) reports that swapping the FFN for a Gated Linear Unit variant yields a small but consistent perplexity gain on T5-base. The GLU layer (Dauphin et al. 2016, as cited by Shazeer, 2020, §2) is the component-wise product of two linear projections, one of which is sigmoid-activated (Shazeer, 2020, §2, Eq. 4)[35]:

$$\mathrm{GLU}(\mathbf{x}, W, V, b, c) \;=\; \sigma(\mathbf{x}W + b) \otimes (\mathbf{x}V + c), \qquad \text{(Shazeer–Eq. 4)}$$

with $W, V \in \mathbb{R}^{\mathsf{D}\times d_{\mathrm{ff}}}$ and $\otimes$ the elementwise (Hadamard) product. The activation in the gate position is a free choice; Shazeer (2020, §2, Eq. 5) lists five variants[36] including the bilinear

[34] KB excerpt: `kb/excerpts/shazeer2020.md#sec-1`; the 2017 form is also transcribed at `kb/notes/architecture/ffn.md#sec-1-1`.

[35] KB excerpt: `kb/excerpts/shazeer2020.md#sec-2`.

[36] KB excerpt: `kb/excerpts/shazeer2020.md#sec-2`.

($\sigma \to \mathrm{id}$), the ReGLU ($\sigma \to \mathrm{ReLU}$), the GEGLU ($\sigma \to \mathrm{GELU}$), and the SwiGLU ($\sigma \to \mathrm{Swish}_\beta$, with $\mathrm{Swish}_\beta(z) = z\,\sigma(\beta z)$).

Bundling the gating with the down-projection yields the bias-free 3-matrix FFN that every modern frontier LLM ships (Shazeer, 2020, §2, Eq. 6)[37]:

$$\mathrm{FFN}_{\mathrm{SwiGLU}}(\mathbf{x}, W, V, W_2) \;=\; \big(\mathrm{Swish}_1(\mathbf{x}W) \otimes (\mathbf{x}V)\big)\, W_2, \qquad \text{(Shazeer–Eq. 6)}$$

with $\mathrm{Swish}_1(z) = z\,\sigma(z)$ (also called SiLU). Per token the tensor flow now uses three projections of width d_{ff} rather than two, so to compare against Eq. (Vaswani–Eq. 2) at matched parameter count Shazeer reduces the inner width by a factor of $\frac{2}{3}$ (Shazeer, 2020, §2):

> "All of these layers have three weight matrices, as opposed to two for the original FFN. To keep the number of parameters and the amount of computation constant, we reduce the number of hidden units d_{ff} ... by a factor of $\frac{2}{3}$ when comparing these layers to the original two-matrix version." (Shazeer, 2020, §2)

This is the origin of the modern $d_{\mathrm{ff}} \approx \frac{8}{3}\mathsf{D}$ rule. For LLaMA-1 7B, $\mathsf{D} = 4096$ gives target $d_{\mathrm{ff}} = 10923$, rounded to 11008 for hardware alignment (Touvron et al., 2023).

The empirical motivation is Shazeer (2020, §3.2, Tab. 1)[38]: at 524k T5-base steps the heldout-set log-perplexities are 1.677 (ReLU), 1.679 (GELU), 1.683 (Swish), 1.633 (GEGLU), and 1.636 (SwiGLU). GEGLU and SwiGLU edge each other within 0.003; the field selected SwiGLU on the inertial strength of LLaMA-1's choice.

Intuition

Vaswani's FFN (Eq. (Vaswani–Eq. 2)) computes a sum of features $\mathrm{ReLU}(\mathbf{x}W_1)W_2 = \sum_k \mathrm{ReLU}(\mathbf{x}\cdot W_1[:,k])\, W_2[k,:]$ — a sparse weighted sum of "value rows" selected by which "key rows" the input activates. SwiGLU (Eq. (Shazeer–Eq. 6)) computes a *product* of features: $\mathrm{Swish}_1(\mathbf{x}W) \otimes (\mathbf{x}V)$. The gating factor $\mathrm{Swish}_1(\mathbf{x}W)$ multiplies the value $\mathbf{x}V$ component-wise, which expresses "this feature, only when that feature is also active" patterns the additive form cannot. Shazeer (2020, §4) closes with the well-known disclaimer — "We offer no explanation as to why these architectures seem to work; we attribute their success, as all else, to divine benevolence" — so the canonical form Eq. (Shazeer–Eq. 6) is the only honest endpoint of the intuition.

7.3 Mixture of Experts: Switch and Mixtral

The dense FFN's parameter cost grows with D^2 at fixed depth. Mixture of Experts (MoE) breaks the coupling: replace the single FFN at depth ℓ with E parallel FFN *experts*, learn a router that picks $k \ll E$ experts per token, and pay compute for k FFN evaluations while holding E times the parameters. Per-token compute scales with k; parameter capacity scales with E.

Let $\mathbf{x}_t \in \mathbb{R}^{\mathsf{D}}$ be the token's residual-stream slice and $\{\mathrm{FFN}_e\}_{e=1}^{E}$ a family of SwiGLU blocks (Eq. (Shazeer–Eq. 6)). The router is a learned linear projection $W_g \in \mathbb{R}^{\mathsf{D}\times E}$ producing per-expert

[37] KB excerpt: `kb/excerpts/shazeer2020.md#sec-2-eq6`.
[38] KB excerpt: `kb/excerpts/shazeer2020.md#sec-3-2`.

logits, with top-k selection and a softmax-renormalized gating weight (Fedus et al., 2021, §2.1)[39]:

$$\mathbf{g}_t = W_g\,\mathbf{x}_t \quad \in \mathbb{R}^E, \tag{29}$$

$$\mathcal{T}_t = \mathrm{TopK}_k(\mathbf{g}_t) \subset \{1,\dots,E\}, \tag{30}$$

$$p_{t,e} = \frac{\exp(g_{t,e})}{\sum_{e'\in\mathcal{T}_t}\exp(g_{t,e'})}, \quad e \in \mathcal{T}_t, \tag{31}$$

$$\mathrm{MoE}(\mathbf{x}_t) = \sum_{e\in\mathcal{T}_t} p_{t,e}\,\mathrm{FFN}_e(\mathbf{x}_t). \tag{32}$$

Switch Transformer (Fedus et al., 2021) fixes $k = 1$: the router sends each token to exactly one expert, $|\mathcal{T}_t| = 1$, and $p_{t,e} = 1$. The choice maximally simplifies routing arithmetic and minimizes communication — there is exactly one all-to-all dispatch per layer. Switch-XXL has $E \approx 64$ experts and ∼395B total parameters at ∼12B active.

Mixtral (et al. , Mistral AI) fixes $k = 2$, $E = 8$: each token's output is the gated combination of *two* expert FFNs. Mixtral-8×7B reports 46.7B total parameters, 12.9B active. Both designs leave the dense SwiGLU FFN unchanged *within* each expert; the router and the combination Eq. (32) are the additions.

Analogy

The MoE layer is a soft committee of specialists. The router inspects each token and dispatches to the few experts most relevant; the chosen experts produce candidate updates $\mathrm{FFN}_e(\mathbf{x}_t)$, and the gate weights $p_{t,e}$ blend them. The committee is "soft" because the weights in Eq. (31) are continuous; the dispatch is "hard" because the top-k selection Eq. (30) is discrete. Returning to math: the layer's output is exactly the gated weighted sum $\sum_{e\in\mathcal{T}_t} p_{t,e}\,\mathrm{FFN}_e(\mathbf{x}_t)$ of Eq. (32); the analogy survives because each expert is a full SwiGLU FFN of the form Eq. (Shazeer–Eq. 6), not a parameter-shared head.

7.4 Routing collapse and the load-balancing loss

The router in Eq. (29)–(31) is differentiable through $p_{t,e}$ on selected experts but *discrete* through Eq. (30). Two failure modes follow.

Routing collapse. Without intervention, the router learns to send most tokens to a small set of "favored" experts; under-used experts receive vanishing gradient and never improve, while over-used experts get richer updates. The dynamic is self-reinforcing.

Load imbalance. Even without total collapse, GPUs hosting cold experts sit idle while GPUs hosting hot experts oversubscribe. Under expert parallelism (one expert per device), throughput is bounded by the slowest device.

The standard mitigation is an auxiliary load-balancing loss $\mathcal{L}_{\text{aux}}$, added to the language-modeling loss with coefficient α. For a batch of T tokens, define the empirical fraction f_e of tokens routed to

[39] KB note: `kb/notes/architecture/moe.md#sec-1`; the equation form is verified against Fedus et al. (2021) for top-1 routing and et al. (Mistral AI) for top-2 routing.

deepen-cite: fedus2021-switch §2.1 (router-and-gate equations) and mixtral2024 §2.2 (top-2 form) — KB excerpt files not yet ground-truthed against the PDFs.

expert e and the mean router probability P_e for expert e (Fedus et al., 2021, §2.2)[40]:

$$f_e = \frac{1}{T} \sum_{t=1}^{T} \mathbb{1}[e \in \mathcal{T}_t], \tag{33}$$

$$P_e = \frac{1}{T} \sum_{t=1}^{T} \frac{\exp(g_{t,e})}{\sum_{e'=1}^{E} \exp(g_{t,e'})}, \tag{34}$$

$$\mathcal{L}_{\text{aux}} = \alpha \cdot E \sum_{e=1}^{E} f_e \, P_e. \tag{35}$$

The product $f_e P_e$ is differentiable through P_e even though f_e depends on the discrete selection Eq. (30), and is minimized at uniform load $f_e = P_e = 1/E$. Mixtral and Switch both use the form Eq. (35); production stacks pair it with a *capacity factor* $C \in [1, 2]$ that caps each expert at $\lceil C\,Tk/E \rceil$ tokens per batch and drops (residual-only passes) tokens routed past the cap (Fedus et al., 2021, §2.3).

DeepSeek-AI (2024b, §2) report that high-α auxiliary losses hurt LM quality at scale, and propose an *auxiliary-loss-free* alternative: maintain a per-expert *bias* b_e that is added to the router logits at selection time but *not* to the gate weight, and update the bias online based on observed load (DeepSeek-AI, 2024b)[41]:

$$\mathbf{g}'_t \;=\; W_g\,\mathbf{x}_t + \mathbf{b}, \qquad b_e \;\leftarrow\; b_e + \gamma\,(\bar{f} - f_e), \quad \bar{f} = 1/E, \tag{36}$$

with γ a small step size. Overloaded experts get b_e decreased so future tokens go elsewhere; underloaded experts get b_e increased. Since $\mathbf{b}$ shifts the *selection* logits but not the renormalized gate $p_{t,e}$ in Eq. (31), the contribution magnitudes stay clean. DeepSeek-V3 reports this strategy recovers the LM quality that high-α auxiliary loss would sacrifice (DeepSeek-AI, 2024b, abstract).

7.5 DeepSeekMoE: fine-grained experts and a shared expert

DeepSeek-AI (2024c) push two changes against the Switch/Mixtral design point: *fine-grained experts* (many narrow experts in place of few wide ones) and a *shared expert* that processes every token (DeepSeek-AI, 2024c)[42].

Fine-grained segmentation. A coarse Mixtral expert at $d_{\text{ff}} = \frac{8}{3}\mathsf{D}$ is replaced by m narrower experts of width d_{ff}/m; the top-k selection becomes top-mk. Active per-token compute is held fixed

[40] KB note: `kb/notes/architecture/moe.md#sec-2-1`.

deepen-cite: fedus2021-switch §2.2 — the auxiliary-loss form below is verified against secondary sources; the original PDF section number should be re-checked before final.

[41] KB note: `kb/notes/architecture/moe.md#sec-2-3`.

deepen-cite: deepseek-v3 §2 (auxiliary-loss-free balancing scheme) — claim ground-truthed against the V3 tech report's training-side excerpt at `kb/excerpts/deepseek-v3-training.md#abstract`, but the exact equation form requires the architecture-side excerpt that is not yet built.

[42] KB note: `kb/notes/architecture/moe.md#sec-3-1,#sec-3-2`.

deepen-cite: deepseekmoe2024 §3 (fine-grained segmentation: split each expert into m narrower experts of width d_{ff}/m and select mk rather than k) — excerpt file not yet built; arithmetic of combinatorial set count $\binom{E}{k}$ verified against the DeepSeek-V2/V3 reported E, k.

(the m scaling cancels), but the number of distinct activated combinations $\binom{E}{mk}$ grows combinatorially. Concretely: Mixtral has $\binom{8}{2} = 28$ active sets; DeepSeek-V2 with $E = 160$, $k = 6$ has $\binom{160}{6} \approx 2 \times 10^{10}$; DeepSeek-V3 with $E = 256$, $k = 8$ has $\binom{256}{8} \approx 4 \times 10^{14}$. The argument is that combinatorial expressiveness of the active set — which functions the layer can dynamically assemble from its experts — scales with this count, so finer experts at fixed active compute give more *flexible specialization* per token.

Shared expert as residual. Layered alongside the E routed experts is a small set E_s of *shared* experts that process *every* token unconditionally. With $E_s = 1$ (the DeepSeek-V3 choice) the layer becomes

$$\mathrm{MoE}^{\mathrm{DS}}(\mathbf{x}_t) \;=\; \mathrm{FFN}_{\mathrm{shared}}(\mathbf{x}_t) \;+\; \sum_{e \in \mathcal{T}_t} p_{t,e}\, \mathrm{FFN}_e(\mathbf{x}_t), \tag{37}$$

with $|\mathcal{T}_t| = k$ routed experts selected by Eq. (29)–(31) and the shared expert $\mathrm{FFN}_{\mathrm{shared}}$ contributing unconditionally. The shared term is structurally a *residual within the FFN sublayer*: it provides the always-on common-knowledge component that the routed experts no longer have to redundantly learn. The motivation is that without a shared expert, each routed expert is forced to re-learn token-level features that every token needs (basic syntax, general vocabulary), wasting capacity that could be spent on specialization. Eq. (37) factorizes that into "shared + specialized."

Frontier-2026 instances. DeepSeek-V3 ships $E = 256$ routed, $E_s = 1$ shared, $k = 8$ active, totaling 9 active experts per token from 257 total per layer (DeepSeek-AI, 2024b, abstract, §2)[43]. With 671B total parameters and 37B activated per token, the parameter-to-active ratio is $\approx 18{:}1$ — the largest such ratio shipped at frontier as of 2026-Q2. Team and Alibaba (2025) ship $E = 128$, $E_s = 0$, $k = 8$ in Qwen3-235B with 22B active, deliberately dropping shared experts and replacing them with a global-batch load-balancing loss (Team and Alibaba, 2025).

7.6 Frontier-2026 choice landscape

The dense-vs-MoE split in 2026 is no longer a question of *whether* to do MoE at frontier scale — the answer is yes — but of *how*. Three concrete design points coexist in publicly-disclosed frontier models (AI, 2024b, 2026; DeepSeek-AI, 2024b; Team and Alibaba, 2025; et al. , Mistral AI):

- **Dense at moderate scale.** LLaMA-3 8B and 70B remain fully dense SwiGLU FFNs at every layer (AI, 2024b); the open-weights ecosystem retains a single dense codepath through the small/medium tier.
- **Coarse MoE.** Mixtral 8×7B and 8×22B keep $E = 8$, $k = 2$, no shared expert, classical auxiliary-loss balancing (et al. , Mistral AI); every layer is MoE.
- **Fine-grained MoE with shared expert.** DeepSeek-V3 with $E = 256$, $E_s = 1$, $k = 8$, auxiliary-loss-free balancing, every FFN sublayer except the first is MoE (DeepSeek-AI, 2024b); the V2/V3 lineage is the most aggressive parameter-to-active ratio shipped.
- **Fine-grained MoE without shared expert.** Qwen3 with $E = 128$, $E_s = 0$, $k = 8$, global-batch load balancing (Team and Alibaba, 2025).
- **LLaMA-4 hybrid.** Scout and Maverick alternate dense and MoE FFN sublayers, with $E = 16$ (Scout) or $E = 128$ (Maverick), $E_s = 1$, $k = 1$ (AI, 2026).

[43] KB excerpt: `kb/excerpts/deepseek-v3-training.md#abstract`.

The two structural primitives common to every MoE in this list — SwiGLU experts and a learned softmax router with top-k — are the same primitives Switch fixed in 2021. What evolved is the *granularity* (E from 8 to 256), the *shared-expert* position, and the *load-balancing* discipline (aux-loss → aux-loss-free → global-batch).

Contradiction

The empirical claim that fine-grained MoE matches dense quality at *matched active-parameter count* is the most load-bearing and least-replicated assertion in this section. DeepSeek-AI (2024b) report DeepSeek-V3 (37B active) tracking or exceeding LLaMA-3 405B (dense) on benchmark suites (DeepSeek-AI, 2024b, abstract); DeepSeek-AI (2024c) make a similar matched-active-FLOPs claim at smaller scale. Independent third-party reproductions at frontier scale are sparse as of 2026-Q2: most public head-to-head numbers come from the DeepSeek/Qwen labs themselves or from secondary aggregators. Team and Alibaba (2025) explicitly disagree with DeepSeek-AI (2024b) on whether shared experts are net beneficial, dropping E_s to zero and reporting comparable quality with global-batch balancing. Treat the "MoE matches dense at matched active params" claim as *strongly supported by primary sources, weakly cross-validated by independent reproductions* until 2027-class non-DeepSeek/Qwen frontier MoE training runs publish their gap numbers.

deepen-cite: matched-active MoE-vs-dense quality at frontier scale — no published independent third-party head-to-head as of 2026-Q2.

7.7 Forward link

Two structural decisions of this section — SwiGLU as the per-expert FFN and a softmax-routed top-k wrapper around it — are now independent of the surrounding block. The next axis (section 8) is what surrounds the FFN in the residual stream: which normalization is applied, and whether it sits before the sublayer (Pre-LN), after it (Post-LN), or wraps it (Peri-LN). The frontier-2026 stack composes a routed SwiGLU MoE of eq. (37) with the RMSNorm + Pre-LN + $1/\sqrt{N}$-init bundle of section 8; the FFN choices made here forward-determine which normalization placements remain stable, because the residual stream's per-block variance budget is shared between attention and FFN sublayers (Xiong et al., 2020).

8 Normalization

The Transformer's normalization layer evolved along two orthogonal axes between 2017 and 2026: the *type* of normalization (LayerNorm vs RMSNorm) and the *placement* of the norm relative to the residual addition (Post-LN vs Pre-LN, plus stabilizer variants such as DeepNorm). The frontier-2026 consensus across LLaMA, Mistral, DeepSeek, and Qwen is **RMSNorm + Pre-LN with $1/\sqrt{N}$ residual scaling at initialization**; the parameters of this consensus are the smallest non-trivial slice of the architecture, and each piece has a directly attributable primary source.

8.1 LayerNorm: the canonical 2016 definition

LayerNorm normalizes across the feature axis of a single example, so that the same statistics are computed at training and inference and batch size has no effect. For an input vector $\mathbf{a}^l \in \mathbb{R}^H$ (the

summed inputs to the H hidden units of layer l), Ba et al. (2016) define[44]

$$\mu^l = \frac{1}{H}\sum_{i=1}^{H} a_i^l, \qquad \sigma^l = \sqrt{\frac{1}{H}\sum_{i=1}^{H}(a_i^l - \mu^l)^2}, \tag{Ba–Eq. 3}$$

and apply a per-feature gain $\mathbf{g}$ and bias $\mathbf{b}$ after the normalization but before the non-linearity:

$$\bar{\mathbf{a}}^l \;=\; \frac{\mathbf{g}}{\sigma^l} \odot (\mathbf{a}^l - \mu^l) + \mathbf{b}. \tag{38}$$

Two structural properties follow directly from Eq. (Ba–Eq. 3)–(38): (i) all hidden units in a layer share the same μ^l, σ^l but *different training cases* get different normalization terms (Ba et al., 2016, §3); (ii) LayerNorm is invariant under re-scaling of the entire weight matrix and under re-centering of the incoming weights (Ba et al., 2016, §4, Table 1). The second invariance is what Zhang and Sennrich (2019) will later argue is dispensable.

In modern decoder-only notation, H is the model dimension D and $\mathbf{a}^l$ is a single token's residual stream slice at depth l; the normalization is applied independently per token, per layer. The shape passing through is $\mathsf{B}\times\mathsf{S}\times\mathsf{D}$.

8.2 RMSNorm: dropping the mean

Zhang and Sennrich (2019) hypothesize that LayerNorm's success rests on its *re-scaling* invariance, not its *re-centering* invariance, and propose Root Mean Square Layer Normalization[45] defined verbatim as

$$\bar{a}_i \;=\; \frac{a_i}{\mathrm{RMS}(\mathbf{a})}\, g_i, \qquad \mathrm{RMS}(\mathbf{a}) \;=\; \sqrt{\frac{1}{n}\sum_{i=1}^{n} a_i^2}. \tag{Zhang–Eq. 4}$$

Comparing Eq. (Zhang–Eq. 4) to Eq. (Ba–Eq. 3): RMSNorm drops the mean μ^l entirely (and with it the bias $\mathbf{b}$), keeping only the scale parameter $\mathbf{g}$. Two reductions over the feature axis become one; one element-wise add disappears.

Empirically, dropping the mean is "free." Zhang and Sennrich (2019) report matched quality across machine translation, reading comprehension, summarization, and image classification, with runtime savings of **7%–64%** depending on architecture and norm placement (Zhang and Sennrich, 2019, §4); on Transformer-base WMT'14 EN–DE the swap costs 0.05 BLEU and saves 9.4% of training time (Zhang and Sennrich, 2019, §4). The shift-invariance that LayerNorm afforded turns out not to be load-bearing in practice (Zhang and Sennrich, 2019, §3.2), and every frontier-2026 decoder-only LLM (LLaMA-1/2/3, Mistral, Qwen, DeepSeek-V2/V3, Gemma, OLMo, Phi-4) ships RMSNorm rather than LayerNorm (Touvron et al., 2023; Jiang et al., 2023; AI, 2024b; DeepSeek-AI, 2024b; Team and Alibaba, 2025; Team, 2025; Team and DeepMind, 2025).

8.3 Post-LN versus Pre-LN

Independent of *which* norm you compute, the question of *where* you compute it is a separate axis. Writing F for a sublayer (attention or FFN) and $\mathbf{x}$ for its input, the canonical placements are

$$\text{Post-LN} \quad \mathbf{y} \;=\; \mathrm{Norm}(\mathbf{x} + F(\mathbf{x})) \tag{39}$$

$$\text{Pre-LN} \quad \mathbf{y} \;=\; \mathbf{x} + F(\mathrm{Norm}(\mathbf{x})). \tag{40}$$

[44] KB excerpt: `kb/excerpts/ba2016-layernorm.md#sec-3`.
[45] KB excerpt: `kb/excerpts/zhang2019.md#sec-3-1`.

Vaswani et al. (2017) used Post-LN (39) in the original Transformer; the norm sits *on* the residual path, so the residual stream itself is renormalized after every block. Radford et al. (2019) moved the norm in GPT-2, in a passage worth quoting verbatim[46]:

> "Layer normalization (Ba et al., 2016) was moved to the input of each sub-block, similar to a pre-activation residual network (He et al., 2016) and an additional layer normalization was added after the final self-attention block. A modified initialization which accounts for the accumulation on the residual path with model depth is used. We scale the weights of residual layers at initialization by a factor of $1/\sqrt{N}$ where N is the number of residual layers." (Radford et al., 2019, §2.3)

Two architectural decisions are bundled in that paragraph: the Pre-LN move (40), and the $1/\sqrt{N}$ residual-path scaling at initialization. The first changes the placement; the second compensates for the fact that an unnormalized residual stream's variance grows with depth. Both are still in effect in frontier-2026 models, with N typically the count of attention + FFN sublayers (i.e., $2L$ for an L-block decoder-only stack).

8.4 The Pre-LN stability proof

Xiong et al. (2020) furnish the theoretical justification for the GPT-2 move via a mean-field analysis of gradient norms at initialization[47]. Stating their two main results:

Theorem 8.1 (Post-LN gradient scaling at init, Xiong et al., 2020, Thm. 1)**.** *For the Post-LN Transformer with standard Xavier initialization, the expected Frobenius gradient norm at the final FFN sublayer is*

$$\|\partial \mathcal{L}/\partial W^{2,L}\|_F \;=\; O\Big(d\sqrt{\ln d}\Big)\,,$$

independent of depth L. The gradient propagated through the residual stream to earlier layers accumulates an additional $O(\sqrt{L})$ near the output: parameter updates near the output layer become disproportionately large as L grows.

Theorem 8.2 (Pre-LN gradient scaling at init, Xiong et al., 2020, Thm. 2)**.** *For the Pre-LN Transformer with the same initialization, the expected gradient norm at every layer is*

$$\|\partial \mathcal{L}/\partial W^{l}\|_F \;=\; O\left(d\sqrt{\ln d/L}\right),$$

balanced across depth and bounded as $L \to \infty$.

The practical consequence (Xiong et al., 2020, §4.1, §5): Post-LN training requires the linear-warmup schedule $\mathrm{lr}(t) = (t/T_{\text{warmup}})\,\mathrm{lr}_{\max}$ to compensate for the output-layer gradient blow-up, while Pre-LN trains stably *without warmup* at the same maximum learning rate. Their WMT'14 and BERT-pretraining experiments confirm the prediction: Post-LN with $\mathrm{lr}_{\max} = 10^{-3}$ and no warmup diverges, but the same setting with $T_{\text{warmup}} = 4000$ trains stably; the Pre-LN counterpart trains stably at $\mathrm{lr}_{\max} = 10^{-3}$ with no warmup at all (Xiong et al., 2020, §5). Theorem 8.2 is the load-bearing reason every 2020+ decoder-only LLM is Pre-LN.

[46] KB excerpt: `kb/excerpts/radford2019-gpt2.md#sec-2-3`.
[47] KB excerpt: `kb/excerpts/xiong2020.md#sec-4`.

8.5 DeepNorm and beyond

A natural follow-up question is whether one could keep the Post-LN placement (which has its own expressivity arguments at very deep stacks) but rescale the residual contribution to defang Theorem 8.1. Wang et al. (2022)'s DeepNet answers yes: introduce a per-block constant $\alpha > 1$ on the residual addition,

$$\mathbf{y} \;=\; \mathrm{Norm}(\alpha\mathbf{x} + F(\mathbf{x})), \tag{41}$$

with α and the sublayer initialization scale chosen as functions of depth L such that the residual stream's variance and the gradient's expected norm both stay $O(1)$ at initialization for L as large as 1000.

deepen-cite: wang2022-deepnorm §3 (DeepNorm scaling constants α, β as functions of L for encoder-only, decoder-only, encoder-decoder)

DeepNorm thus generalizes the Pre-LN / $1/\sqrt{N}$ trick to a regime where Pre-LN's residual-stream-norm drift becomes a problem on its own, and demonstrates that the 1000-layer regime is reachable. It is used in some Microsoft tech-report models but has not displaced Pre-LN at frontier scale; modern frontier LLMs run at $L = 32$–126 layers (DeepSeek-AI, 2024b; AI, 2024b) where Pre-LN is comfortable and the DeepNorm machinery is not yet necessary.

Intuition

LayerNorm is "make every token's residual stream have the same energy along the feature axis, then re-scale per-channel." Energy of $\mathbf{a}$ along the feature axis is $\sum_i a_i^2$, so RMSNorm (Eq. (Zhang–Eq. 4)) is the same statement with the offset removed: divide each component by $\sqrt{(1/n)\sum_i a_i^2}$ so the resulting vector has unit RMS, then apply a learned per-channel gain g_i. The mean-subtraction in LayerNorm pre-projects onto the $\mathbf{1}^\perp$-hyperplane; if the model doesn't use that projection's invariance (Zhang & Sennrich's §3.2 finding), dropping it costs nothing. Returning to math: RMSNorm preserves $\mathrm{RMS}(\bar{\mathbf{a}}) = \|\mathbf{g}\|_2/\sqrt{n}$; the energy of the normalized stream is fixed by $\mathbf{g}$ alone.

8.6 Why frontier-2026 settled here

Composing the four moves: take LayerNorm (Eq. (Ba–Eq. 3)–(38)), drop the mean and the bias to get RMSNorm (Eq. (Zhang–Eq. 4)); place the norm *before* each sublayer per Eq. (40); scale the residual-path weights at init by $1/\sqrt{N}$ per Radford et al. (2019, §2.3); rely on Theorem 8.2 for warmup-free trainability. The total cost on top of the canonical 2017 Transformer is one bias parameter removed, one mean-subtraction removed, one norm relocated, and one initialization scale changed – each backed by a primary source. Touvron et al. (2023)'s LLaMA-1 specification adopts the full bundle, and every subsequent LLaMA-/Mistral-/DeepSeek-/Qwen-class model has shipped some variant of it (Jiang et al., 2023; AI, 2024b; DeepSeek-AI, 2024b; Team and Alibaba, 2025), with two open splits: 2025+ models add QK-Norm to the attention path (Team, 2025; covered in §5 of this paper), and a separate stream switches to Peri-LN for very-deep stacks (Team, 2025; Team and DeepMind, 2025; various, 2025c). Both are extensions of, not replacements for, the RMSNorm + Pre-LN + $1/\sqrt{N}$ baseline established between 2016 and 2020.

The next section (§9) leaves the attention-and-FFN-with-residuals architecture entirely and asks what happens when the residual stream is replaced by a state-space recurrence – a setting in which the normalization choices of this section largely transfer (state-space-model-based LLMs use RMSNorm and analogues of Pre-LN), but the gradient-scaling argument of Theorem 8.2 no longer applies because the residual structure is gone.

9 State-space models and hybrids: the alternative path

The architectural axes of §4–§8 all live inside the attention-and-FFN-with-residuals scaffold inherited from Vaswani et al. (2017). A different lineage drops that scaffold for the residual stream while keeping it for the per-token mixing function: the *state-space model* (SSM) replaces the attention block's $\mathrm{softmax}(QK^\top)V$ with a *recurrent state* $\boldsymbol{h}_t \in \mathbb{R}^d$ that summarizes the prefix in fixed size. The thesis of this section is that the SSM lineage – S4/S5 → Mamba → Mamba-2 – has produced (i) an architecture with $O(Nd^2)$ training cost and $O(d^2)$ per-token decode, (ii) a theorem (Dao and Gu, 2024's State Space Duality) that recasts a class of SSMs as structurally restricted attention, and (iii) a family of *hybrid* models that interleave SSM and attention blocks. As of mid-2026, no pure-SSM model has been deployed at frontier scale; the deployed long-context architectures are either attention-based or SSM-attention hybrids, and the gap between hybrid demos and frontier attention models has not been closed by third-party reproduction.

9.1 The state-space recurrence form

A linear continuous-time SSM of dimension d on scalar input $u(t) \in \mathbb{R}$ is the classical control-theory system

$$\frac{d\boldsymbol{h}(t)}{dt} \;=\; A\,\boldsymbol{h}(t) + B\,u(t), \qquad y(t) \;=\; C\,\boldsymbol{h}(t) + D\,u(t), \tag{42}$$

with $A \in \mathbb{R}^{d\times d}$, $B \in \mathbb{R}^{d\times 1}$, $C \in \mathbb{R}^{1\times d}$, $D \in \mathbb{R}$.

deepen-cite: gu2023-mamba §2 (KB excerpt missing; verify notation against PDF Eq. (1a-1b))

Discretization with step Δ via zero-order hold yields the discrete recurrence used at training and inference time:

$$\boldsymbol{h}_t \;=\; \bar{A}\,\boldsymbol{h}_{t-1} + \bar{B}\,u_t, \qquad y_t \;=\; C\,\boldsymbol{h}_t, \tag{Mamba–Eq. 2a}$$

with $\bar{A} = \exp(\Delta A)$ and $\bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I)\,(\Delta B)$.

deepen-cite: gu2023-mamba §2.1 (KB excerpt missing; verify discretization formula)

Two properties of (Mamba–Eq. 2a) make it attractive as a sequence mixer. First, applied to a length-N sequence the recurrence costs $O(Nd^2)$ flops – *linear* in N where attention is quadratic. Second, for fixed $\bar{A}, \bar{B}, C$ the output equals a 1-D convolution of u with a kernel $\bar{K} = (C\bar{B},\, C\bar{A}\bar{B},\, C\bar{A}^2\bar{B}, \ldots)$, so $y_{1:N}$ can be computed in $O(N \log N)$ via FFT during training while still streamable as an RNN at decode time – the *convolution-recurrence dual* that Gu and Dao (2023) inherit from S4.

The S4 line (, no KB entry;

deepen-cite: gu2022-s4 (Gu, Goel, Ré 2022): HiPPO-structured A matrix, Eq. (2)–(3) of S4 paper

) chooses A to be a HiPPO operator that "optimally compresses" input history under an orthogonal-polynomial basis; S5

deepen-cite: smith2023-s5 (no KB entry): diagonal-plus-low-rank A replacing S4's HiPPO-NPLR parameterization, Eq. (4)

simplifies the parameterization to a diagonal A with multi-input-multi-output recurrence. Both share (Mamba–Eq. 2a); both rely on the convolution-recurrence dual to get parallel training. In modern decoder-only notation, the SSM mixer takes a tensor of shape $\mathsf{B} \times \mathsf{S} \times \mathsf{D}$ and applies (Mamba–Eq. 2a) independently along each of the D feature channels with a shared (or per-channel) parameterization.

9.2 Selective SSMs: Mamba

Gu and Dao (2023)'s "Mamba" breaks the convolution-recurrence dual deliberately. The SSM (Mamba–Eq. 2a) is *linear-time-invariant*: $\bar{A}, \bar{B}, C$ do not depend on u_t, so the same kernel $\bar{K}$ applies to every position. This means the SSM cannot *select* which inputs to incorporate vs. ignore – a problem on associative-recall, copying, and induction-head tasks where the model must conditionally route information based on content. Mamba's fix: make $\bar{A}, \bar{B}, C$ *input-dependent* via per-token linear projections $s_B(\cdot), s_C(\cdot), s_\Delta(\cdot)$ of the current input,

$$\bar{B}_t \;=\; s_B(u_t), \quad C_t \;=\; s_C(u_t), \quad \Delta_t \;=\; \tau_\Delta(s_\Delta(u_t)), \tag{43}$$

with A kept as a learned diagonal matrix and Δ_t passed through a softplus τ_Δ.

deepen-cite: gu2023-mamba §3.1 (KB excerpt missing; verify Eq. (4) and the s_B, s_C, s_Δ parameterization)

Substituting into (Mamba–Eq. 2a) gives

$$\boldsymbol{h}_t \;=\; \bar{A}_t\, \boldsymbol{h}_{t-1} + \bar{B}_t\, u_t, \qquad y_t \;=\; C_t\, \boldsymbol{h}_t, \tag{Mamba–Eq. 2b}$$

where $\bar{A}_t, \bar{B}_t$ now vary with position. The *selection mechanism* is Δ_t: large Δ_t means the recurrence "forgets" more of $\boldsymbol{h}_{t-1}$ and absorbs more of u_t, while small Δ_t preserves state. Per-token control of Δ_t lets Mamba implement input-dependent gating analogous to the update gate of an LSTM, but with the recurrence's matrix form intact.

The cost is that (Mamba–Eq. 2b) is no longer a convolution (the kernel changes per position), so the FFT-based parallel training recipe of S4/S5 fails. Mamba recovers training-time GPU parallelism via a hardware-aware *parallel scan* (Gu and Dao, 2023, §3.3).

deepen-cite: gu2023-mamba §3.3 (KB excerpt missing; verify the selective-scan algorithm and the SRAM-tiled implementation described as "hardware-aware")

The scan uses an associative combination of the per-position transition tuples $(\bar{A}_t, \bar{B}_t u_t)$ to compute all $\boldsymbol{h}_{1:N}$ in $O(\log N)$ depth on GPU, and avoids materializing the full state by keeping the scan in SRAM. The Mamba block wraps the selective SSM in a gated-MLP envelope (a SiLU-gated linear unit, with a residual addition and an RMSNorm at the input) (Gu and Dao, 2023, §3.4) so the unit is drop-in-replaceable for an attention + FFN sublayer pair.

Intuition

The state $\boldsymbol{h}_t \in \mathbb{R}^d$ is a fixed-size *compression buffer* for the prefix $u_{1:t}$. Each new u_t overwrites part of the buffer; Δ_t controls how much. Attention, by contrast, keeps every prior $u_{1:t}$ in addressable form via the KV cache and pays $O(N)$ memory for it. The SSM's buffer is $O(d^2)$ regardless of N – cheap, but lossy: any token whose information has been overwritten is unrecoverable. Returning to math: per-token decode cost is $O(d^2)$ for the SSM (one application of (Mamba–Eq. 2a)) versus $O(N d_h)$ for attention (one query against the cache). The crossover where SSM is cheaper is at $N \sim d^2/d_h$, which for $d \approx 16$ (Mamba's state width) and $d_h = 128$ is on the order of a few tokens – the SSM is essentially always cheaper at decode.

9.3 State Space Duality: Mamba-2

Dao and Gu (2024)'s contribution is the *State Space Duality* (SSD): a theorem-and-framework showing that a class of selective SSMs and a restricted form of attention compute the same function. Concretely, the authors show that the input-output map of a Mamba-2 block can be written

$$\boldsymbol{y} \;=\; M\, \boldsymbol{u}, \qquad M \in \mathbb{R}^{N \times N} \text{ lower-triangular, 1-semi-separable of rank} \le d, \tag{44}$$

where $\boldsymbol{u} = u_{1:N}$ is the stacked input and M is a lower-triangular *semi-separable* matrix with rank at most the SSM state width d.

deepen-cite: mamba2 §3 (KB excerpt missing; verify the SSD theorem statement, the semi-separable rank bound, and the duality between the recurrent form (Mamba–Eq. 2b) and the matrix form (44))

The semi-separable structure both (i) recovers $O(Nd^2)$ flops via a structured matmul that reuses FlashAttention-style block tiling, and (ii) frames Mamba-2 as a vanilla attention with a structurally restricted attention matrix. The reported 2–8× wall-clock speedup over Mamba-1 comes from this matmul-friendly form.

deepen-cite: mamba2 §6 (KB excerpt missing; benchmarks vs Mamba-1 and FlashAttention-2)

Analogy

Mamba-2 is "attention with a bandwidth budget." Vanilla causal attention computes a dense lower-triangular $N \times N$ matrix $A_{ij} = \text{softmax}(QK^\top)_{ij}$ with no rank constraint; FlashAttention-class kernels just compute it efficiently. Mamba-2 constrains the analogous matrix M in (44) to be rank-$\leq d$ semi-separable – the same recurrent state's information bottleneck rewritten in attention notation. The continuum is: vanilla attention (unconstrained) → sliding-window attention (banded) → Mamba-2 (rank-restricted). Returning to math: the cost of attention is $O(N^2 d_h)$ flops to fill the unconstrained $N \times N$ matrix; the cost of Mamba-2 is $O(Nd \cdot d)$ flops to apply the rank-d factorization (44). The bandwidth budget d is exactly the ratio of the two.

The SSD theorem matters less for raw speed (FlashAttention-3 closes much of that gap on H100/H200; cf. §**??**) than for taxonomy: it tells us that the practical distinction between attention and SSM is not "recurrent vs. parallel" but rather *which structural restriction on the attention matrix the architecture imposes.* This is the theoretical lens under which hybrids stop being weird and start looking like a deliberate mix of low- and high-rank attention layers.

9.4 Hybrid architectures

The hybrid lineage drops the all-or-nothing question – "SSM or attention?" – and treats the decoder stack as a place to interleave both. The layout question becomes: at what ratio, in what pattern, with what bridge between the two block types? Four representative designs illustrate the choices:

Jamba (AI21, 2024). et al. (AI21 Labs) interleave *1 attention layer per 7 Mamba layers* on a 12B active / 52B total MoE backbone, with full causal attention on the attention layers (using GQA; §5) and Mamba-1 selective SSMs on the SSM layers.

deepen-cite: lieber2024-jamba §3 (KB excerpt missing; verify the 1:7 ratio, the MoE configuration, and the placement of attention layers within the stack)

The 1:7 ratio is enough attention to cover recall-style needs (associative-recall, induction, copying) while keeping the cheaper SSM substrate dominant; AI21 report comparable quality to similar-active-parameter dense Transformers at lower KV-cache cost.

Samba (Microsoft, 2024). A finer interleaving: SSM and sliding-window attention alternate at a near-1:1 ratio, with the attention restricted to a local window rather than full causal. The design assumes the SSM substrate handles long-range mixing while the local-window attention handles short-range exact-recall.

deepen-cite: ren2024-samba (no KB entry; verify the alternation pattern, the window size, and the small-LM scale at which Samba is reported)

Zamba2 (Zyphra, 2024). A different bridge: a Mamba-2 backbone with two *shared* attention blocks applied periodically across the stack. The shared-block design lets a single set of attention weights act at multiple depths, amortizing the attention parameter cost; the authors report a $\sim 6\times$ KV-cache reduction relative to Zamba1.

deepen-cite: zamba2-2024 (no KB entry; verify the shared-attention-block mechanism, the backbone-layer count, and the 6× KV-cache claim)

Hymba (NVIDIA, 2024). A *parallel* rather than serial hybrid: each block has SSM heads and attention heads operating on the same input in parallel, with the outputs concatenated before the FFN. The design avoids the serial attention-bottleneck on long sequences (the attention layer in a Jamba stack must wait for all 7 prior SSM outputs at decode) at the cost of running both mixers per block.

deepen-cite: hymba-2024 (no KB entry; verify the parallel-head architecture, the head-count split between SSM and attention, and the small-LM benchmark scale)

The interleaving ratio is the key knob. Jamba's 1:7 says attention is *rare-but-essential*; Samba's ∼1:1 says local attention is worth pairing with every SSM layer; Zamba2's shared-attention says one set of attention weights amortizes across depth; Hymba's parallel design says compute both mixers always and let the concat-and-project step choose. None of these are head-to-head compared at matched parameters/compute in published work, and all four are at sub-frontier scale (12B active is the largest deployed example, et al. (AI21 Labs)).

Speculation

Why the 1:7 hybrid ratio is roughly right. An attention layer once every ~ 8 tokens of stack-depth gives the model enough exact-recall capacity to handle the rare positions where the SSM substrate cannot, while keeping the cheaper SSM dominant for the routine residual-stream work. The optimal ratio likely depends on the task distribution: code probably benefits from more attention, narrative summarization benefits from more SSM. The published lineage has not run that ratio sweep at frontier scale, so this remains a plausible heuristic rather than a derived constant.

9.5 Why pure SSM has not won at frontier

Three obstacles keep pure SSM models from displacing attention at the frontier as of mid-2026, all traceable to the fixed-state information bottleneck of (Mamba–Eq. 2a).

In-context learning and exact recall. A pure SSM compresses the entire prefix into $\boldsymbol{h}_t \in \mathbb{R}^d$ ($d \approx 16$ for Mamba; $d \approx 128$ for some follow-ups). Tasks that require *exact retrieval* of a specific prior token – copying, hashing, multi-hop induction, needle-in-haystack – expose this bottleneck: by the time the relevant token's information matters, intervening updates may have overwritten it.

deepen-cite: mamba2 §7, jamba2024 ablations (KB excerpts missing; both papers' ablations show pure-Mamba below Transformer baselines on associative-recall and needle-in-haystack at matched compute)

This is the empirical motivation for hybrids: an attention layer is the straightforward fix to the recall bottleneck, and one or two attention layers per stack already close most of the gap.

Retrieval-style task brittleness. Beyond pure recall, tasks that depend on attending to specific tokens based on *content* (retrieval-augmented QA, code-completion that references a specific function name, multi-document reasoning) degrade faster on pure SSM than on attention as context grows. The forum-signal consensus is that this is the real ceiling, but it has not been quantitatively reported at frontier scale.

deepen-cite: tier-B benchmarks: HuggingFace open-LLM leaderboard 2025 entries for Falcon-Mamba 7B vs Llama-class 7B at matched compute, no primary paper

Training-data efficiency. A subtler claim made informally in the Mamba-2 follow-up literature

deepen-cite: mamba2 §7 (KB excerpt missing; verify whether the paper makes a quantitative training-token-efficiency claim or only an inference-FLOPs claim)

: pure SSMs at matched parameters require *more* pretraining tokens to reach the same loss as Transformers, because the recall-limited mixer makes some training signal harder to absorb. If true, this changes the cost calculus: the SSM's training flop savings are partly offset by needing more tokens.

Contradiction

Quality of SSM-attention hybrids at frontier scale (the 100B-active range, where DeepSeek-V3, Llama-4, and the Claude/GPT-class systems operate) is an open question as of 2026-Q2. Vendor demos (et al. , AI21 Labs at 52B total / 12B active; Zamba2 at 7B; Hymba at 1.5B) are not third-party-reproduced at frontier scale, and no controlled head-to-head exists between (Jamba 1:7), (Samba ∼1:1), (Zamba2 shared-attention), and (Hymba parallel) at matched parameters/compute. Dao and Gu (2024)'s SSD theorem is *evidence* that a hybrid stack with a small attention budget should be Pareto-better than pure SSM, but the optimal interleaving pattern, ratio, and attention-block count remain settled by neither theory nor the publicly-deployed model zoo.

The frontier-2026 deployed long-context architectures (DeepSeek-V3 (DeepSeek-AI, 2024b), Llama-4 (AI, 2024b), Qwen3 (Team and Alibaba, 2025), Mistral Large (Jiang et al., 2023)) are all attention-based; none ship an SSM block. The forum signal is that hybrids will appear at scale eventually – the SSD theorem and the demonstrated long-context FLOP advantage are real – but the public leaderboard as of 2026-05 contains no SSM-hybrid model in the top quality tier.

9.6 What this section settles, and what comes next

The state-space line is the most theoretically distinct alternative to the attention scaffold of §4–§8, and it has produced a real architectural family with a real cost advantage on long sequences. Two facts are stable: (i) selective SSMs (Mamba–Eq. 2b) are linear-time per-token mixers with $O(d^2)$ decode cost regardless of N; (ii) the SSD theorem (44) unifies a class of SSMs with rank-restricted attention. Two facts are open: (a) whether hybrid stacks scale to frontier quality, and (b) what the optimal interleaving pattern is. The conclusion is that as of mid-2026 the field has *retained* attention as the high-rank exact-recall mechanism, and added SSM blocks where cheap context-mixing is the binding constraint – but only in secondary models, not in the deployed frontier.

The next section (§10) returns to the attention scaffold and asks a different question: at the *output* side of the decoder, what changes when the model predicts multiple tokens at once? Multi-Token Prediction (Gloeckle et al., 2024; productionized in DeepSeek-AI, 2024b) is an architectural choice on the output head, orthogonal to the SSM-vs-attention question of this section, and to the KV-sharing axis of §5.

10 Multi-token output: speculation, MTP, and parallel decoding

The architectural axes catalogued in sections 3 to 9 all assume the canonical autoregressive decoding loop: one forward pass produces one token. That assumption is load-bearing for both training mathematics (the next-token cross-entropy of section 3) and serving cost (the per-step KV / weight read of section 5). The 2023–2026 literature breaks this one-pass-one-token coupling along two distinct axes that the field sometimes conflates. *Speculative decoding* (Leviathan et al., 2023; Chen et al., 2023) is a serving-side algorithm that produces multiple tokens per target-model forward pass while preserving the target's output distribution exactly. *Multi-token prediction* (MTP) (Gloeckle et al., 2024) is an architectural and training-objective choice that adds heads predicting tokens at offsets $t+1, \ldots, t+n$, reporting both quality lift on code-heavy benchmarks and (when the heads are reused at inference) a built-in self-speculative drafter. The thesis of this section is the distinction: speculative decoding is *inference-only* and never touches the training loss; MTP is *architectural* and changes both the loss landscape and what the deployable system can do. They compose — a DeepSeek-V3-style MTP-trained model (DeepSeek-AI, 2024b, §2.3.1) is its own draft model in a Leviathan-style verification loop — but they are not interchangeable.

10.1 Speculative decoding: the rejection rule

Let M_p be a target LLM with conditional distribution $p(x_t \mid x_{<t})$ over a vocabulary of size V, and M_q a smaller draft model with $q(x_t \mid x_{<t})$. Leviathan et al. (2023) sample γ draft tokens $x_1, \ldots, x_\gamma$ from M_q autoregressively, run M_p in a single parallel forward over the $\gamma+1$ extended prefixes, then accept or reject each draft sequentially using (Leviathan et al., 2023, §2.3)[48]:

$$\text{accept } x_i \text{ with prob } \min\left(1, \frac{p_i(x_i)}{q_i(x_i)}\right), \quad \text{else resample } x_i \sim \text{norm}(\max(0, p_i - q_i)). \tag{45}$$

The proof in Leviathan et al. (2023, App. A.1) shows that any token emitted by eq. (45) is i.i.d. from p *regardless of* M_q. The output distribution is not approximated; it is preserved. This is what distinguishes speculative decoding from earlier parallel-decoding heuristics (block-parallel decoding, soft-decoding ensembles), which trade quality for speed.

Define the per-token acceptance rate $\alpha = \mathbb{E}_{x\sim q}[\min(1, p(x)/q(x))]$. Under the i.i.d. assumption, the expected number of tokens emitted per algorithm call is the capped geometric (Leviathan et al., 2023, §3.1, Eq. 1)[49]

$$\mathbb{E}[\#\text{tokens}] = \frac{1-\alpha^{\gamma+1}}{1-\alpha}, \tag{46}$$

and the expected walltime improvement, with cost coefficient c equal to the ratio of M_q's per-step cost to M_p's, is (Leviathan et al., 2023, §3.3, Thm. 3.8)

$$\text{walltime gain} = \frac{1-\alpha^{\gamma+1}}{(1-\alpha)(\gamma c+1)}. \tag{47}$$

A key analytic result is Leviathan et al. (2023, Thm. 3.5, Cor. 3.6): $\alpha = \mathbb{E}[\min(p,q)] = 1 - \mathbb{E}[D_{LK}(p,q)]$, where D_{LK} is total-variation distance. The speedup curve is therefore a function of how close q is to p in TV. Reported α on T5-XXL with a T5-Small drafter is ~ 0.75, yielding 2–3× wallclock speedup at $\gamma = 7$ (Leviathan et al., 2023, §3.6, Tab. 3). Concurrent work (Chen et al., 2023) derives the same rule and is treated as joint prior in the field.

[48] KB excerpt: `kb/excerpts/leviathan2023.md#sec-2-3`.
[49] KB excerpt: `kb/excerpts/leviathan2023.md#sec-3-1`.

10.2 Self-speculation without a drafter: Medusa

The drafter-design problem is operational: a separate M_q requires its own training, versioning, and quantization, and a too-small drafter has low α while a too-large drafter has high c. Medusa (Cai et al., 2024) eliminates the auxiliary model by appending K small *decoding heads* to the target backbone's last hidden state h_t. The k-th head predicts the token at position $t+k+1$ (Cai et al., 2024, §2.1.1)[50]:

$$p_t^{(k)} = \text{softmax}\Big(W_2^{(k)} \cdot \big(\text{SiLU}(W_1^{(k)} h_t) + h_t\big)\Big), \qquad W_1^{(k)} \in \mathbb{R}^{D\times D},\ W_2^{(k)} \in \mathbb{R}^{D\times V}, \tag{48}$$

with $W_2^{(k)}$ initialized identically to the original LM head and $W_1^{(k)}$ initialized to zero, so each head starts as a pass-through of the unmodified next-token distribution. To exploit multiple drafts per step, Medusa proposes a *tree* of candidates: each head emits its top-s_k tokens, and candidates are formed by Cartesian product over heads (Cai et al., 2024, §2.1.2)[51], evaluating $\sum_k \prod_i s_i$ continuations in one parallel target forward. *Tree attention* is the mechanism that makes this a single pass: the attention mask is constructed so each candidate token attends only to its unique ancestor chain in the tree, not to its siblings, which shares the prefix's KV state across all branches.

Two training regimes: Medusa-1 freezes the backbone and trains heads with the weighted cross-entropy $\mathcal{L}_{\text{Medusa-1}} = \sum_{k=1}^{K} -\lambda_k \log p_t^{(k)}(y_{t+k+1})$ with $\lambda_k = 0.8^k$ (Cai et al., 2024, §2.2.1, Eq. 1); the original LM head is unchanged, so the model's next-token distribution is preserved exactly under eq. (45). Medusa-2 unfreezes the backbone with the joint loss $\mathcal{L}_{\text{LM}} + \lambda_0 \mathcal{L}_{\text{Medusa-1}}$ (Cai et al., 2024, §2.2.2), recovering higher acceptance at the cost of a backbone fine-tune. Reported speedups: Medusa-1 $\sim 2.2\times$, Medusa-2 2.3–2.8× on Vicuna 7B/13B/33B (Cai et al., 2024, abstract).

Cai et al. (2024, §2.3.1) also proposes *typical acceptance* as an alternative to eq. (45), accepting any draft whose target probability exceeds an entropy-adaptive threshold $\min(\epsilon, \delta \exp(-H(p)))$. This is faster but *not* distribution-preserving for temperature $\neq 0$ — a tradeoff sometimes elided in deployment notes.

Contradiction

Medusa with rejection sampling (eq. (45) applied to heads-as-drafter) is mathematically distribution-preserving; Medusa with typical acceptance is not. Vendor blogs and benchmark tables sometimes report "Medusa speedup" without specifying which verification rule was used, conflating the two regimes. Any deployment that requires reproducible sampling (eval harnesses, structured-output RAG, RLHF rollouts) must verify the verification rule, not just the draft mechanism.

10.3 Self-speculation at the feature level: EAGLE

EAGLE (Li et al., 2024) reframes the drafter as predicting *features* — the second-to-top-layer hidden state f, the input to the LM head — rather than tokens directly. The motivating observation (Li et al., 2024, abstract): token sequences have high local entropy because many tokens are sampled from a flat tail, but feature sequences are smoother and more predictable, since the LM head absorbs the entropy. The drafter is a single decoder layer (an FC projection plus one transformer block) taking a fused input of past features $F_{1:i}$ and the *shifted* token sequence $T_{2:i+1}$, and predicting $\hat{f}_{i+1}$ (Li et al., 2024, §3.1)[52]. The shift by one is the load-bearing trick: at the time of predicting f_{i+1},

[50] KB excerpt: `kb/excerpts/cai2024-medusa.md#sec-2-1-1`.
[51] KB excerpt: `kb/excerpts/cai2024-medusa.md#sec-2-1-2`.
[52] KB excerpt: `kb/excerpts/li2024-eagle.md#sec-3-1`.

the token t_{i+1} that branched the actual generation has already been sampled and is conditioned on, which resolves the multi-mode uncertainty that pure feature autoregression cannot commit to.

The drafter is trained with a combined objective (Li et al., 2024, §3.2):

$$L = \text{SmoothL1}(f_{i+1}, \hat{f}_{i+1}) + w_{\text{cls}} \cdot \text{CE}\Big(\text{softmax}(\text{LM\_Head}(f_{i+1})), \text{softmax}(\text{LM\_Head}(\hat{f}_{i+1}))\Big), \quad (49)$$

with $w_{\text{cls}} = 0.1$. The classification term routes the target's frozen LM head over both true and predicted features, so the drafter optimizes *token agreement* via the LM head's projection, not raw feature L2 distance. EAGLE inherits Leviathan-style rejection sampling at the verification step (Li et al., 2024, §3.3), so distribution preservation is by construction. Reported speedups on LLaMA2-Chat 70B at MT-bench: 3.01× at temperature 0, 2.67× at temperature 1 (Li et al., 2024, Fig. 1). EAGLE-3 (et al., 2025) (NeurIPS 2025) abandons feature prediction in favor of direct token prediction with multi-layer feature fusion, claiming up to 6.5×; the lineage is open at frontier scale.

10.4 Multi-Token Prediction (MTP) as architectural choice

Gloeckle et al. (2024) take the dual position: rather than bolt a drafter on at inference, modify the *training* objective so the trunk produces hidden states predictive of multiple future tokens simultaneously. The standard next-token loss is generalized to (Gloeckle et al., 2024, §2, Eq. 1)[53]

$$\mathcal{L}_n = -\sum_t \log P_\theta(x_{t+n:t+1} \mid x_{1:t}) = -\sum_t \sum_{i=1}^{n} \log P_\theta(x_{t+i} \mid x_{1:t}), \quad (50)$$

implemented with n *independent* output heads $f_1, \ldots, f_n$ on a shared trunk producing $z_t = f_s(x_{1:t}) \in \mathbb{R}^D$ and $P_\theta(x_{t+i} \mid x_{1:t}) = \text{softmax}(f_i(z_t))$ (Gloeckle et al., 2024, §3). The training-time overhead is under 5% for $n \in \{2, 4\}$ (Gloeckle et al., 2024, §3). Crucially, the next-token head f_1 is unchanged at inference — so a model can be MTP-trained and served as if it were a vanilla next-token model, with the auxiliary heads either discarded or reused as drafters. Headline quality results: 13B-parameter MTP models solve 12% more HumanEval and 17% more MBPP problems than matched next-token baselines (Gloeckle et al., 2024, abstract); the method is *increasingly* useful at larger scale.

DeepSeek-V3 (DeepSeek-AI, 2024b, §2.3.1) productionizes a chained variant: instead of n independent heads each conditioned on z_t alone, a sequence of k small Transformer blocks M_i refine a chain of hidden states using the previous predicted token's embedding,

$$\mathbf{h}_t^i = M_i\Big(\text{Norm}(\mathbf{h}_t^{i-1}); \text{Norm}(E_{\text{in}}[x_{t+i}])\Big), \quad \text{logits}_{t+i+1}^{(i)} = \mathbf{h}_t^i E_{\text{out}}, \quad (51)$$

with the LM head E_{out} *shared* across all heads (consistent with the embedding-tying choice of section 3). The total objective adds an auxiliary term per chained head with weight λ that decays during training: $\mathcal{L} = \mathcal{L}_{\text{NTP}} + \frac{\lambda}{D} \sum_{i=1}^{k} \mathcal{L}_{\text{MTP},i}$ (DeepSeek-AI, 2024b, §2.3.1). DeepSeek-V3 ships $k = 1$ (one MTP module beyond the main head) and reports $\sim 85\%$ acceptance for self-speculation and $\sim 1.8\times$ wall-clock speedup (DeepSeek-AI, 2024b, §2.3.1). The parallel-vs-chained distinction matters because chained heads condition each future-token prediction on the previous prediction — the natural autoregressive structure eq. (45) exploits at inference.

Intuition

MTP is one model, many heads, one objective. The shared trunk produces a single hidden state $z_t = f_s(x_{1:t})$; the heads $f_1, \ldots, f_n$ each project it to a different future-position vocab

[53] KB excerpt: `kb/excerpts/gloeckle2024-mtp.md#sec-2`.

distribution. The objective in eq. (50) is the sum of cross- entropies of those projections against the corresponding ground-truth tokens. Returning to the canonical multi-objective form: the gradient of $\mathcal{L}_n$ flows back through every f_i into the trunk, so the trunk's hidden states are shaped to encode information predictive of *all* of $x_{t+1}, \ldots, x_{t+n}$, not just x_{t+1}. DeepSeek's chained variant in eq. (51) adds a small recurrent refinement: each M_i takes the previous hidden plus the embedding of the previous predicted token, so the i-th head's distribution is conditional on the realized chain rather than only on z_t. Either way, the training signal is multi-objective; either way, inference can use just f_1 at zero overhead.

10.5 The training-vs-inference axis

The four mechanisms above sit on a spectrum indexed by *when* the multi-token capability is paid for:

- **Speculative decoding** (Leviathan et al., 2023): pure inference. The target model is unchanged; the drafter M_q is trained separately or pulled from the same family. Distribution-preserving by construction. Cost: separate drafter to deploy.
- **Medusa-1** (Cai et al., 2024): post-hoc fine-tune. The backbone is frozen; only the heads of eq. (48) are trained. Distribution-preserving under eq. (45). Cost: one fine-tuning stage; no drafter; no backbone change.
- **Medusa-2 / EAGLE**: post-hoc with backbone unfrozen (Medusa-2) or with a feature-level drafter trained on the target's features (EAGLE). Distribution-preserving under inherited rejection sampling. Cost: backbone fine-tune (Medusa-2) or drafter parameters $\sim$ 1–3% of target (EAGLE).
- **MTP** (Gloeckle et al., 2024; DeepSeek-AI, 2024b): pretraining- time architectural choice. The training objective is eq. (50) (parallel) or eq. (51) (chained). The trunk's hidden states are shaped by the multi-token signal during pretraining; the heads are integrated into the architecture, not bolted on. Distribution-preserving when reused as drafter under eq. (45). Cost: $\sim$ 5% training-time overhead and $\sim$ 1 Transformer block of parameters (DeepSeek-V3).

The key invariant: every mechanism above commutes with eq. (45), so they can be *composed*. An MTP-trained model can use its built-in heads as the drafter, dropping the cost coefficient c in eq. (47) to nearly zero (no separate forward pass) and pushing α high (the drafter's distribution *is* the target's marginal distribution at offset i, so $D_{LK}(p, q)$ is small by construction). The compositionality is why MTP is increasingly the convergent choice at frontier 2026: it collapses two engineering investments (drafter training; drafter maintenance) into one (a slightly modified pretraining objective) while leaving sections 5 to 7 and 9 untouched. Whether the quality lift Gloeckle reports survives at non-DeepSeek frontier scale, and how MTP interacts with the long chain-of-thought generation patterns of reasoning models like DeepSeek-R1, are open — and the subject of the contradiction Gloeckle et al. (2024) surface vs. DeepSeek-AI (2024b) surface about *whether MTP is mainly a speedup or also a quality lift*, addressed in section 14.

10.6 Forward link

The mechanisms in this section assume a token-stream output: the target distribution is over a single discrete vocabulary $\{1, \ldots, V\}$, and "multi-token output" means more positions in that same stream.

section 11 relaxes this assumption in a different direction: the input and output streams may include image patches, audio frames, or video tokens that are not natively part of V. The three-piece pattern there — vision encoder, projector, LLM body — turns out to be largely orthogonal to the choices in this section: a multimodal LLM with MTP heads is a thing one can build (and DeepSeek-V3 implicitly is, in its multimodal-extended variants), and a speculatively-decoded multimodal model is also a thing. The two axes multiply rather than interact.

11 Multimodal extensions

The architectures of §§5–10 are decoder-only language models over a single token stream. Frontier 2026 systems also take images, video, and audio. Three integration patterns dominate the record: gated cross-attention insertion into a frozen LLM (Alayrac et al., 2022), linear projection of vision features into the LLM input-embedding space (Liu et al., 2023), and native multimodal pretraining where vision and text patches share the residual stream from the first token of pretraining (AI, 2026; various, 2025a; DeepMind, 2025). The thesis of this section is that the projector and the choice of pattern are genuinely architectural, while the vision encoder and the freeze-versus-tune schedule are training-pipeline choices that we defer to Paper 2.

11.1 The three-piece pattern

Across all three integration patterns the same three pieces appear: a *vision encoder* E_V that turns an image into patch features, a *projector* π that maps those features into the LLM's residual width, and the *LLM body* that consumes them. The encoder is typically a CLIP-style or SigLIP-style ViT producing a sequence of M patch tokens of dimension d_V; for a 336×336 image with 14×14 patches, M=576 and d_V=1024 in the canonical LLaVA setup[54] (Liu et al., 2023). We write the encoder output and the projection generically as

$$v = E_V(I) \ \in \ \mathbb{R}^{n_v \times d_V}, \tag{52}$$

$$\mathbf{X}_{\text{vis}} = \pi(v) \ \in \ \mathbb{R}^{n_v \times \mathsf{D}}, \tag{53}$$

with n_v the number of visual tokens and D the LLM model dimension. The interesting structure is in π and in how the body consumes $\mathbf{X}_{\text{vis}}$.

Intuition

$\mathbf{X}_{\text{vis}}$ is a *soft prompt*: a contiguous block of n_v vectors in $\mathbb{R}^{\mathsf{D}}$ that the LLM treats as if they were output of its input embedding. The vision encoder is a tokenizer for a non-text modality, π is the embedding lookup, and the LLM body is unchanged from §2. Returning to math: the residual-stream initial state becomes $\mathbf{X}^0 = [E_{\text{in}}(\mathbf{t}_{1:k}); \pi(E_V(I)); E_{\text{in}}(\mathbf{t}_{k+1:N})]$ with the LLM's ordinary update rule applied above it.

This soft-prompt framing is exactly the LLaVA pattern. The other two patterns deviate by either sending $\mathbf{X}_{\text{vis}}$ through new *cross-attention* layers rather than through the residual stream (Flamingo) or by training E_V and the LLM body jointly from scratch on multimodal data (native multimodal). The pieces are the same; the plumbing differs.

[54] KB excerpt: `kb/excerpts/llava2023.md#sec-4-1`

11.2 Cross-attention bridge: Flamingo

Alayrac et al. (2022) introduced the first widely-adopted multimodal extension by inserting new trainable blocks between the existing layers of a frozen LLM. Two pieces are added: a *Perceiver Resampler* that compresses the variable-length encoder output v into a fixed-length set of k latent tokens $\mathbf{V} \in \mathbb{R}^{k \times \mathsf{D}}$ (typically k=64), and *gated cross-attention dense* blocks inserted between every few frozen LLM blocks[55]. The inserted block has the form

$$\mathbf{y} = \mathbf{x} + \tanh(\alpha_{\text{xattn}}) \cdot \text{XAttn}(\mathbf{x}, \mathbf{V}), \tag{54}$$

$$\mathbf{z} = \mathbf{y} + \tanh(\alpha_{\text{ffw}}) \cdot \text{FFW}(\mathbf{y}), \tag{55}$$

where $\mathbf{x}$ is the language hidden state at the insertion point, and $\alpha_{\text{xattn}}, \alpha_{\text{ffw}}$ are learnable scalars *initialized to zero*[56] (Alayrac et al., 2022). With $\tanh(0)$=0 the inserted block is the identity at initialization: the conditioned model exactly reproduces the frozen base LM, and as training proceeds the gates open and visual information enters the language stream.

The Perceiver Resampler is itself a small Transformer with k learned query latents that cross-attend to the encoder output v, producing a fixed-size compressed visual representation regardless of input image or video length. Crucially, both the vision encoder and the base LM remain frozen for the duration of training; only the Resampler and the inserted gated cross-attention blocks update (Alayrac et al., 2022, §2.1).

11.3 Linear projection: LLaVA

Liu et al. (2023) took the opposite approach: change nothing about the LLM's architecture, and replace Flamingo's Resampler-plus-gated-XAttn machinery with a single trainable linear matrix. With $Z_v = g(X_v)$ the CLIP ViT-L/14 grid features for image X_v, LLaVA defines

$$H_v = W \cdot Z_v, \qquad W \in \mathbb{R}^{\mathsf{D} \times d_V}, \tag{56}$$

and concatenates the resulting visual tokens H_v with the textual input as ordinary embedding-space vectors[57]. This is the entire architectural intervention. The LLM body sees a sequence that begins with n_v visual "tokens" and continues with text; its forward pass is identical to a unimodal LLM's.

Training proceeds in two stages[58]. *Stage 1: feature alignment*, where both the CLIP encoder g and the LLM body f_ϕ are frozen and only W trains on 595K filtered CC3M image- text pairs — the projection learns to produce visual tokens that look, to the frozen LLM, like word embeddings. *Stage 2: end-to-end fine-tuning*, where the encoder stays frozen but $\theta = \{W, \phi\}$ trains together on 158K GPT-4-generated visual instruction examples. Later LLaVA-1.5 replaced the linear W with a 2-layer MLP for small additional gains; the architectural premise is unchanged.

Analogy

Flamingo treats the LLM as sealed and bolts new vision-conditioning hardware onto its sides. LLaVA treats the LLM's input-embedding space as a public API: anything that can be projected into $\mathbb{R}^{\mathsf{D}}$ becomes a token. Returning to math: Flamingo introduces a new operator $\text{XAttn}(\cdot, \mathbf{V})$ that mixes vision into the residual stream via Eqs. (54)–(55); LLaVA only changes the input sequence to the unmodified LLM via Eq. (56).

[55] KB excerpt: `kb/excerpts/alayrac2022-flamingo.md#sec-2-1`
[56] KB excerpt: `kb/excerpts/alayrac2022-flamingo.md#sec-2-1-gated`
[57] KB excerpt: `kb/excerpts/llava2023.md#sec-4-1`, Eq. 1
[58] KB excerpt: `kb/excerpts/llava2023.md#sec-4-2`

LLaVA's parametric simplicity is the reason it dominated the open multimodal stack from 2024 onward: a single linear matrix is fast to train, the LLM body needs no surgery, and ablations on data and encoder choice (CLIP vs. SigLIP, image resolution, dynamic patching) become cheap. Llama 3.2-Vision, InternVL, and the Qwen2-VL line all build on this LLaVA recipe before diverging on resolution policy and positional encoding.

11.4 Native multimodal: Gemini, GPT-4o, Claude, Llama 4, InternVL3

The 2025–2026 frontier moves past adapter patterns. The pretraining corpus itself is multimodal — interleaved web pages with text, images, video frames, and audio — and the model is trained from scratch on it (AI, 2026; various, 2025a; DeepMind, 2025). Architecturally this is a standard decoder-only Transformer where the *input embedding is multi-source*: text tokens flow through $E_{\text{in}}^{\text{text}}$, image patches through a small ViT-like $E_{\text{in}}^{\text{vis}}$, and audio frames through $E_{\text{in}}^{\text{audio}}$, all producing D-dimensional vectors that share the residual stream. The body is unchanged; the "vocabulary" has been generalized from text tokens alone to a union of token sources.

deepen-cite: gpt-4o-card §architecture

deepen-cite: claude-3-5-sonnet §architecture

InternVL3 reports that joint pretraining outperforms an adapter baseline at matched compute on internal benchmarks (various, 2025a); Llama 4 makes a similar claim (AI, 2026). The case for native multimodal at the frontier is empirical and lab-internal; the architectural commitment is that no explicit cross-modality bridge (XAttn, Q-Former, or projector) is needed once the residual stream has seen both modalities since pretraining step zero.

Forum signal

Vendor reports (Gemini 2.5, Llama 4, InternVL3) consistently claim native multimodal pretraining produces better quality at smaller active parameter count than adapter-based alternatives at the same compute. The published evidence is internal benchmark comparisons against each lab's own adapter baselines; no controlled cross-lab study with data, compute, and post-training fixed has appeared. Open replication at frontier scale is absent — treat as forum signal pending reproduction (various, 2025a; AI, 2026).

11.5 Where multimodality is an architecture question

The framing of this paper is that the modern Transformer is a small set of architectural choices; multimodality adds two genuinely new ones and cleanly externalizes a third.

1. **The integration pattern.** Cross-attention bridge (Flamingo, Eqs. (54)–(55)), prefix-token projection (LLaVA, Eq. (56)), or unified-token native pretraining are three architecturally distinct choices. They differ in which weights see vision-conditioning gradients and at what depth in the network vision can interact with text. This is the load-bearing axis.

2. **The projector design.** Whether π is a single linear matrix, a 2-layer MLP, a Q-Former, or a Perceiver Resampler determines the visual token count n_v, the parameter cost, and how aggressively visual information is compressed before it reaches the residual stream. Within the prefix-token pattern this is the dominant variable.

3. **The vision encoder family.** CLIP ViT-L/14, SigLIP, SigLIP2, Qwen-VL ViT, InternVL ViT — the choice of E_V is architectural *at the boundary* between the multimodal interface

and the unimodal vision literature. Inside the LLM body its contribution is summarized by d_V and n_v at the encoder output; the rest is a separate field's design space.

What is *not* a §11 architecture question:

- *Frozen-versus-tuned LLM body.* LLaVA's two-stage recipe freezes then unfreezes the LLM body; Flamingo keeps it frozen throughout; native multimodal trains everything jointly from scratch. This is a training-pipeline choice with the same architecture under all three regimes; we defer it to Paper 2 (cf. Paper 2's SFT and multimodal-data-mixture sections).
- *Visual instruction-tuning data.* LLaVA's 158K GPT-4-generated visual conversations (Liu et al., 2023, §3) are a synthetic-data engineering choice, not an architectural one — again Paper 2.
- *Multi-axis positional encoding.* M-RoPE and Interleaved-MRoPE extend RoPE to 2D and 3D coordinates for image and video patches; the mechanism is positional and inherits the analysis of §4, so we treat it there rather than here.

The cleanest architectural summary is: vision-language extension adds one new axis (integration pattern), refines one existing component (the input-embedding side, now multi-source), and otherwise leaves the choices of §§5–8 untouched.

A multimodal LLM is mostly a unimodal LLM with a generalized input-embedding stage, plus a single new architectural axis for how visual tokens enter the residual stream. The remaining components — KV-sharing, attention implementation, FFN family, normalization, position encoding — are the same choices Paper 1 has been documenting throughout. §12 now turns to the inference-time companions of these choices: KV-cache lifecycle, quantization regime, and structured-output decoding, which constrain which architectural choices §§5–10 can afford in deployment.

12 Inference adjuncts

The architectural choices catalogued in section 3 through section 11 fix the model's mathematical surface; they do not by themselves determine what the deployed system can serve at matched cost. Three categories of *inference-time* concern — the KV-cache lifecycle and its memory-allocation discipline, weight / activation / KV quantization regimes, and structured-output enforcement through constrained decoding — are the load-bearing reason a 2026 frontier model is feasible to ship. Each constrains which choices in sections 5 to 7 and 9 can be taken: paged allocation requires a block-friendly attention kernel, ternary pretraining requires LLaMA-alike normalization and gating, FSM-masked decoding requires an LM-head whose logit space is tokenizer-aligned. The thesis of this section is that these are *architectural concerns, not bolt-ons.*

12.1 KV-cache lifecycle and PagedAttention

The autoregressive decode loop reads, at every step, the cached K/V projection of every prior token at every layer. For an OPT-13B request of 2048 tokens the KV cache is ≈ 1.6 GB, with the per-token cost $2 \cdot L \cdot n_h \cdot d_h \cdot b$ bytes where b is the element width (Kwon et al., 2023, §3).[59] Pre-2023 serving stacks (FasterTransformer, Orca) pre-allocated each request a contiguous slab sized to its *maximum* sequence length, which produced three sources of waste (Kwon et al., 2023, §3):

[59] KB excerpt: `kb/excerpts/kwon2023.md#sec-3`.

reservation (slots held for future tokens not yet generated), *internal fragmentation* (slack between actual and maximum length, realized only at request completion), and *external fragmentation* (allocator holes between slabs). Empirically the contiguous regime put only 20.4–38.2% of allocated KV memory onto actual token state (Kwon et al., 2023, §3, Fig. 2).

Kwon et al. (2023) resolves the waste taxonomy by partitioning each sequence's KV cache into fixed-size *KV blocks* of B tokens (typical $B = 16$) and decomposing the attention sum block-wise (Kwon et al., 2023, §4.1, Eq. 4)[60]:

$$A_{ij} = \frac{\exp(q_i^\top K_j/\sqrt{d})}{\sum_{t=1}^{\lceil i/B \rceil} \exp(q_i^\top K_t \mathbf{1}/\sqrt{d})}, \qquad o_i = \sum_{j=1}^{\lceil i/B \rceil} V_j A_{ij}^\top, \tag{57}$$

with $K_j = (k_{(j-1)B+1}, \ldots, k_{jB})$, $V_j = (v_{(j-1)B+1}, \ldots, v_{jB})$, and the kernel reading each block from a possibly non-contiguous physical address. The arithmetic is the same as the canonical scaled-dot-product attention of section 6; the contribution is layout, not arithmetic.

A *block table* per request maps logical KV blocks to physical KV blocks, exactly the role of an OS page table (Kwon et al., 2023, §4.2).[61] Three OS idioms transfer unchanged: allocate-on-demand (no maximum-length pre-reservation), copy-on-write at the block boundary for parallel sampling and beam search (Kwon et al., 2023, §4.4), and swap-or-recompute for preemption (Kwon et al., 2023, §4.5). The reported headline is 2–4× throughput at matched latency vs. FasterTransformer/Orca on OPT-13B/66B/175B (Kwon et al., 2023, abstract, §6.1); the cache-utilization number rises from $\sim 30\%$ to 96.3% (Kwon et al., 2023, §3, Fig. 2).

Analogy

PagedAttention is virtual memory for the KV cache. The block table is a page table; allocate-on-demand is demand paging; copy-on-write across sampled continuations is the same pattern as `fork()`'s COW pages; swap evicts blocks to CPU RAM as a swap device. Returning to math: the block decomposition in eq. (57) is the canonical attention sum re-indexed over physical blocks rather than contiguous positions; the kernel's flexibility to fetch each K_j, V_j from a non-contiguous address is what frees the allocator from the maximum-length reservation that produced the $\sim 70\%$ waste.

The architectural pressure runs the other way too: PagedAttention's block-decomposed kernel is what enables the cross-request prefix sharing of RadixAttention (Zheng et al., 2024, §3)[62], the training-time KV compression of MQA / GQA / MLA in section 5 (which act on the per-token block size), and the KV-quantization regimes below (which change the element width b but not the layout).

12.2 Quantization regimes

LLM quantization is a family of decisions indexed by *what* is quantized (weights, activations, KV cache), *when* (post-training vs. training-aware vs. native low-bit pretraining), and *to what precision* (8-bit, 4-bit, 2-bit, ternary). The uniform integer map underlying every method below is

$$\bar{X}^{\mathrm{INT}N} = \left\lceil \frac{X^{\mathrm{FP16}}}{\Delta} \right\rfloor, \qquad \Delta = \frac{\max(|X|)}{2^{N-1} - 1}, \tag{58}$$

[60] KB excerpt: `kb/excerpts/kwon2023.md#sec-4-1`.
[61] KB excerpt: `kb/excerpts/kwon2023.md#sec-4-2`.
[62] KB excerpt: `kb/excerpts/zheng2024-sglang.md#sec-3`.

with N-bit signed levels and step size Δ (Xiao et al., 2023, §2, Eq. 1).[63] Hardware constrains the granularity: tensor-core GEMMs apply scale factors only along outer dimensions (token T or output channel C_o), never along the inner contraction dimension C_i (Xiao et al., 2023, §3). Per-channel-of-activations is infeasible without re-engineering the kernel. This single constraint generates the lineage of methods.

INT8 (W8A8) — SmoothQuant. Past ~ 6.7B parameters, LLM activations develop systematic outliers $\sim 100\times$ larger than typical values, persistent in fixed channels across tokens (Xiao et al., 2023, §3). Naive per-tensor INT8 catastrophically degrades quality (OPT-175B accuracy 71.6% FP16 $\to$ 32.3% per-tensor INT8 (Xiao et al., 2023, §5.2, Table 1)).[64] Xiao et al. (2023) resolves this with a *migration* trick that is mathematically a free rescaling (Xiao et al., 2023, §4, Eq. 3):

$$Y = (X\,\mathrm{diag}(s)^{-1}) \cdot (\mathrm{diag}(s)W) = \hat{X}\hat{W}, \tag{59}$$

with the per-channel smoothing factor s_j chosen to balance the quantization difficulty between activations and weights via a migration-strength hyperparameter α (Xiao et al., 2023, §4, Eq. 4):

$$s_j = \frac{\max(|X_j|)^{\alpha}}{\max(|W_j|)^{1-\alpha}}, \qquad \alpha = 0.5 \text{ default.} \tag{60}$$

At $\alpha = 0$ the difficulty stays on activations (broken); at $\alpha = 1$ it migrates entirely to weights (also broken because weights now have the outlier shape they previously lacked); $\alpha \approx 0.5$ is the sweet spot. Crucially, s is fused offline into the preceding linear or normalization layer so the runtime cost is zero, and per-tensor INT8 then recovers the FP16 baseline (71.4% on OPT-175B (Xiao et al., 2023, §5.2, Table 1)).

INT4 (W4A16) — GPTQ. For weight-only INT4, Frantar et al. (2022) adapt the layer-wise reconstruction objective (Frantar et al., 2022, §3, Eq. 1)[65]

$$\underset{\widehat{W}}{\mathrm{argmin}}\ \|WX - \widehat{W}X\|_2^2 \tag{61}$$

to LLM scale by accelerating the Optimal Brain Quantization baseline along three axes: arbitrary fixed column order rather than per-row greedy ordering, lazy batched updates over $B = 128$ columns, and a Cholesky reformulation of the inverse-Hessian update with damping $\lambda \approx 1\%$ of the diagonal (Frantar et al., 2022, §4).[66] The joint effect: OPT-175B and BLOOM-176B at 3–4 bits/weight track FP16 perplexity, where round-to-nearest at the same precision diverges (110 and 571 respectively (Frantar et al., 2022, §1, Fig. 1)).[67] GPTQ is the canonical accurate one-shot weight-only PTQ method at the 100B+ scale and is the reference against which AWQ and the GGUF Qk_M family are still benchmarked.

FP8 (training). Training in FP8 with tile-based scales (128×128 weight blocks, 1×128 activation vectors) was first validated at trillion-token scale by DeepSeek-V3 (DeepSeek-AI, 2024b, §2.5), with H100/H200/Blackwell hardware FP8 paths. We treat FP8 in **??**; the relevance to this section is that an FP8-trained checkpoint can be served *without* PTQ, collapsing two stages into one.

[63] KB excerpt: `kb/excerpts/xiao2023-smoothquant.md#sec-2`.
[64] KB excerpt: `kb/excerpts/xiao2023-smoothquant.md#sec-5-2`.
[65] KB excerpt: `kb/excerpts/frantar2022.md#sec-3-layerwise`.
[66] KB excerpt: `kb/excerpts/frantar2022.md#sec-4-algorithm`.
[67] KB excerpt: `kb/excerpts/frantar2022.md#sec-1-figure-1`.

NVFP4 (frontier 2026). Microscaling formats from the OCP standard pair small blocks with a shared exponent. NVFP4 is NVIDIA's Blackwell-generation 4-bit floating-point with per-microblock scales and is the first 4-bit format shipped with native tensor-core acceleration; vendor materials report it as production-ready for inference at frontier scale, though third-party reproductions outside NVIDIA-published benchmarks are sparse as of 2026-Q2 (DeepSeek-AI, 2024b, FORUM-SIGNAL).[68]

1.58-bit — BitNet. et al. (Microsoft) take the dual position: rather than quantize a trained model, train from scratch with ternary weights. Every weight is constrained to $\{-1, 0, +1\}$ ($\log_2 3 \approx 1.585$ bits) via an absmean RoundClip (et al. , Microsoft, §2, Eq. 1–3)[69]:

$$\widetilde{W} = \text{RoundClip}\left(\frac{W}{\gamma + \epsilon}, -1, 1\right), \qquad \gamma = \frac{1}{nm}\sum_{ij} |W_{ij}|, \tag{62}$$

with $\text{RoundClip}(x, a, b) = \max(a, \min(b, \text{round}(x)))$; activations remain INT8 (the regime is W1.58A8, not pure 1-bit). The architecture is otherwise LLaMA-alike (RMSNorm, SwiGLU, RoPE, no biases) so it integrates into the open-source serving stack unchanged (et al. , Microsoft, §2). At 3B parameters, BitNet b1.58 matches FP16 LLaMA perplexity (9.91 vs. 10.04) at 2.71× lower latency and 3.55× lower memory (et al. , Microsoft, §3, Table 1).[70]

Intuition

Each regime operates on a different part of the inference graph. SmoothQuant rescales activations and weights pre-runtime so a hardware-friendly per-tensor INT8 GEMM suffices — the MM kernel is unchanged, the inputs are reshaped by eq. (59). GPTQ adjusts only the weight matrix via second-order layer-wise reconstruction in eq. (61) — activations stay FP16, the LM-head and attention paths are untouched. BitNet replaces multiplications with sign-flip-plus-select inside the BitLinear layer of eq. (62) — the architecture changes, the energy cost on a 7nm chip drops 71× on the matmul-arithmetic component (et al. , Microsoft, §3). Each regime answers a different question about which part of the deployment graph is precious.

Contradiction

NVFP4's production status at frontier scale is contested. NVIDIA's Blackwell-launch materials and vendor blogs report NVFP4 as production-ready for inference of large models with quality matching FP8 / FP16 baselines; third-party reproduction as of 2026-Q2 is sparse, with most published benchmarks coming from NVIDIA-collaborator labs. By contrast, FP8 production training has independent confirmation through DeepSeek-V3's open release (DeepSeek-AI, 2024b, §2.5) and multiple downstream replications. The KB reflects this asymmetry: FP8 is a tier-A primary claim, NVFP4 is a tier-B forum-signal claim pending peer-reviewed third-party benchmarks.

deepen-cite: NVFP4 frontier-scale quality

[68] No peer-reviewed primary source for NVFP4 LLM quality at frontier scale exists in the KB as of `2026-05`; the citation here points to the closest analogue (FP8 production training) and flags the gap.

[69] KB excerpt: `kb/excerpts/ma2024-bitnet.md#sec-2`.

[70] KB excerpt: `kb/excerpts/ma2024-bitnet.md#sec-3-table-1`.

12.3 Structured output as constrained decoding

The third inference-time concern is enforcing that generated text conforms to a target language $\mathcal{G}$ (regex, JSON schema, GBNF, arbitrary CFG). The mechanism is *logit masking*: at each step, zero out the probability of any token whose acceptance would leave the partial output outside $\mathcal{G}$, then sample from the renormalized distribution. Formally, if $\ell_t \in \mathbb{R}^V$ are the model logits at step t and s_t is the grammar-state (FSM state, parser stack summary), the constrained-decoding distribution is

$$P(y_t = v \mid y_{<t}) = \frac{\mathbb{1}[v \in \mathcal{V}_{\text{allowed}}(s_t)] \cdot \exp(\ell_{t,v})}{\sum_{v' \in \mathcal{V}_{\text{allowed}}(s_t)} \exp(\ell_{t,v'})}, \tag{63}$$

implemented by setting $\ell_{t,v} \leftarrow -\infty$ for $v \notin \mathcal{V}_{\text{allowed}}(s_t)$ before sampling. The expensive operation is computing $\mathcal{V}_{\text{allowed}}(s_t) \subseteq \{1, \dots, V\}$ at every step: a naive grammar engine queries acceptance for every token in a 100K-vocab model, costing $\sim$ 100ms/step, which dominates the forward pass.

Willard and Louf (2023) reformulate the problem as *transitions in an FSM whose alphabet is the model's vocabulary*. The standard FSM operates over bytes; the contribution is a precomputed *index* mapping each FSM state s to the set of vocabulary tokens whose byte sequence keeps the FSM in a valid path (Willard and Louf, 2023, §3)[71]:

$$\mathcal{V}_{\text{allowed}}(s) = \{v \in \mathcal{V} : \text{FSM accepts the byte sequence of } v \text{ starting from } s\}. \tag{64}$$

The index is built once per grammar; per-step lookup is $O(1)$ amortized. Extension to context-free grammars goes via LALR(1) parsers where the parser state and a stack-summary jointly key the index (Willard and Louf, 2023, §4); for JSON, Python, and SQL the stack-summary stays small enough to keep the index practical.

et al. (MLC) extend Willard–Louf with a partition that exploits a property of typical schemas: most vocabulary tokens' allowed/disallowed status depends only on the current pushdown-automaton state, not on stack contents (et al. , MLC, §3).[72] XGrammar precomputes a per-state *bitmask* over $\mathcal{V}$ for the context-independent fraction (typically $\sim$ 99% on JSON schemas); the small context-dependent set ($\sim$ 1%, e.g. closing brackets that must match the correct opening) is checked at runtime against a persistent stack (et al. , MLC, §4–5). The two-class engine runs grammar checks on the CPU concurrently with the GPU forward pass (et al. , MLC, §6), achieving up to 100$\times$ speedup over prior solutions and $\leq 40\,\mu$s per token of grammar overhead on 100K-vocab models (et al. , MLC, §7). By 2026 XGrammar is the default constrained-decoding backend in vLLM, SGLang, and TensorRT-LLM (et al. , MLC, FORUM-SIGNAL).

Analogy

Constrained decoding is the *product* of the FSM (which says which tokens are allowed) with the language model (which says which tokens are likely): the masked distribution in eq. (63) is the model's softmax times the indicator function of $\mathcal{V}_{\text{allowed}}(s_t)$, renormalized. This is why it is architectural rather than a sampling choice: the LM head's columns must align one-to-one with the tokenizer that the FSM compiles against eq. (64), and the head's output dimension V is the same V over which the bitmask is stored. A model whose tokenizer or LM-head is replaced post-hoc invalidates every precomputed index. The choice in section 3 forward-determines what the constrained-decoding stack can do.

[71] KB excerpt: `kb/excerpts/willard2023-outlines.md#sec-3`.
[72] KB excerpt: `kb/excerpts/xgrammar2024.md#sec-3`.

12.4 Forward link

The three concerns in this section — KV-block layout, quantization regime, constrained-decoding stack — jointly with the architectural choices of section 3 through section 11 fix the deployable surface of a 2026 frontier LLM. We turn next (section 13) to a model-by-model table of the architectural choices the leading 2026 systems share, and where they diverge. PagedAttention has become universal; GQA-or-MLA is near-universal; ternary pretraining at 70B+ remains open; NVFP4 production at frontier scale remains contested. The localized open frontiers we have surfaced through sections 5, 6 and 9 and the ones surfaced here are the ones section 14 will address as a single concentrated discussion.

13 Frontier-2026 snapshot: what choices the leading models share

The previous ten sections walked the architectural axes one at a time. This section composes them. We tabulate the per-axis choices made by the 2026 frontier model lineup — DeepSeek V3 and R1, Llama 4, Mistral Large 2, Gemma 3, Qwen 2.5 / Qwen 3, plus the partially disclosed Claude 3.7-class, GPT-class, and Gemini 2.5 systems — on the same eight axes used throughout this paper, and call out where the published architectures agree and where they diverge. **The thesis of this section: the convergence is real, narrow, and concentrated in five of the eight axes; the remaining three (KV-sharing strategy, dense-vs-MoE on the FFN, and the multimodal integration pattern) are the live splits, and only one of them (KV-sharing) cleanly partitions the disclosed lineup.**

The table that follows replaces the model-zoo prose of vendor blog posts with a uniform ten-column row per model. Where a lab has published a primary source we cite it; where a detail is vendor-blog or release-card disclosure only we mark

deepen-cite: citation-pending

; where the architecture is undisclosed we record "not disclosed" rather than guess from inference-stack reverse engineering, which is unreliable across point releases.

13.1 The snapshot table

Table 4 is the headline figure of this paper. §13.2–§13.3 walk the disclosed and undisclosed clusters; then §13.4–§13.5 read the table for points of agreement and points of divergence.

13.2 Disclosed cluster: the open-weights frontier

The seven rows in the upper block of table 4 are either fully open-weights with an architectural tech report, or disclose architectural detail in a public release card. We walk them briefly to anchor each cell.

DeepSeek V3 and R1. DeepSeek V3 is a 671B-total / 37B-active MoE with $E = 256$, $E_s = 1$, $k = 8$, auxiliary-loss-free balancing (DeepSeek-AI, 2024b, §2.2).[73] The attention sublayer uses MLA with joint-compression rank $d_c = 512$ and the decoupled-RoPE construction of DeepSeek-AI (2024a, §2.1.2 Eq. 9). Multi-Token Prediction with one auxiliary head is part of the pretraining objective (DeepSeek-AI, 2024b, §2.3.1). RMSNorm in Pre-LN placement is inherited from the LLaMA template (Touvron et al., 2023). DeepSeek-R1 is a continued-RL fine-tune of V3-base with

[73] KB excerpt: `kb/excerpts/deepseek-v3.md#sec-2`.

no architectural changes other than a training-stage thinking-mode toggle (DeepSeek-AI, 2025, §2); its row repeats V3's choices.

Llama 4 (Scout, Maverick). LLaMA 4 ships two MoE configurations: Scout with $E = 16$ and Maverick with $E = 128$, both with $E_s = 1$, $k = 1$, and the FFN sublayer alternated between dense SwiGLU and MoE per layer (AI, 2026, §2.1).

deepen-cite: meta-llama4 §2.1 — Llama 4 release card / Meta release blog, citation-pending: the `meta-llama4` cite-key currently maps only the long-context / iRoPE detail; the MoE and native-multimodal architectural detail needs consolidation into a separate `meta-llama4-release` entry.

Position encoding is iRoPE (3:1 RoPE-to-NoPE) with temperature scaling on the NoPE layers (AI, 2026, §2.1, §2.4), trained natively at 256K. KV-sharing is GQA-8, inherited from LLaMA-3 (AI, 2024b, §3.1). Multimodality is native: image and text are jointly pretrained from scratch (AI, 2026, §2).

Mistral Large 2. Mistral Large 2 is dense SwiGLU throughout, with GQA-8 attention and RMSNorm + Pre-LN inherited from the Mistral 7B template (Jiang et al., 2023, §2).

deepen-cite: citation-pending: Mistral Large 2 release notes / Hugging Face model card, July 2024; `mistral-large-2` is not yet a references.bib entry.

Position encoding is RoPE; multi-token output is not disclosed beyond single-head autoregression. The Pixtral variant adds a projector + vision encoder (section 11); the text-only Mistral Large 2 is not multimodal.[74]

Gemma 3. Gemma 3 27B uses GQA with a 5:1 local-to-global interleave (sliding window 1024 tokens) of Gemma 2's pattern (Team and DeepMind, 2025, §2.1) — five sliding-window local-attention layers per one full global-attention layer. Normalization is the Peri-LN placement of various (2025c) combined with QK-Norm on the attention path (Team and DeepMind, 2025; Team, 2025, §2.1). Position encoding is RoPE; the FFN is dense SwiGLU; multimodality is native via a SigLIP-class vision encoder.

deepen-cite: gemma3 §3: multimodal pipeline detail; verify SigLIP variant against the Gemma 3 release card.

Qwen 2.5 and Qwen 3. Qwen 2.5 is dense SwiGLU with GQA, RMSNorm + Pre-LN, RoPE + YaRN, and a 152K-vocab BPE tokenizer.

deepen-cite: citation-pending: Qwen 2.5 release card; `qwen3` maps the Qwen 3 paper, not Qwen 2.5 architectural detail.

Qwen 3 (235B-A22B) replaces the dense FFN with a fine-grained MoE at $E = 128$, $E_s = 0$, $k = 8$, with global-batch load balancing in place of auxiliary-loss balancing (Team and Alibaba, 2025, §2.1). Crucially Qwen 3 keeps GQA rather than MLA, despite training in the same year as DeepSeek V3 — the lab-level KV-sharing split flagged in section 5. The multimodal line (Qwen 2.5-VL) uses a projector, not native multimodal.

[74] **Superseded 2025-12-02.** Mistral Large 3 (AI, 2025) crosses the Mistral flagship line over to MoE: 675B-total / 41B-active sparse MoE under Apache 2.0, 256K context, native multimodal. The `Mistral Large 2` row is retained here because its detail is the publicly-documented dense reference point Paper 1 §7 uses as a counter-example; on the dense-vs-MoE roster of §13.5, the Mistral row should now be read as MoE going forward, and the dense-vs-MoE split tilts further toward MoE than the row alone shows.

13.3 Undisclosed cluster: Claude, GPT, Gemini

The lower block of table 4 is the closed-source frontier. None of Anthropic (Claude 3.5 / 3.7 / 4-class), OpenAI (GPT-4o, o1 / o3, GPT-5), or Google DeepMind (Gemini 2.5) has published per-axis architectural detail at the level of the open labs. The model cards report capabilities, training data scale, context length, and multimodal coverage; they do not report KV-sharing strategy, attention-implementation kernel, FFN family, or normalization placement.

deepen-cite: Anthropic Claude 3.7 system card / OpenAI GPT-4o and GPT-5 system cards / Google Gemini 2.5 technical report — citation-pending: vendor system cards are discovery-only sources (Tier B/C in `theory/sources/README.md`) and cannot solely back hard architectural claims; consolidate into separate cite-keys once formal venues catch up.

What is publicly known is restricted to:

- *Tokenization family.* OpenAI's tiktoken family is publicly documented; Anthropic's BPE-class tokenizer is documented in the API reference but not in a primary technical report; Gemini uses a SentencePiece-class tokenizer disclosed in the Gemma 3 paper (Team and DeepMind, 2025, §2.3) as a sibling to Gemini's own.
- *Multimodal pattern.* GPT-4o, Claude 3.5 Sonnet onward, and Gemini 2.5 are native-multimodal (image + audio + text in a single pretraining); this is disclosed at capability level in each vendor's system card and is the third integration pattern of section 11.
- *Reasoning-mode toggle.* OpenAI o1 / o3 and (since 3.7) Claude expose a thinking-mode generation phase, mirroring the DeepSeek-R1 / Qwen 3 toggle (DeepSeek-AI, 2025; Team and Alibaba, 2025, §2). Whether this requires architectural support beyond training-stage changes is contested (sections 9 and 10) and not resolved by the closed-vendor disclosures.

Forum signal

Inference-stack reverse-engineering of closed-vendor models (latency patterns; per-token cost differentials; rough head-count estimation from KV-cache utilization scaling) circulates in tier-C community sources but is unreliable across point releases and unverifiable without vendor cooperation. We treat all three undisclosed cells as "ND" rather than risk laundering forum speculation as architectural fact. The convergence claim of this section is therefore restricted to the disclosed seven rows; whether the closed labs sit inside or outside the disclosed convergence cluster is a falsifiable prediction of section 14, not a claim of this section.

Caveat: closed-vendor row labels age faster than the architectural picture. The closed-row labels in table 4 ("Claude 3.7-class," "GPT-class (o-/4o/5)," "Gemini 2.5") are accurate as a 2025-Q1 vintage of the closed lineup. By 2026-Q2 each lab has shipped multiple point releases beyond those labels — Anthropic's Claude 4-family (May 2025) plus subsequent Opus 4.x and Sonnet 4.x updates; OpenAI's GPT-5 (August 2025) folding the o-series into a unified fast-vs-thinking tier, with subsequent GPT-5.x point releases; and Gemini 2.5 Deep Think (August 2025) with its parallel-thinking RL disclosure. The architectural picture this section's thesis depends on *does not move* across those point releases at any disclosed level: the closed rows remain "ND" on every architectural axis the table tracks, and the productionised exposure of inference-time compute (extended-thinking budgets, reasoning-effort levels, parallel-thinking modes) converges on the same API surface as the disclosed labs rather than diverging from it.

deepen-cite: closed-vendor system cards (Anthropic Claude 4-family, OpenAI GPT-5.x, Google Gemini 2.5 Deep Think) are tier-B per `theory/sources/README.md` and back the version-lineage / API-surface claims, not the architectural cells; those cells remain "ND."

We retain the 2025-Q1 row labels with this date stamp rather than extend them per release: the column-by- column architectural cells would still be "ND" for every newer label, and the row's load-bearing function in the table is the disclosure-ceiling, not the version stamp. The convergence claim of this section is therefore stable across the closed-row lineage shift between 2025-Q1 and 2026-Q2; the lineage shift itself is documented in section 14 as a separate question about whether the disclosure ceiling is moving.

13.4 Where the disclosed frontier agrees

Five of the eight axes show near-universal agreement across the seven disclosed rows of table 4.

1. **Normalization: RMSNorm + Pre-LN-class placement.** All seven disclosed models use RMSNorm with the norm placed before each sublayer rather than after the residual add. Gemma 3 and OLMo 2 extend this to Peri-LN with QK-Norm (Team and DeepMind, 2025; Team, 2025); the base choice (RMSNorm + Pre-LN + $1/\sqrt{N}$ residual init) is universal (section 8).

2. **Positional encoding: RoPE plus a long-context extension.** All seven disclosed rows apply RoPE to Q, K at every attention layer (section 4). The extension varies — vanilla, YaRN, iRoPE, decoupled-RoPE for MLA, interleaved local / global — but the base mechanism (multiplicative rotation of Q, K parameterised by $\theta_i = \theta^{-2(i-1)/d_h}$) is shared.

3. **FFN gating: SwiGLU.** Every disclosed model uses SwiGLU as the per-position FFN nonlinearity, whether the sublayer is dense or MoE. The 3-matrix $\mathrm{FFN}(x) = (W_3(\mathrm{Swish}(W_1 x) \odot W_2 x))$ form of Shazeer (2020, Eq. 6) is universal; the dense-vs-MoE split is on the multiplicity around it, not on the activation.

4. **KV-sharing: head-count reduction.** No disclosed 2026-frontier model uses full MHA. Every row sits at GQA-8, GQA-with-windowing, or MLA — all reducing H_{kv} below H_q. The lab-aligned split between GQA and MLA persists, but *some* form of head-count reduction is universal, and every major inference framework (FlashAttention, vLLM, SGLang) now ships first-class GQA and increasingly first-class MLA (Kwon et al., 2023; Dao, 2023).

5. **Attention implementation: FlashAttention-family exact attention.** Every disclosed model uses a FlashAttention-class exact-attention kernel: FA2 on Ampere, FA3 on Hopper, with MLA-aware variants for the DeepSeek stack (Dao, 2023; Shah et al., 2024; DeepSeek-AI, 2024b). NSA's three-branch sparse attention (Yuan et al., 2025) has not yet shipped at production scale at any disclosed lab; the formula $\mathrm{softmax}(QK^\top/\sqrt{d_h})V$ is unchanged across implementations.

Intuition

Five-of-eight convergence is striking but consistent with the rest of the paper: the axes that have converged are the ones where the *primary citation set* of section 4 through section 8 contains a single dominant lineage (RoPE, SwiGLU, RMSNorm + Pre-LN, head-count-reduced KV-sharing, FlashAttention-class implementation). The axes that remain split are the ones where the primary set still contains parallel lineages (MoE-vs-dense; cross-attention-

vs-projector-vs-native multimodal; GQA-vs-MLA at the lab level). Returning to canonical math: the residual-stream forward pass on a 2026 frontier model collapses to the same five-line equation (RMSNorm → rotated-Q, K attention with reduced H_{kv} → residual-add → RMSNorm → SwiGLU FFN or routed SwiGLU MoE → residual-add) across every disclosed row. The convergence is not a stylistic preference; it is a fixed point of the per-axis arguments in sections 3 to 8.

13.5 Where the disclosed frontier diverges

Three axes carry live splits in table 4.

KV-sharing strategy: GQA versus MLA. The split is lab-aligned, not capability-aligned. DeepSeek V2 / V3 / R1 use MLA throughout (DeepSeek-AI, 2024a,b, 2025); every other disclosed lab (Meta, Mistral, Google, Alibaba) uses GQA. The contradiction in section 5 is that MLA's quality story — matched MHA quality at 4× smaller KV-cache — is a single-source primary claim (DeepSeek-AI, 2024a) and has not been independently reproduced at frontier scale by a non-DeepSeek lab. The serving-stack maturity gap (FlashAttention's GQA path being older than its MLA path) and the recipe risk of MLA's decoupled-RoPE construction (section 5) are plausible explanations for the non-crossover. section 14 returns to this.

Dense versus MoE on the FFN. The dense-versus-MoE split is genuine but tilting further toward MoE through 2025–26. LLaMA-3 8B and 70B are dense (AI, 2024b); Qwen 2.5 base is dense; Mistral Large 2 was dense, but the lab's flagship line crossed to MoE with Mistral Large 3 (Dec 2025; AI, 2025: 675B-total / 41B-active sparse MoE). DeepSeek V3 is MoE with $E = 256$, $E_s = 1$, $k = 8$; Qwen 3 is MoE with $E = 128$, $E_s = 0$, $k = 8$; Llama 4 is hybrid (alternating dense and MoE layers); Mixtral 8×7B remains the canonical 8-expert classical MoE point. As discussed in section 7.6, this is no longer a question of whether to do MoE at scale — the answer is yes for the largest models in this list — but of which MoE design point: E, E_s, k, and load-balancing discipline. The contradiction in section 7 is the matched-active-parameter quality claim (DeepSeek-AI, 2024b,c): that fine-grained MoE matches dense quality at matched active parameters. Cross-lab reproduction at frontier scale is sparse, and Qwen 3's shared-expert-free choice (Team and Alibaba, 2025) explicitly disagrees with DeepSeek V3 on whether $E_s > 0$ helps.

Multimodal integration pattern. Three patterns coexist in table 4: native multimodal pre-training (LLaMA 4, Gemma 3 multimodal variant, Claude 3.5+, GPT-4o / 5, Gemini 2.5), projector + LLM (Qwen 2.5-VL, Pixtral / Mistral Pixtral), and the cross-attention-bridge variant (no current frontier model; Flamingo descendants are off-frontier). The cross-attention bridge of Alayrac et al. (2022) has effectively been replaced by the two simpler patterns; the live split is between projector and native multimodal. The vendor claim that native multimodal produces better quality at smaller active parameter count (section 11) is a forum signal pending controlled cross-lab reproduction, and the closed-vendor cluster's coverage here is uniform but undisclosed at architectural detail.

13.6 Closing: which axes have converged, which remain mixed

The empirical state of the disclosed 2026 frontier is summarized in two sentences. **The convergence is a five-axis fixed point across all disclosed labs**: RoPE-with-extension, RMSNorm in a

Pre-LN-class placement, SwiGLU per-position FFN, head-count-reduced KV-sharing (some form of GQA or MLA), FlashAttention-class exact attention; **the residual splits are concentrated in three axes**: KV-sharing strategy (GQA vs MLA, lab-aligned, single-source quality claim), dense-vs-MoE on the FFN multiplicity (genuine quality / serving-cost trade with cross-lab disagreement on $E_s > 0$), and multimodal integration pattern (projector vs native, no controlled cross-lab study). The undisclosed cluster (Claude, GPT, Gemini) is silent on all eight axes at architectural detail and provides no signal on whether it sits inside or outside the disclosed convergence neighbourhood.

The remaining contradictions surfaced through sections 4 to 7 and 9 to 12 — MLA-vs-MHA at frontier scale, NSA-vs-FA3 head-to-head, SSM-hybrid quality at frontier, the matched-active MoE quality gap, long-context extrapolation method comparison, MTP at non-DeepSeek scale, native-multimodal quality lift — are now visible as a single concentrated set. Section 14 treats them as the open frontier of the architecture story, and asks for each what evidence would close the gap. The five-axis convergence of this section is the substrate against which those open questions are posed; without naming the convergence, the contradictions cannot be localized; without localizing the contradictions, the field's remaining architectural questions cannot be made falsifiable.

14 Contradictions live and open frontiers

The previous twelve sections traced the convergence: along five of the eight axes the disclosed 2026 frontier models agree (§13), and the remaining three are split in a small number of camps. This section consolidates the `[CONTRADICTION]` markers raised across §**??**–§12 into an enumerated catalogue. For each open question we state the strongest form of each side's claim, list the evidence currently on the table, and name the experiment that would settle it. **The thesis of this section: the live architectural disputes are localized, falsifiable, and almost all bottlenecked by the same scarce resource — controlled head-to-head training runs at the $\geq$ 70B-active scale on independent training stacks. We predict that 2027-class disclosures will close §14.1, §14.2, and §14.6 along axes §5–§9, and that §14.4 and §14.5 will remain mixed for at least one further model generation.**

We use a uniform sub-template per contradiction: *strongest claim on each side*, *evidence on the table*, *experiment that closes it*. Where the supporting source for a side is single-laboratory or pre-print only, we mark it

deepen-cite:

so the next deepening pass can trace the lineage.

14.1 C1 — MLA vs. MHA at frontier scale

Pro-MLA (DeepSeek): Multi-head Latent Attention with the joint-low-rank decomposition W^{DKV}, W^{UK}, W^{UV} matches MHA quality at $\sim 4\times$ smaller KV cache; the cache absorption identity (§5, eq. MLA-absorb) means no quality penalty at decode (DeepSeek-AI, 2024a, §2.1.4). **Pro-GQA (default):** GQA with $G \in \{4, 8\}$ groups recovers MHA quality at matched cache budget on every published independent ablation (Ainslie et al., 2023, §3.1), and Western frontier labs that do disclose attention details (AI, 2024b, 2026; Team and DeepMind, 2025; Team and Alibaba, 2025) all stay on GQA.

Evidence on the table. The MLA-MHA matched-cache comparison is single-source (DeepSeek-AI, 2024a, §2.1.4): a 16B-active prototype on DeepSeek's training corpus. DeepSeek-AI (2024b) extends MLA to 671B total / 37B active with strong downstream benchmarks but does not publish

a head-to-head MLA-vs-MHA at matched compute at that scale. GQA's matched-cache neutrality result (Ainslie et al., 2023) has been replicated by multiple labs.

Experiment that closes it. A $\geq$ 70B-active controlled three-way ablation — MHA / GQA-8 / MLA at matched compute, matched data, matched optimizer — run by an independent laboratory on non-DeepSeek pre-training data. As of 2026-Q2 no such run is published.

deepen-cite: independent MLA-vs-GQA-vs-MHA at $\geq$ 70B-active scale

.

14.2 C2 — NSA vs. FA3 head-to-head

Pro-NSA: Native Sparse Attention with the three-branch (compressed / fine / sliding) selection scheme delivers 11.6× decode at 64K and improves long-context reasoning quality over the dense baseline, because pre-training with sparse attention teaches the model to use it (Yuan et al., 2025). **Pro-FA3:** FlashAttention-3 with H100 asynchronous warp-specialization reaches $\sim$ 75% of H100's theoretical peak FP16 throughput on dense attention (Shah et al., 2024)

deepen-cite: shah2024 §3 (excerpt not yet in KB; first-use anchor in §6)

; community indications are that NSA's 11.6× margin shrinks substantially when the dense baseline is FA3 rather than FA2 and the context length is closer to 32K than 64K.

Evidence on the table. Yuan et al. (2025) compare against FA2-on-GQA, not FA3, leaving the FA3 baseline unexamined. NSA's training-time recipe is also tied to the DeepSeek pre-training stack; no third-party laboratory has published a Hopper reproduction at matched precision and matched task suite as of 2026-05.

Experiment that closes it. A controlled NSA-vs-FA3 benchmark suite on H100 / H200 with matched precision (BF16 or FP8), matched sequence lengths $\{8K, 16K, 32K, 64K\}$, and matched downstream evaluation (RULER, HELMET, not just NIAH). The decode-throughput claim is the easy half; the harder claim is whether NSA's quality lift survives when the baseline is FA3 rather than FA2 and when the pre-training data is not DeepSeek's.

deepen-cite: NSA-vs-FA3 third-party head-to-head on Hopper

.

14.3 C3 — SSM-attention hybrid quality at frontier

Pro-hybrid: A small attention budget interleaved with selective SSM blocks (et al. , AI21 Labs 1:7, Samba $\sim$1:1) gives Pareto-better long-context FLOP cost than attention-only, because Mamba-2's state-space-duality theorem (Dao and Gu, 2024, §3) bounds the loss at small r. **Pro-attention-only:** The publicly deployed top-quality 2026 lineup (DeepSeek-V3, Llama-4, Qwen3, Mistral Large) is uniformly attention-based; no SSM-hybrid appears in the top quality tier (DeepSeek-AI, 2024b; AI, 2026; Team and Alibaba, 2025; Jiang et al., 2023), suggesting that whatever advantage the SSD theorem predicts is not yet operationalized at the 100B-active scale.

Evidence on the table. Vendor demos at sub-frontier scale (Jamba 52B total / 12B active; Zamba2 7B; Hymba 1.5B) show competitive within-tier quality, but neither the optimal interleaving ratio nor the attention-block count is settled by theory or by a controlled ablation. The negative evidence is the absence of frontier deployment, which is informative but not dispositive — the SSM line has had less engineering investment than attention, and that confound is not controlled.

Experiment that closes it. Three matched-compute training runs at $\geq$ 70B-active: pure-attention, hybrid 1:7 (Jamba-style), hybrid $\sim$1:1 (Samba-style). Evaluate on RULER / HELMET / 1M-token reasoning tasks at deployment-scale serving cost.

deepen-cite: ssm-frontier-quality, see section 9.5

.

14.4 C4 — MoE quality at matched active parameters

Pro-MoE: Fine-grained MoE with $E = 256$, $k = 8$, $E_s = 1$ shared expert at 37B-active matches or exceeds dense LLaMA-3 405B on benchmark suites (DeepSeek-AI, 2024b, abstract), with the shared-expert absorbing always-on capability and the fine-grained routing supplying specialization (DeepSeek-AI, 2024c). **Pro-dense (or shared-free MoE):** Team and Alibaba (2025) reach comparable quality with $E_s = 0$ and global-batch load balancing, contradicting the shared-expert claim. The absence of independent third-party reproductions at frontier scale leaves the matched-active gap empirically open; secondary aggregations (largo.dev 2026) treat the gap as closed but the supporting controlled ablations are scarce.

Evidence on the table. Primary-source claims for MoE's matched-active parity come from DeepSeek and Qwen labs; the labs disagree among themselves on shared-expert necessity. No published non-Chinese-lab frontier MoE training run reports its dense-MoE gap. Anthropic's Claude family architecture is undisclosed and may not be MoE at all (see `kb/index/contradictions.md#architecture/trans`

Experiment that closes it. A non-DeepSeek, non-Qwen lab publishes a $\geq$ 100B-total / $\geq$ 30B-active fine-grained MoE trained with documented E, k, E_s, with a matched-active dense baseline at the same compute budget on the same pre-training data, and reports the gap on standard benchmarks. We expect this in 2027 from either Mistral, Meta (Llama-5 generation), or an Allen-AI / OLMo follow-up.

deepen-cite: matched-active MoE-vs-dense gap, independent-lab reproduction

.

Contradiction

The shared-experts sub-question (a sub-contradiction within C4) is itself unresolved within the disclosed Chinese-lab MoE family. DeepSeek-AI (2024b) report on-loss regressions when ablating the shared expert; Team and Alibaba (2025) drop E_s and report comparable quality. The reconciling claim — that shared-experts help when the routing is top-k with very fine-grained E, and not when global-batch balancing is sufficiently aggressive — is currently an `[INTUITION]`, not a derivation. A controlled ablation that varies (E, k, E_s) on a matched-compute grid would settle it.

14.5 C5 — Long-context extrapolation method

Pro-RoPE-with-NTK-scaling: For the $\leq 4\times$ extension regime (e.g. LLaMA-3's bump from 8K to 128K), NTK-aware base rescaling on RoPE (Su et al., 2021) is sufficient and is what most disclosed 2026 models actually do. **Pro-YaRN:** For the $\sim 10\times$ regime, per-frequency schedule (YaRN) is required to avoid out-of-distribution rotational frequencies; the original Su et al. (2021) embedding under naive scaling collapses on needle-in-haystack at 32K-trained-256K-evaluated.

deepen-cite: peng2023-yarn primary-source numbers

. **Pro-iRoPE / LongRoPE2:** For the $\geq 40\times$ regime (Llama-4's 10M, LongRoPE2's 2M), full retraining with mixed RoPE/NoPE layers and per-position search beats both NTK-aware and YaRN (various, 2025b; AI, 2026).

Evidence on the table. Each camp's benchmarks are largely single-source, and the benchmarks themselves disagree: NIAH-style retrieval saturates at $\sim 100\%$ for all three at $\leq$ 1M, while RULER and HELMET show methodologically inconsistent degradation patterns past 32K, making "which extension method is best" contingent on which long-context evaluation is treated as ground truth.

Experiment that closes it. A standardized long-context benchmark (RULER + HELMET + a 1M-token reasoning suite) applied to a single base model with three deterministically-trained extension recipes, with full disclosure of the per-frequency schedule. We do not expect this in 2026; the closing depends on benchmark consensus, which the field has not yet built.

Intuition

The three positions are not actually competing for the same regime. Read horizontally on the extension factor: NTK-scaling owns the short-extension regime where the unmodified rotational frequencies are still mostly in-distribution; YaRN owns the medium-extension regime where per-frequency correction is needed but a single base-rate adjustment still works; iRoPE / LongRoPE2 own the long-extension regime where the rotational backbone alone is insufficient and content-derived position channels (NoPE layers) must absorb the residual. Returning to the canonical math: the inner-product invariance $\langle R_m q, R_n k\rangle = \langle q, R_{n-m} k\rangle$ from Su et al. (2021, §3.4) only protects relative position when $|n-m|$ stays in the trained distribution of phases. NTK-scaling, YaRN, and iRoPE are three points on a hierarchy of how much of the rotational identity each method preserves.

14.6 C6 — MTP at non-DeepSeek scale

Pro-MTP-as-quality: DeepSeek-AI (2024b) report a quality lift from the multi-token prediction auxiliary objective itself, not just from inference-time draft-token use; the auxiliary gradient pressure makes hidden states more predictive of the next several tokens jointly (DeepSeek-AI, 2024b, §2.2). **Pro-MTP-as-speedup-only:** Gloeckle et al. (2024) present MTP primarily as a speedup mechanism, with a smaller per-task quality lift than DeepSeek-V3 reports; the gap between the two papers is not yet ablated.

Evidence on the table. The two primary sources frame MTP differently and report quality lifts of different magnitudes on different benchmark suites. No $\geq$ 70B-active non-DeepSeek training run has published an MTP ablation at frontier scale; the question of whether the quality lift survives outside DeepSeek's training recipe is open.

Experiment that closes it. A non-DeepSeek lab trains two matched-compute runs at $\geq$ 70B-active — with-MTP and without-MTP — on the same data and reports the quality gap on a standardized benchmark suite. The cost is small relative to a frontier training run (MTP adds $\sim$1%–3% training compute) and the information value is high, so we expect this within 12 months.

deepen-cite: MTP at non-DeepSeek frontier scale

.

14.7 Subsidiary open questions raised in passing

Three further `[CONTRADICTION]` markers from §??–§12 are not load-bearing for Paper 1's convergence thesis but deserve naming. **C7 — Tied-vs-untied embeddings at scale.** The historical Press and Wolf (2017) result favored tying; disclosed frontier-2026 models all untie (AI, 2024b; Team, 2025; DeepSeek-AI, 2024b; Team and Alibaba, 2025); the reconciling `[INTUITION]` (gradient-imbalance dominates only when $|V|d$ is no longer a large fraction of total parameters) is not a derivation. A

controlled ablation across two or three model sizes spanning the crossover would settle it. **C8 — Medusa verification rule.** Medusa with rejection sampling is distribution-preserving; Medusa with typical acceptance is not (§10, eq. (45)). Vendor reports of "Medusa speedup" sometimes elide which rule was used; a single deployment audit would close the reporting gap, though the underlying mathematics is settled. **C9 — NVFP4 production quality.** Vendor materials report NVFP4 as production-ready with quality matching FP8; third-party reproduction at frontier scale is sparse. Independent reproductions through 2026-Q4 should settle whether NVFP4 inference is a tier-A production claim or a tier-B vendor claim.

14.8 What the catalogue says about the field

Six load-bearing contradictions and three subsidiary ones is a small number for a field this active. Two structural facts emerge. **First**, the live disputes are localized to specific axes — KV-sharing (§14.1), hardware implementation (§14.2), the dense–SSM frontier (§14.3, §14.4), long-context method (§14.5), and multi-token output (§14.6) — not distributed across the entire architecture. The normalization, positional-encoding-base, FFN-activation, and tokenization axes are quietly settled within ±1 design choice. **Second**, the closing experiment for almost every contradiction is the same: an independent laboratory at $\geq$ 70B-active runs a controlled head-to-head on a non-DeepSeek pre-training stack. This is a small number of experiments, not a research program. The bottleneck is not theory but compute access and a willingness to publish negative or null results.

We therefore predict that the *shape* of the architectural convergence visible in §13 will not change in 2027-class disclosures: the same five axes will be settled, three will remain split, and the splits will narrow. We further predict that §14.1, §14.2, and §14.6 are the contradictions most likely to close — they are the cheapest to adjudicate, the most pre-trainable as side-by-side ablations, and already have at least one well-funded non-DeepSeek lab with the incentive to publish. §14.4 and §14.5 will remain mixed for at least one further generation, the former because the dense-MoE economics depend on serving regime (§12) which differs across labs, the latter because the long-context evaluation suite has not yet converged.

This closes Paper 1. Papers 2 (training and post-training), 3 (reasoning models), and 4 (interpretability) inherit this catalogue's discipline: each has its own `[CONTRADICTION]` cluster, and their synthesis sections will follow the same enumerate-claim-evidence- experiment template used here.

Glossary

ALiBi Attention with Linear Biases. Parameter-free additive bias $-|m-n| \cdot s_h$ at attention logits, with per-head slope. Used by BLOOM, MPT. (Press et al. 2021, arXiv 2108.12409)

attention weights The softmax-normalized scores ($a_j \geq 0$, $\sum a_j = 1$) that determine how much each position contributes to the output.

Autoregressive Generating tokens one at a time, left to right, where each prediction depends only on previous tokens.

AWQ (Activation-aware Weight Quantization) Lin et al. 2023. Per-channel weight scaling driven by *activation* magnitudes (preserving 1

BitLinear BitNet's drop-in replacement for 'nn.Linear': weights quantized to $\{-1, 0, +1\}$ via absmean, activations to INT8 via absmax, SubLN replacing LayerNorm. Constructs a model

whose linear-algebra reduces to integer addition. (et al. , Microsoft, §2.1; kb/excerpts/ma2024-bitnet#bitlinear)

BPE (Byte Pair Encoding) Subword tokenization algorithm that iteratively merges the most frequent adjacent character pairs to build a vocabulary.

Chain-of-thought (CoT) A prompting and decoding pattern in which the model produces an intermediate sequence of reasoning steps z before emitting a final answer y. In few-shot CoT (Wei 2022), the prompt contains exemplars with hand-written reasoning traces; in zero-shot CoT (Kojima 2022), the trigger phrase "Let's think step by step" elicits the same behaviour without exemplars. (et al. , Google Brain, §1; kojima2022 §3)

Circuit a sub-graph C of a model's computational graph M

Claude Anthropic's proprietary decoder-only LLM family. No public architecture paper; structural family shares the decoder-only Transformer lineage.

Compressed FSM (SGLang) FSM transformation that merges chains

Constrained decoding Generation procedure that, at each step,

Context extension Sub-stage of late-stage pre-training that increases trained sequence length (e.g., 4 K $\rightarrow$ 32 K $\rightarrow$ 128 K) by adjusting RoPE scaling and continuing on long-context data. (DeepSeek-AI, 2024b, §1)

Context-independent / context-dependent vocabulary split (XGrammar) XGrammar's core trick: precompute (offline) the mask for tokens whose grammar-allowed-or-not status depends only on the grammar state (99

cross-attention Attention where Q comes from the decoder and K, V come from the encoder output. Removed in decoder-only models.

Data Mixing Law (Eq. 1) $L_i(\mathbf{r}) = c_i + k_i \exp(\sum_j t_{ij} r_j)$. Validation loss on domain i as an exponential of a linear combination of training-mixture proportions r_j. Coefficients (c_i, k_i, t_{ij}) are fit per target domain. (et al., 2024, §1, eq.1)

decoder-only Simplified Transformer architecture (GPT, LLaMA) that removes the encoder and cross-attention, using only masked self-attention and FFN layers.

encoder-decoder The original Transformer layout: an encoder processes the full input bidirectionally, a decoder generates output autoregressively, with cross-attention connecting them.

Feature a direction $\mathbf{d} \in \mathbb{R}^n$ in some

FFN (Feed-Forward Network) A position-wise two-layer (or three-layer for GLU variants) fully-connected network applied independently at each sequence position. Transforms representations within each position, complementing attention which mixes across positions.

FlashAttention Exact attention computed with re-organized memory access: tile K, V into SRAM-resident blocks and use **online softmax** to combine block results, never materializing the $N \times N$ attention matrix in HBM. Forward + backward IO complexity $\Theta(N^2 d^2 M^{-1})$ vs. $\Theta(Nd + N^2)$ for the standard implementation '[dao2022 §3]'.

FP8 (E4M3 / E5M2) IEEE-style 8-bit floating-point. E4M3: 4-exp / 3-mantissa, narrower range / better resolution (typical for activations). E5M2: 5-exp / 2-mantissa, wider range (typical for gradients). Production choice from H100 onward. (Shah et al., 2024, §1; kb/notes/inference/quantization#§4)

GBNF (GGML BNF) llama.cpp's grammar specification format.

GEGLU GLU with GELU activation: $(\text{GELU}(xW) \otimes xV)W_2$. Equivalent quality to SwiGLU; used in some Google models (T5.1.1, GLaM, mT5). (Shazeer, 2020, §2 Eq.6)

GELU (Gaussian Error Linear Unit) Smooth activation $x \cdot \Phi(x)$ where Φ is the standard normal CDF. Used by GPT-1 (2018).

Global-batch load balancing Qwen3's strategy: apply load-balancing loss across the entire training mini-batch (cluster-wide), not within microbatches. Compensates for dropping shared experts. [qwen3]

GLU (Gated Linear Unit) $\text{GLU}(x, W, V) = \sigma(xW) \otimes (xV)$, the elementwise product of a sigmoid-gated and an identity-gated linear projection. Building block for FFN-GLU variants. (Shazeer, 2020, §2 Eq.4)

GPTQ Frantar et al. 2022. OBQ-style second-order weight update with three engineering modifications: arbitrary fixed quantization order, lazy batched updates (block size 128), and Cholesky reformulation for Hessian-inverse stability. 4 GPU-hours for 175B at 3–4 bit. (Frantar et al., 2022, §3; kb/notes/inference/quantization#§3.1)

GQA (Grouped-Query Attention) Interpolation between MHA and MQA: query heads are partitioned into g groups, each group shares one W^K, W^V. GQA-1 = MQA; GQA-h = MHA. KV cache per token is $2n_g d_h$ elements '[ainslie2023 §2.2]'.

HBM (High-Bandwidth Memory) The large, slow memory tier on a GPU (40–80 GB on A100, 1.5–2.0 TB/s). Most operations in standard attention are HBM-bound '[dao2022 §2.1]'.

hidden state The d_{model}-dimensional vector representation at each position, flowing through the residual stream across layers.

HiPPO "High-order Polynomial Projection Operator." Specific structured choice of A matrix that optimally compresses input history under polynomial bases; foundation for S4. (Gu et al. 2020)

Interleaved-MRoPE Qwen3-VL refinement of M-RoPE that interleaves the temporal/height/width axes per dimension-pair instead of partitioning the channel range; every dimension-pair sees all three axes.

iRoPE "interleaved RoPE." LLaMA 4's position scheme: alternates RoPE-equipped attention layers with NoPE attention layers (3:1 ratio) plus temperature scaling in the NoPE layers. Trained at 256K, claimed to extrapolate to 10M-token inference. [meta-llama4]

Key (K) Projected representation of positions being "queried" against. The Q·K dot product determines attention weights.

KV cache The per-layer cache of key/value tensors $K_{1:t}, V_{1:t}$ produced during autoregressive decoding so each new token's attention is $O(t)$ rather than $O(t^2)$. Size per token per layer is $2 \cdot n_{\text{kv}} \cdot d_h \cdot$ bytes. (Kwon et al., 2023, §2; kb/notes/inference/kv-cache-management#§1)

Layer Normalization (LayerNorm) Normalizes across the feature dimension per position: subtract mean, divide by standard deviation, apply learned scale (γ) and bias (β).

LLM Large Language Model. A neural network with billions of parameters trained on large text corpora to predict and generate language.

Logits Unnormalized log-probabilities over the vocabulary, produced by the output projection ($h_{\text{final}} \cdot W_e^\top$).

LSTM Long Short-Term Memory. An RNN variant with gating mechanisms to better preserve long-range dependencies.

M-RoPE Multi-axis RoPE for vision-language models. Splits the per-head dimension d_h into temporal/height/width thirds; each third gets its own RoPE rotation. Introduced by Qwen2-VL.

MHA (Multi-Head Attention) The Vaswani 2017 default: h independent heads, each with its own W_i^Q, W_i^K, W_i^V. KV cache size per token per layer is $2n_h d_h$ elements '[vaswani2017 §3.2.2]'.

Mistral Open-weights decoder-only LLM family from Mistral AI (Jiang et al., 2023). 7B variant uses sliding-window attention and grouped-query attention on top of the LLaMA-style architecture.

MQA (Multi-Query Attention) Variant where all heads share a single W^K, W^V (only W^Q remains per-head). KV cache shrinks by factor h. Cheaper per-token decode at the cost of capacity and stability '[shazeer2019 §2.4; kb/notes/architecture/attention-mechanism.md#sec-3-1]'.

multi-head attention Running h parallel attention operations on different learned projections, then concatenating and projecting the results. Allows attending to different representation subspaces simultaneously.

Multi-head Latent Attention (MLA) DeepSeek's architectural compression: project to a low-rank latent $C \in \mathbb{R}^{d_c}$ shared across heads, expand back to per-head (K, V) at attention time. Achieves 93

Multi-token prediction (MTP) Training objective + architectural feature: in addition to the standard next-token head, the model has auxiliary heads predicting tokens at offsets $t+2, t+3, \ldots, t+k$. Used by DeepSeek-V3. (DeepSeek-AI, 2024b, §2.3)

Native Sparse Attention Three-branch sparse attention (compress + select + sliding-window) trained natively (not retrofit). GQA-group-aligned blockwise selection enables FlashAttention-style kernels under sparsity. (Yuan et al., 2025, §3.2–3.4)

NoPE "No Position Encoding." Decoder-only Transformer trained without any explicit positional encoding; relies on the causal mask to recover position information implicitly. Better length generalization on synthetic tasks, but underperforms on realistic LLM workloads. (Kazemnejad et al. 2023)

NSA (Native Sparse Attention) Yuan et al. 2025: hardware-aligned natively sparse attention trained end-to-end (sparsity is part of pre-training, not retrofitted). Aimed at long context. Treated in 'kb/notes/architecture/long-context.md' '[yuan2025]'.

NVFP4 4-bit floating-point format (E2M1) with native

Online softmax Numerically stable streaming softmax: maintain running max m and normalizer ℓ across blocks; when a new block arrives, rescale the existing partial sum by $e^{m_{\text{old}}-m_{\text{new}}}$ before adding the block's contribution. Enables block-wise softmax without seeing the full row '[dao2022 §3.1]'.

PagedAttention vLLM's KV-cache layout: KV is stored in fixed-size **blocks** (typically 16 tokens), each sequence holds a per-layer **block table** mapping logical block indices to physical blocks, and attention is computed block-wise per Eq.4. Eliminates the contiguous-allocation requirement that produced 50–90

PaLM Google's 540B-parameter decoder-only LLM (Chowdhery et al., 2022). Uses SwiGLU, RoPE, parallel attention/FFN layers. Reference point for scaling experiments.

Parallel scan GPU-efficient associative-scan algorithm used by Mamba to compute the input-dependent recurrence in $O(N \log N)$ during training while the recurrence is sequential. (Blelloch 1990; Mamba uses Smith et al. 2022's hardware-aware variant.)

Parameters The learned weights of the model (embedding matrices, projection matrices, normalization scales, etc.).

Perplexity $2^{\mathcal{L}}$ (or $e^{\mathcal{L}}$ with natural log). Measures how "surprised" the model is by the data. Lower is better.

Position interpolation (PI) Simplest RoPE extension: scale positions by $s = L_{\text{test}}/L_{\text{train}}$. Cheap, degrades short-context quality. (Chen et al. 2023)

Post-LN Original Transformer placement: normalize after the residual addition. Requires learning rate warmup for stable training.

Pre-LN Modern placement: normalize before the sub-layer, then add residually. Gradients scale as $O(d\sqrt{\ln d/L})$, enabling stable training without warmup.

Precision pinning (DeepSeek-V3) Components held at BF16/FP32 even with FP8 elsewhere: embedding module, output head, MoE gating, normalization, attention. (DeepSeek-AI, 2024b, §3.3.1)

Query (Q) Projected representation of the position that is "asking" for information in attention.

RadixAttention SGLang's cross-request KV reuse: a radix tree keyed on token-id sequences indexes which prefixes are still in cache, with LRU + reference-count eviction and cache-aware DFS scheduling that runs requests in tree-DFS order to maximise hit rate. (Zheng et al., 2024, §3, Theorem 3.1; kb/excerpts/zheng2024-sglang#radix)

Recall bottleneck SSM weakness: fixed-size state $\boldsymbol{h}_t$ cannot store every prior token's exact features, so exact-retrieval tasks (copy, NIAH) underperform attention. Primary motivation for hybrids.

Rejection sampling Llama-3 SFT quality gate: sample N candidates from the model, score by reward model, keep top-k, SFT on those. Used as a training-loop stage in modern multi-round pipelines '[meta-llama3]'.

Relative position encoding (T5 bias) Per-(query, key) distance bias added to attention logits; binned by relative distance. (Su et al., 2021, §2.3)

ReLU Rectified Linear Unit: $\max(0, x)$. Original Transformer activation (2017).

residual connection (Skip Connection) Adding the sub-layer input directly to its output (output $= x + \text{SubLayer}(x)$). Enables gradient flow and makes each layer learn a correction rather than a full representation.

Residual stream the linear data bus running through a

RLHF (Reinforcement Learning from Human Feedback) The 3-stage post-training pipeline: SFT $\rightarrow$ reward-model training on pairwise preferences $\rightarrow$ PPO with KL anchor against π^{SFT}. Canonical reference is InstructGPT '[ouyang2022-instructgpt §3]'.

RMSNorm (Root Mean Square Normalization) Simplified LayerNorm that drops mean-centering and bias, normalizing only by the RMS of the input. 7–64

RNN Recurrent Neural Network. Processes sequences one step at a time, maintaining a hidden state. Predecessor to the Transformer.

RoPE (Rotary Position Embedding) Positional encoding that rotates query and key vectors by position-dependent angles, making attention dot products depend on relative position. Applied at every layer to Q and K only. Used by LLaMA and most modern LLMs.

S4 Structured State Space sequence model with HiPPO-NPLR A, FFT-based convolutional training, recurrent inference. Dual-form architecture. (Gu, Goel, Ré 2022)

scaled dot-product attention Core attention operation: $\text{softmax}(QK^\top/\sqrt{d_k})V$. Scaling by $\sqrt{d_k}$ prevents large dot products from saturating the softmax.

Selective SSM (Mamba) Mamba's contribution: input-dependent $\bar{B}, C, \Delta$ (parameters produced by linear projection of current input), enabling content-aware selection. Breaks convolutional dual; uses parallel scan for training-time parallelism. [gu2023-mamba]

self-attention Mechanism where each position in a sequence computes a weighted sum over all positions, with weights determined by learned query-key similarity.

SentencePiece A tokenizer that operates on raw byte sequences rather than Unicode characters, used by LLaMA. Provides byte-fallback so any input can be tokenized.

Sliding-window attention Each position attends only to the previous w tokens; compute $O(Nwd)$ vs $O(N^2d)$. Local-only by construction; relies on stacked layers' growing receptive field for long-range dependencies. (Longformer 2020, Mistral 2023)

Softmax Converts logits to a probability distribution: $P(j) = e^{z_j} / \sum_k e^{z_k}$.

Speedup theorem (Leviathan) $\mathbb{E}[\text{tokens per cycle}] = (1-\alpha^{\gamma+1})/(1-\alpha)$ (Eq.1, capped geometric). Walltime improvement $= [(1-\alpha^{\gamma+1})/(1-\alpha)]/(c\gamma+1)$ with $c = T_q/T_p$ (Theorem 3.8). (Leviathan et al., 2023, §3.2, Eq.1, Theorem 3.8; kb/excerpts/leviathan2023#theorems)

State Space Duality (SSD) Mamba-2's framing: selective SSMs and a structurally-restricted form of attention compute the same function. The restriction: attention matrix is lower-triangular semi-separable with rank $\leq d$. [mamba2]

State-space model (SSM) Sequence model with recurrent state $\boldsymbol{h}_t$ updated linearly: $\boldsymbol{h}_t = \bar{A}\boldsymbol{h}_{t-1} + \bar{B}u_t$. Per-token compute $O(d^2)$, sequence-linear. (Gu, Goel, Ré 2022)

SwiGLU $\mathrm{FFN}_{\mathrm{SwiGLU}}(x) = (\mathrm{Swish}_1(xW) \otimes xV)W_2$. Three-matrix FFN with Swish gating. Universal in modern decoder-only LLMs. (Shazeer, 2020, §2 Eq.6)

T5 (Text-to-Text Transfer Transformer) Google's encoder-decoder Transformer (Raffel et al., 2019) that frames every NLP task as text-to-text. Originator of several conventions reused elsewhere (e.g., SentencePiece vocab, relative positional bias).

Tiling Restructuring a computation to operate on sub-blocks that fit in fast memory. Used in FlashAttention to keep Q, K, V blocks in SRAM '[dao2022 §3.1]'.

token embedding A learned lookup table ($W_e \in \mathbb{R}^{V \times d_{\text{model}}}$) mapping each token index to a dense vector.

Tokenization Converting raw text into a sequence of discrete integer token IDs that a model can process.

Transformer Neural network architecture introduced by Vaswani et al. (2017) that replaces recurrence with self-attention, allowing parallel processing of sequences.

Tree attention (Medusa/EAGLE) A draft tree of token candidates verified in one target forward pass via a custom attention mask that lets every node attend only to its ancestors. Enables draft-width > 1 per position. (Cai et al., 2024, §2.2; kb/excerpts/cai2024-medusa#tree-attention)

Typical acceptance Medusa's quality–throughput knob: accept any draft token whose probability under the target exceeds $\min(\epsilon, \delta \cdot \exp(-H(p)))$. Trades exact-distribution preservation for higher acceptance and longer accepted runs. (Cai et al., 2024, §2.4; kb/excerpts/cai2024-medusa#typical-acceptance)

Uptraining The Ainslie 2023 recipe for converting an existing MHA checkpoint to GQA/MQA: mean-pool the W^K, W^V matrices within each group, then continue pre-training for $\alpha \approx 5\%$ of the original budget '[ainslie2023 §2.1]'.

Value (V) Projected representation of the content to be aggregated. Weighted by attention scores to produce the output.

Vocabulary (V) The fixed set of all tokens a model can recognize. Size ranges from 32K (LLaMA) to 50K (GPT-3).

Warmup Gradually increasing the learning rate from zero during early training steps. Critical for Post-LN; unnecessary for Pre-LN.

weight tying Sharing the same weight matrix W_e for both input embeddings and the output projection to logits, halving vocabulary-related parameters.

YaRN "Yet another RoPE extensioN method." Combines NTK-aware scaling with a per-frequency interpolation schedule (no scaling on short-wavelength dims, full scaling on long-wavelength dims, smooth ramp between) and an attention-temperature rescale. $10\times$ more token-efficient than position interpolation. (Peng et al. 2023, arXiv 2309.00071)

References

Meta AI. Byte latent transformer: Patches scale better than tokens. In *ACL 2025*, 2024a. URL https://arxiv.org/abs/2412.09871. KB excerpt: kb/excerpts/blt2024.md.

Meta AI. The llama 3 herd of models. Meta Tech Report, 2024b. URL https://arxiv.org/abs/2407.21783.

Meta AI. The llama 4 herd: Architecture, training, evaluation, and deployment notes. Meta Tech Report, 2026. URL https://arxiv.org/abs/2601.11659.

Mistral AI. Mistral 3: introducing mistral large 3 and ministral. Mistral AI release blog (2 December 2025), 2025. URL https://mistral.ai/news/mistral-3.

Ainslie, Lee-Thorp, de Jong, Zemlyanskiy, Lebrón, and Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. In *EMNLP*, 2023. URL https://arxiv.org/abs/2305.13245. KB excerpt: kb/excerpts/ainslie2023.md.

Alayrac, Donahue, Luc, Miech, Barr, Hasson, Lenc, Mensch, Millican, Reynolds, Ring, Rutherford, Cabi, Han, Gong, Samangooei, Monteiro, Menick, Borgeaud, Brock, Nematzadeh, Sharifzadeh, Binkowski, Barreira, Vinyals, Zisserman, and Simonyan. Flamingo: a visual language model for few-shot learning. In *NeurIPS 2022*, 2022. URL https://arxiv.org/abs/2204.14198. KB excerpt: kb/excerpts/alayrac2022-flamingo.md.

Ba, Kiros, and Hinton. Layer normalization. arXiv:1607.06450, 2016. URL https://arxiv.org/abs/1607.06450. KB excerpt: kb/excerpts/ba2016-layernorm.md.

Cai, Li, Geng, Peng, Lee, Zhang, Chen, and Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. In *ICML 2024*, 2024. URL https://arxiv.org/abs/2401.10774. KB excerpt: kb/excerpts/cai2024-medusa.md.

Chen, Borgeaud, Irving, Lespiau, Sifre, and Jumper. Accelerating large language model decoding with speculative sampling. DeepMind tech report (arXiv), 2023. URL https://arxiv.org/abs/2302.01318.

Chowdhery, Narang, Devlin, and et al. Palm: Scaling language modeling with pathways. JMLR, 2022. URL https://arxiv.org/abs/2204.02311.

Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. arXiv, 2023. URL https://arxiv.org/abs/2307.08691.

Dao and Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. In *ICML*, 2024. URL https://arxiv.org/abs/2405.21060.

Dao, Fu, Ermon, Rudra, and Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *NeurIPS*, 2022. URL https://arxiv.org/abs/2205.14135. KB excerpt: kb/excerpts/dao2022.md.

Google DeepMind. Gemini 2.5 technical report. arXiv (DeepMind Tech Report), 2025. URL https://arxiv.org/abs/2507.06261.

DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. arXiv (DeepSeek Tech Report), 2024a. URL https://arxiv.org/abs/2405.04434. KB excerpt: kb/excerpts/deepseek-v2.md.

DeepSeek-AI. Deepseek-v3 technical report. arXiv (DeepSeek Tech Report), 2024b. URL https://arxiv.org/abs/2412.19437. KB excerpt: kb/excerpts/deepseek-v3-training.md.

DeepSeek-AI. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. arXiv (DeepSeek Tech Report), 2024c. URL https://arxiv.org/abs/2401.06066.

DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv (Nature 2025), 2025. URL https://arxiv.org/abs/2501.12948. KB excerpt: kb/excerpts/deepseek-r1.md.

Devlin, Chang, Lee, and Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *NAACL 2019*, 2019. URL https://arxiv.org/abs/1810.04805.

Li et al. Eagle-3: Scaling up inference acceleration via training-time test. In *NeurIPS 2025*, 2025. URL https://arxiv.org/abs/2503.01840.

Liu et al. Ring attention with blockwise transformers for near-infinite context. arXiv, 2023. URL https://arxiv.org/abs/2310.01889.

Liu et al. Data mixing laws: Optimizing data mixtures by predicting language modeling performance. In *ICLR 2025*, 2024. URL https://arxiv.org/abs/2403.16952. KB excerpt: kb/excerpts/data-mixing-laws-2024.md.

Lieber et al. (AI21 Labs). Jamba: A hybrid transformer-mamba language model. arXiv (AI21 Tech Report), 2024. URL https://arxiv.org/abs/2403.19887.

Wei et al. (Google Brain). Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022. URL https://arxiv.org/abs/2201.11903. KB excerpt: kb/excerpts/wei2022-cot.md.

Abdin et al. (Microsoft). Phi-4 technical report. Microsoft Tech Report, 2024a. URL https://arxiv.org/abs/2412.08905. KB excerpt: kb/excerpts/phi4.md.

Ma et al. (Microsoft). The era of 1-bit llms: All large language models are in 1.58 bits. arXiv, 2024b. URL https://arxiv.org/abs/2402.17764. KB excerpt: kb/excerpts/ma2024-bitnet.md.

Jiang et al. (Mistral AI). Mixtral of experts. arXiv (Mistral Tech Report), 2024. URL https://arxiv.org/abs/2401.04088.

Dong et al. (MLC). Xgrammar: Flexible and efficient structured generation engine for llms. MLSys 2025, 2024. URL https://arxiv.org/abs/2411.15100. KB excerpt: kb/excerpts/xgrammar2024.md.

Fedus, Zoph, and Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. JMLR, 2021. URL https://arxiv.org/abs/2101.03961.

Frantar, Ashkboos, Hoefler, and Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. In *ICLR 2023*, 2022. URL https://arxiv.org/abs/2210.17323. KB excerpt: kb/excerpts/frantar2022.md.

Gloeckle, Idrissi, Roziere, Lopez-Paz, and Synnaeve. Better & faster large language models via multi-token prediction. In *ICML 2024*, 2024. URL https://arxiv.org/abs/2404.19737. KB excerpt: kb/excerpts/gloeckle2024-mtp.md.

Gu and Dao. Mamba: Linear-time sequence modeling with selective state spaces. arXiv (COLM 2024), 2023. URL https://arxiv.org/abs/2312.00752.

Hoffmann and Borgeaud et al. (DeepMind). Training compute-optimal large language models. arXiv, 2022. URL https://arxiv.org/abs/2203.15556. KB excerpt: kb/excerpts/hoffmann2022-chinchilla.md.

Jiang, Sablayrolles, Mensch, and et al. Mistral 7b. arXiv (Mistral AI Tech Report), 2023. URL https://arxiv.org/abs/2310.06825.

Kudo and Richardson. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *EMNLP 2018 demos*, 2018. URL https://arxiv.org/abs/1808.06226. KB excerpt: kb/excerpts/kudo2018-sentencepiece.md.

Kwon, Li, Zhuang, Sheng, Zheng, Yu, Gonzalez, Zhang, and Stoica. Efficient memory management for large language model serving with pagedattention. SOSP 2023, 2023. URL https://arxiv.org/abs/2309.06180. KB excerpt: kb/excerpts/kwon2023.md.

Leviathan, Kalman, and Matias. Fast inference from transformers via speculative decoding. In *ICML 2023*, 2023. URL https://arxiv.org/abs/2211.17192. KB excerpt: kb/excerpts/leviathan2023.md.

Li, Wei, Zhang, and Zhang. Eagle: Speculative sampling requires rethinking feature uncertainty. In *ICML 2024*, 2024. URL https://arxiv.org/abs/2401.15077. KB excerpt: kb/excerpts/li2024-eagle.md.

Liu, Li, Wu, and Lee. Visual instruction tuning. In *NeurIPS*, 2023. URL https://arxiv.org/abs/2304.08485. KB excerpt: kb/excerpts/llava2023.md.

Press and Wolf. Using the output embedding to improve language models. In *EACL 2017*, 2017. URL https://arxiv.org/abs/1608.05859. KB excerpt: kb/excerpts/press2017-tied-embed.md.

Press, Smith, and Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *ICLR 2022*, 2022. URL https://arxiv.org/abs/2108.12409.

Radford, Wu, Child, Luan, Amodei, and Sutskever. Language models are unsupervised multitask learners. OpenAI tech report, 2019. URL https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf. KB excerpt: kb/excerpts/radford2019-gpt2.md.

Raffel, Shazeer, Roberts, and et al. Exploring the limits of transfer learning with a unified text-to-text transformer. JMLR, 2019. URL https://arxiv.org/abs/1910.10683.

Sennrich, Haddow, and Birch. Neural machine translation of rare words with subword units. In *ACL*, 2016. URL https://arxiv.org/abs/1508.07909. KB excerpt: kb/excerpts/sennrich2016.md.

Shah, Bikshandi, Zhang, Thakkar, Ramani, and Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. In *NeurIPS*, 2024. URL https://arxiv.org/abs/2407.08608.

Shazeer. Fast transformer decoding: One write-head is all you need. arXiv, 2019. URL https://arxiv.org/abs/1911.02150. KB excerpt: kb/excerpts/shazeer2019.md.

Shazeer. Glu variants improve transformer. arXiv, 2020. URL https://arxiv.org/abs/2002.05202. KB excerpt: kb/excerpts/shazeer2020.md.

Su, Lu, Pan, Wen, and Liu. Roformer: Enhanced transformer with rotary position embedding. arXiv, 2021. URL https://arxiv.org/abs/2104.09864. KB excerpt: kb/excerpts/su2021.md.

AI2 OLMo Team. Olmo 2 furious. COLM 2025, 2025. URL https://arxiv.org/abs/2501.00656.

Gemma Team and Google DeepMind. Gemma 3 technical report. arXiv (DeepMind Tech Report), 2025. URL https://arxiv.org/abs/2503.19786.

Qwen Team and Alibaba. Qwen3 technical report. arXiv (Qwen Tech Report), 2025. URL https://arxiv.org/abs/2505.09388.

Touvron, Lavril, Izacard, and et al. Llama: Open and efficient foundation language models. Meta Tech Report, 2023. URL https://arxiv.org/abs/2302.13971.

various. Internvl3: Exploring advanced training and test-time recipes. arXiv, 2025a. URL https://arxiv.org/abs/2504.10479.

various. Longrope2: Near-lossless llm context window scaling. arXiv, 2025b. URL https://arxiv.org/abs/2502.20082.

various. Peri-ln: Revisiting normalization layer in the transformer architecture. In *ICML 2025*, 2025c. URL https://arxiv.org/abs/2502.02732.

Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser, and Polosukhin. Attention is all you need. In *NeurIPS*, 2017. URL https://arxiv.org/abs/1706.03762. KB excerpt: kb/excerpts/vaswani2017.md.

Wang, Ma, Dong, Huang, Zhang, and Wei. Deepnet: Scaling transformers to 1,000 layers. arXiv:2203.00555, 2022. URL https://arxiv.org/abs/2203.00555.

Willard and Louf. Efficient guided generation for large language models. arXiv (Outlines library), 2023. URL https://arxiv.org/abs/2307.09702. KB excerpt: kb/excerpts/willard2023-outlines.md.

Xiao, Lin, Seznec, Wu, Demouth, and Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In *ICML 2023*, 2023. URL https://arxiv.org/abs/2211.10438. KB excerpt: kb/excerpts/xiao2023-smoothquant.md.

Xiong, Yang, He, and et al. On layer normalization in the transformer architecture. In *ICML*, 2020. URL https://arxiv.org/abs/2002.04745. KB excerpt: kb/excerpts/xiong2020.md.

Yuan, Gao, Dai, and et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention. arXiv, 2025. URL https://arxiv.org/abs/2502.11089. KB excerpt: kb/excerpts/yuan2025.md.

Zhang and Sennrich. Root mean square layer normalization. In *NeurIPS*, 2019. URL https://arxiv.org/abs/1910.07467. KB excerpt: kb/excerpts/zhang2019.md.

Zheng, Yin, Xie, Sun, Huang, Yu, Cao, Kozyrakis, Stoica, Gonzalez, Barrett, and Sheng. Sglang: Efficient execution of structured language model programs. In *NeurIPS 2024*, 2024. URL https://arxiv.org/abs/2312.07104. KB excerpt: kb/excerpts/zheng2024-sglang.md.

Table 4: Frontier-2026 architectural choices, per axis. Columns: KV-sharing (MHA / GQA-g / MLA), attention implementation (FA2 / FA3 / NSA), FFN family (dense SwiGLU / MoE / fine-grained MoE / shared-expert MoE), normalization (RMSNorm + Pre/Peri-LN), positional encoding (RoPE plus extension), tokenizer, multi-token output (none / MTP), multimodal integration. "ND" = not publicly disclosed at architectural detail. Sources are cited inline in §13.2–§13.3.

Model	KV	Attn impl	FFN	Norm	Pos	Tok	Multi-tok	MM
DeepSeek V3	MLA, d_c=512	FA + custom MLA kernel	MoE E=256, k=8, E_s=1	RMSNorm + Pre-LN	decoupled-RoPE	BPE 129K	MTP, 1 head	text-only
DeepSeek R1	MLA (V3 base)	FA + custom MLA kernel	MoE (V3 base)	RMSNorm + Pre-LN	decoupled-RoPE	BPE 129K	MTP, 1 head	text-only
Llama 4 (Scout / Maverick)	GQA-8	FA3	dense↔MoE alternating; $E\in\{16, 128\}$, E_s=1, k=1	RMSNorm + Pre-LN	iRoPE (3 RoPE : 1 NoPE) + temperature	SentencePiece BPE	none disclosed	native multimodal
Mistral Large 2	GQA-8	FA2 / FA3	dense SwiGLU	RMSNorm + Pre-LN	RoPE + position-IDs ext.	tekken (BPE)	none disclosed	text-only (Pixtral variant: native MM)
Gemma 3 27B	GQA + 5 local : 1 global (window 1024)	FA2	dense SwiGLU	RMSNorm + Peri-LN + QK-Norm	RoPE + interleaved windows	SentencePiece BPE	none disclosed	native MM (4B+ Vision)
Qwen 2.5 / Qwen 2.5-VL	GQA	FA2 / FA3	dense SwiGLU	RMSNorm + Pre-LN	RoPE + YaRN	BPE 152K	none disclosed	projector + LLM (VL)
Qwen 3 235B-A22B	GQA	FA3	MoE E=128, k=8, E_s=0, global-batch balancing	RMSNorm + Pre-LN	RoPE	BPE 152K	none disclosed	text-only (separate MM line)
Claude 3.7-class	ND	ND	ND	ND	ND	proprietary BPE	ND	native MM
GPT-class (o-/4o/5)	ND	ND	ND	ND	ND	tiktoken-family	ND	native MM (4o, 5)
Gemini 2.5	ND	ND	ND	ND	ND	SentencePiece	ND	native MM